Radboud University Nijmegen



Faculty of Science

From Local to Global Constraints: An Empirical Study of Phase Transitions in CSPs

Thesis BSc Computing Science

Author:
Alexandru Gavajuc

Supervisor:
prof. dr. J.H. Geuvers

Second reader:
dr. J.S.L. Junges

June 2026

Contents

1 Introduction **4**
- 1.1 Phase Transitions and Constraint Arities . . . 5
- 1.2 Problem Statement and Research Question . . . 5
- 1.3 Thesis Overview . . . 5

2 Complexity Theory and Phase Transition Phenomena **7**
- 2.1 Preliminaries on Mathematical and Computation Concepts . . . 7
  - 2.1.1 Mathematical Notation . . . 7
  - 2.1.2 Computational Complexity . . . 8
- 2.2 The Constraint Satisfaction Problem . . . 9
  - 2.2.1 The CSP parameterized by the constraint language . . . 9
  - 2.2.2 Constraint Arity, Local and Global Ensembles . . . 10
- 2.3 Random Constraint Ensembles . . . 12
  - 2.3.1 The Random k-SAT Model . . . 12
  - 2.3.2 The Random ITE Tree Model . . . 13
- 2.4 Phase Transitions and Sharp Thresholds . . . 15
  - 2.4.1 Monotone Properties and Thresholds . . . 16
  - 2.4.2 Sharp versus Coarse Transitions . . . 16
  - 2.4.3 The Satisfiability Threshold in Random k-SAT . . . 17
  - 2.4.4 The Variable Influence (Threshold) in Random ITE . . . 17

3 Theoretical methods and findings **19**
- 3.1 The First Moment Method (1MM): An Upper Bound on the Threshold 19
- 3.2 The Second Moment Method (2MM): Attempt at a Lower Bound . . . 21
- 3.3 Restricting the Solution Space: NAE k-SAT . . . 26
- 3.4 Asymptotic Evaluation and the Lower Bound . . . 31

4 Empirical Analysis of Random k-SAT **34**
- 4.1 Background and Prerequisites . . . 34
  - 4.1.1 The DIMACS Encoding . . . 35
  - 4.1.2 SAT Solvers . . . 35
- 4.2 The Phase Transition as an Empirical Feature . . . 35
  - 4.2.1 Experimentation with Random 3-SAT . . . 36
  - 4.2.2 Results and Interpretation . . . 37
- 4.3 Inherent Hardness and Inherent Easiness: the Random 2-SAT Counterexample . . . 41
  - 4.3.1 Experimental Setup . . . 41
  - 4.3.2 Results and Interpretation . . . 41
- 4.4 The Geometry of the Solution Space . . . 43

4.4.1 The Solution Space, the Energy Landscape and the Structural Phase Transitions . . . 43
4.4.2 Experimental Setup: Two-Tiered Local Sampling . . . 46
4.4.3 Results and Interpretation: The Algorithmic Barrier . . . 48

5 Global Constraints and the ITE Tree Ensemble 53
5.1 Investigating for a Satisfiability Threshold . . . 54
5.1.1 Experiment Setup . . . 54
5.1.2 Results and Interpretation . . . 56
5.2 The Contingent Behavior of the ITE Ensemble . . . 61
5.2.1 Experiment Setup . . . 61
5.2.2 Results and Interpretation . . . 62
5.3 The Variable Influence Phase Transition . . . 66
5.3.1 Experiment Setup . . . 66
5.3.2 Results and Interpretation . . . 67

6 Discussion 74
6.1 Related Work . . . 74
6.2 Interpretation of findings . . . 74
6.3 Limitations . . . 75
6.4 Further Work . . . 75

7 Conclusion 76

A Supplementary Materials 77
A.1 Source Code . . . 77
A.2 Additional Figures . . . 77

ABSTRACT

Phase transitions have been an active field of study in statistical physics and complexity theory for the past few decades. This phenomenon has garnered significant attention due to the peculiar connection between transition thresholds and peaks in computational hardness, providing insight into how a problem's solution space is structured.

In this thesis, we investigate random Constraint Satisfaction Problems (CSPs) by comparing the macroscopic behaviors of local versus global constraints. We first explore the well-known random κ-SAT model, outlining the theoretical bounds of its sharp satisfiability threshold and empirically visualizing the clustering of its solution space. We then compare this with a novel global CSP ensemble based on randomly generated If-Then-Else (ITE) trees. By applying finite-size scaling, we provide evidence that the ITE ensemble exhibits a continuous phase transition in variable influence.

Keywords: Complexity phase transition, Constraint Satisfaction Problem, Solution space, Statistical Physics, Individual Variable Influence

Chapter 1

Introduction

In computational complexity theory, the traditional classification of algorithms relies on worst-case complexity analysis. While this provides asymptotic bounds on algorithmic performance, it often fails to capture how algorithms perform in practice. This motivates the study of *average-case* or *typical-case* complexity, which investigates the resource requirements of algorithms over randomly generated inputs.

A well-known family of problems used to study these typical cases is that of *constraint satisfaction problems* (CSPs). Such problems consist of some set of variables and a set of constraints. The set of variables can have different *configurations* or *states*. To solve the constraint satisfaction problem, the configuration of the variable set must satisfy the set of constraints. The interest in such a general problem family comes from the fact that many well-known classes of problems can be generalized under the CSP formulation [1], and any advances in the generalized class are easily translated to more specific areas of study. To demonstrate this valuable abstraction property of the CSP class, we can think of various problems and puzzles which are all included in the constraint satisfaction family:

- Sudoku Puzzle: the goal is to fill a grid of numbers (with the cells as the set of variables and numbers as the states) such that the rows, columns and sub-grids all contain distinct numbers (the set of constraints).

- Map Coloring Problem: given a map of countries and a set of colors (variables and states, respectively), can we color the map such that no two bordering countries are of the same color (constraints)?

- Scheduling problems: can we plan employee shifts or schedule flights such that the tasks assigned follow time limits and availability rules?

Due to their vast problem coverage, CSPs have been extensively studied for many real-world applications, especially in AI research [1]. In the context of complexity theory, the usefulness of the CSP formulation has been firmly established, since every decision problem is polynomial-time equivalent to a CSP with an infinite template [2].

1.1 Phase Transitions and Constraint Arities

The study of typical-case complexity fundamentally transformed when researchers discovered a profound connection between computer science and statistical mechanics. It was observed that many random combinatorial problems undergo macroscopic *phase transitions*, which are abrupt, large changes in their global properties driven by tiny changes in a control parameter [3, 4]. The most striking realization was the direct correlation between these structural phase transitions and peaks in algorithmic hardness [5].

Historically, the intersection of statistical physics and complexity theory has focused heavily on *local* CSP ensembles, most notably the random Boolean Satisfiability (K-SAT) problem. In random K-SAT, constraints are applied uniformly and independently across small, fixed tuples of variables. The application of probabilistic techniques, such as the second moment method or the statistical physics finite-size scaling methodology, has successfully proven that these local constraints yield sharp satisfiability thresholds and cause the geometry of the solution space to radically cluster and shatter [6, 7, 8].

However, many real-world computational structures are not uniformly flat, in the local sense that random K-SAT is. Hierarchical systems, such as Binary Decision Diagrams (BDDs), rely on *global* constraints where the evaluation of one variable results in major changes in the entire structure. While the macroscopic scaling of local CSPs is rigorously understood, the application of statistical physics techniques to randomly generated, global constraint structures remains significantly less explored.

1.2 Problem Statement and Research Question

While the macroscopic properties of local constraints are extensively documented, the generalization of these phenomena to global, hierarchical structures remains largely unexplored. Specifically, it is not currently known whether the structural scaling and macroscopic phase transitions observed in standard local CSPs generalize to random ensemble logic gate circuits.

To address this gap, this thesis investigates how the architectural scope of a constraint (local versus global) derives its macroscopic structural properties. Specifically, we apply the experimental and theoretical frameworks of probabilistic analysis and statistical physics to both a well-known local CSP and a novel global CSP to answer the following research question:

What macroscopic threshold behaviors, if any, emerge from analyzing a global CSP ensemble, specifically random If-Then-Else (ITE) trees, using the finite-size scaling techniques traditionally applied to local CSPs?

1.3 Thesis Overview

To answer this question, we first formalize important mathematical notions and complexity theory concepts, as well as formally define the phase transition phenomena in relation to CSPs. This, together with introducing the two problems we will be focusing on in the remainder of the thesis, will be laid out in Chapter 2. In Chapter 3, we address part of the research question by reconstructing standard analytical methods for bounding the phase transition thresholds in local CSPs. Specifically, we focus on the

random κ-SAT problem and apply the probabilistic method of moments, as previously done by Achlioptas et al. [8, 9]. Chapter 4 bridges our theoretical methodology with an empirical analysis, focused on visualizing the well-known sharp threshold phenomenon and its apparent connection to algorithmic hardness, which we will address in detail. Additionally, we provide a visualization of the clustering phase transition for random 3-SAT, giving insight on the deeper correlation between hardness and the structure of typical formulas. To address the research question fully, Chapter 5 shifts the analysis to the random ITE problem, our global CSP ensemble. We empirically investigate this ensemble's satisfiability property and highlight how its global constraint structure fundamentally changes macroscopic properties. We apply a finite-size scaling analysis to discover that, while the problem lacks a satisfiability threshold, it exhibits a continuous phase transition in variable influence. In Chapter 6 and Chapter 7, we gather our findings, interpret and compare the results of our investigation. Furthermore, we discuss the limitations of our methodology and propose new, related directions for future research.

Chapter 2

Complexity Theory and Phase Transition Phenomena

We present mathematical definitions and concepts which will be used in later chapters, after which we define the notion of a constraint satisfaction problem. Next, we introduce the two random constraint ensembles which will be analyzed in this thesis. Finally, we discuss the notion of the phase transition phenomenon and present the relevant properties of thresholds.

2.1 Preliminaries on Mathematical and Computation Concepts

Before formalizing the specific models investigated in this thesis, we present the following fundamental mathematical notation and complexity theory concepts which we will use throughout this work.

2.1.1 Mathematical Notation

In our probabilistic analyses (Chapter 3), we frequently need to map boolean events to real-valued random variables. We achieve this using the *indicator* function:

Definition 2.1.1. For any event A, the indicator function, denoted by $\mathbb{1}_{\{A\}}$, is defined as:

$$\mathbb{1}_{\{A\}} = \begin{cases} 1 & \text{if } A \text{ holds} \\ 0 & \text{if } A \text{ does not hold} \end{cases} \tag{2.1}$$

This formulation of the indicator function conveniently allows us to use it in sums over discrete sequences of events, which will be useful later on. For example, if we want to find the expected number of heads after tossing 10 fair coins, we can define the event A_i for the i-th coin toss resulting in a head, which gives us $\mathbb{E}\left[\mathbb{1}_{\{A_i\}}\right] = 1/2$. Using linearity of expectation, we find the indicator function's usefulness in simplifying sums: $\mathbb{E}\left[\mathbb{1}_{\{A_1\}} + \cdots + \mathbb{1}_{\{A_{10}\}}\right] = \mathbb{E}\left[\mathbb{1}_{\{A_1\}}\right] + \cdots + \mathbb{E}\left[\mathbb{1}_{\{A_{10}\}}\right] = 5$.

Furthermore, to find the limiting behavior of the models we will analyze, as well as their algorithmic scaling, we use standard Big-O asymptotic notation (cf. [10]):

Definition 2.1.2. Let $f(n)$, $g(n)$ be two non-negative functions defined for all positive integers. We write that

$$f(n) = \mathcal{O}(g(n))$$

if there exist positive constants c and n_0 such that

$$f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0$$

Big-O notation establishes the worst-case complexity of $f(n)$, in terms of $g(n)$. It tells us that $g(n)$ acts as an upper bound for $f(n)$, which may or may not be asymptotically tight. In the context of this thesis, the notation is important for understanding the time complexity of algorithms; for example, an algorithm requiring $3n^2 + 2n + 4$ steps runs in time $\mathcal{O}(n^2)$ (but also in time $\mathcal{O}(n^3)$ or $\mathcal{O}(n^{10})$).

Additionally, in our probabilistic analysis, we will encounter functions that grow or shrink at a polynomial rate, the exact degree of which might be irrelevant in the given context. In such cases, we employ the poly(n) notation.

Definition 2.1.3. We use poly(n) to denote a function that is bounded by a polynomial in n. That is, $f(n) = \text{poly}(n)$ if $f(n) = \mathcal{O}(n^c)$ for some constant $c > 0$.

In Chapter 4, we will need to formalize the distance between two distinct assignments (or vectors) in space. We use the Hamming distance for this purpose:

Definition 2.1.4. Let $\sigma, \tau \in \{0,1\}^n$ be two boolean vectors (or assignments) of equal length n. The *Hamming distance* between σ and τ, denoted by $d_H(\sigma, \tau)$, is defined as the number of positions at which the corresponding bits are different. Using our indicator function (Definition 2.1.1), this can be expressed as:

$$d_H(\sigma, \tau) = \sum_{i=1}^{n} \mathbb{1}_{\{\sigma_i \neq \tau_i\}} \tag{2.2}$$

The Hamming distance thus measures the number of bit flips required to transform one vector into another. For example, the assignments $\sigma = (0,1,1,0)$ and $\tau = (0,1,0,1)$ differ in their last two bits, giving a Hamming distance of $d_H(\sigma, \tau) = 2$.

2.1.2 Computational Complexity

In computational complexity theory, we often analyze *decision problems*, which can be formulated as a yes-or-no question, and it is of interest to classify these problems based on the algorithmic resources required to solve them, as the input increases. Such an analysis reveals that, while some decision problems can be quickly solved by algorithms, others are simply much harder to solve. These different kinds of problems are categorized in *complexity classes*.

Definition 2.1.5. We present here two important such classes:

- The complexity class **P** (Polynomial Time) contains all decision problems that can be solved by a deterministic algorithm in polynomial time. That is, the worst-case

running time for a solution is poly(n) (i.e., bounded by $\mathcal{O}(n^c)$ for some constant $c > 0$).

- The complexity class **NP** (Non-deterministic Polynomial Time) contains all decision problems for which every solution (a "yes" answer) has a corresponding certificate (or proof) of polynomial size in terms of the input, which can be checked by a deterministic algorithm in polynomial time.

While any problem in **P** is also in **NP**, it is widely believed that **P** $\neq$ **NP**. The hardest problems in the class **NP** are called **NP-complete**. A problem is **NP-complete** if it is in **NP** and every other problem in **NP** can be reduced (translated) to it in polynomial time.

2.2 The Constraint Satisfaction Problem

The family of Constraint Satisfaction Problems (CSPs) is well-known in computational complexity theory, and its problems are extensively studied in Artificial Intelligence and Operations research [11, 1]. Generally, a CSP is a problem in which the solution must meet a number of constraints. CSPs often serve as a model for many decision problems, optimization problems and counting problems, both in the **P** and **NP** classes. For example, SAT is an **NP-complete** problem where we ask whether a boolean formula can be satisfied; this foundational problem, together with many others, is a particular form of the more general CSP.

2.2.1 The CSP parameterized by the constraint language

We follow here the standard definition of a CSP, in terms of *languages* and *relations* [12, 13, 14]:

Definition 2.2.1. Let D be a finite set called the *domain*. For any positive integer $k \in \mathbb{N}^+$, a subset $R \subseteq D^k$ is called a k-ary relation over D. The set of all finitary relations over D is denoted by R_D, defined as the infinite union:

$$R_D = \bigcup_{k=1}^{\infty} \mathcal{P}(D^k) \tag{2.3}$$

A *constraint language* Γ over D is any subset of R_D ($\Gamma \subseteq R_D$).

In general, a constraint language Γ is typically assumed to only contain *local* constraints, those being relations of a fixed, bounded arity k (e.g. $k = 3$ in 3-SAT). Limiting the language to only having these fixed-arity local constraints proves to be very useful when analyzing problems with probabilistic methods (as we will see in Chapter 3). However, this limitation does not allow for relations across an arbitrary, or dynamic number of variables. Such relations give rise to other kinds of problems whose languages contain *global* constraints. This is very much of interest to us, and we will be looking at what the differences between these kinds of relations entail in our analyses in later chapters.

Definition 2.2.2. For a domain D and some constraint language Γ over D, the constraint satisfaction problem **CSP**(Γ) is the decision problem with the following:
Instance: A triple $I = (V, D, \mathcal{C})$, where:

- $V = \{v_1, \ldots, v_n\}$ is the fixed set of n variables

- $\mathcal{C}$ is the set of constraints, with each constraint $C_i \in \mathcal{C}$ being a pair $\langle \mathbf{s}_i, R_i \rangle$ of the form:
  - $\mathbf{s}_i = (v_1, \ldots, v_k) \in V^k$ is a tuple of variables from V of size k, called the *constraint scope*
  - $R_i \in \Gamma$ is a k-ary relation chosen from the constraint language Γ called the *constraint relation*

Question: Is there a function $\varphi : V \to D$, such that, for every constraint $\langle \mathbf{s}_i, R_i \rangle \in \mathcal{C}$, we have that $\varphi(\mathbf{s}_i) = (\varphi(v_1), \ldots, \varphi(v_k)) \in R_i$? This φ, if it exists, we will call a *solution*.

2.2.2 Constraint Arity, Local and Global Ensembles

As mentioned above, the distinction between local and global constraints is quite important for the analysis we will be doing in later chapters. This is especially the case when evaluating the thermodynamic limit, i.e. where the number of variables n from Definition 2.2.2 goes to infinity. With this in mind, it is important to classify and distinguish the two types of CSP ensembles, which we will do in terms of how a sequence of problem instances behaves as the problem size grows to infinity (i.e. $|V| = n \to \infty$):

Definition 2.2.3. Let $\mathcal{I}_n = (V_n, D, \mathcal{C}_n)$ be a sequence of **CSP**(Γ) instances as defined in Definition 2.2.2, with Γ a constraint language over D, where the number of variables $n = |V_n|$. Let the arity k_i of $C_i = \langle \mathbf{s}_i, R_i \rangle$ be defined as the size of its scope, $k_i = |\mathbf{s}_i|$. Then we have the following two ensembles:
The locally constrained ensemble: $\mathcal{I}_n$ is locally constrained if there exists a finite constant k_{max} such that, for all n, the maximum arity of any constraint is strictly bounded by k_{max}. That is,

$$\sup_{n \in \mathbb{N}} \left(\max_{C_i \in \mathcal{C}_n} k_i \right) \leq k_{max} < \infty \tag{2.4}$$

The globally constrained ensemble: $\mathcal{I}_n$ is globally constrained if the maximum constraint arity grows as a function of the instance size n. That is,

$$\lim_{n \to \infty} \left(\max_{C_i \in \mathcal{C}_n} k_i \right) = \infty \tag{2.5}$$

In later sections of this thesis, when the notion of a "local CSP" is mentioned, we refer to a locally constrained CSP ensemble, and by "global CSP" we mean a globally constrained CSP ensemble. This distinction is important, since we do not refer to how one might approach solving such problems. Local approaches (such as local search methods) and global ones (such as backtracking methods) are important in later chapters where we look at experimental works, however, it will be explicitly mentioned, if needed, when the solving approaches are taken into account.

We give some examples below to show how the CSP model is used in different well-known problems, and classify them according to Definition 2.2.3:

Example 1. A 3-SAT problem instance involves taking a propositional logic formula consisting of a conjunction of clauses of three literals. The question asked is whether there exists an assignment of the variables such that the formula evaluates to true. Let $\mathcal{F} = F_1 \wedge \cdots \wedge F_m$ be such a formula, with each F_i being a clause. We then take the set

$\{0,1\}$ as the domain D, since we only evaluate boolean values in this problem. The CSP instance is thus $I_{\mathcal{F}} = (V, \{0,1\}, \mathcal{C})$, where V is the set of variables in $\mathcal{F}$ (with $n = |V|$ the number of variables) and $\mathcal{C}$ is the set of constraints of the form $\langle \mathbf{s}_i, R_i \rangle$ where:

- $\mathbf{s}_i = (x_1^i, x_2^i, x_3^i)$, with x_1^i, x_2^i, x_3^i the variables appearing in F_i
- $R_i = \{0,1\}^3 \setminus \{(a_1, a_2, a_3)\}$, with $a_j = 1$ if x_j^i is negated in F_i and $a_j = 0$ otherwise. Since each clause only contains disjunctions, there is only one assignment for the three variables to make F_i false, and R_i contains the rest of the assignments.

Defining the constraint language $\Gamma_{3\text{-SAT}}$ over $\{0,1\}$ which contains every relation expressible by 3-clauses, we can now express every 3-SAT instance by a $\mathbf{CSP}(\Gamma_{3\text{-SAT}})$ instance. Notice that the arity of every constraint is exactly 3 in this problem. As the number of variables $n \to \infty$, this arity remains strictly bounded by $k_{max} = 3$. Therefore, sequences of 3-SAT instances form a locally constrained ensemble.

Example 2. We now look at a K-COLORING problem instance, in which we are given a graph $G = (V, E)$ and asked whether we can label the vertices of G in k colors such that no two adjacent vertices end up with the same color. The domain is now the set of colors Q, with size k. We will use the inequality relation $\neq_Q$ over Q, defined by:

$$\neq_Q = \{(a, b) \in Q \times Q \mid a \neq b\} \tag{2.6}$$

The instance is $I_G = (V, Q, \mathcal{C})$ and the constraints follow the form:

- $\mathbf{s}_i = (v_1^i, v_2^i)$, where $e_i = (v_1^i, v_2^i)$ is an edge in G
- $R_i = \neq_Q$ (every relation is the same)

We can then see that an instance of K-COLORING corresponds to $\mathbf{CSP}(\{\neq_Q\})$. Here, the arity of every constraint (edge) is exactly 2. As the size of the graph grows to infinity, the constraint arity remains bounded by $k_{max} = 2$. Thus, K-COLORING is a locally constrained ensemble.

Example 3. The classic N-QUEENS problem requires placing N queens on an $N \times N$ chessboard such that no two queens attack each other. We can model this by defining N variables representing the rows of the board, and a domain representing the column placements. The CSP instance is then $I_N = (V, D, \mathcal{C})$, with $V = \{x_1, \ldots, x_N\}$ the set of variables, as the rows, and $D = \{1, \ldots, N\}$ the domain, as the columns. The set of constraints $\mathcal{C} = \{C_1\}$ contains a *single* global constraint $C_1 = \langle \mathbf{s}_1, R_1 \rangle$ that enforces all board rules simultaneously [1]:

- $\mathbf{s}_1 = (x_1, x_2, \ldots, x_N)$ is the scope, which includes all N variables.
- $R_1 \subset D^N$ is the N-ary relation containing all valid board configurations. In the Constraint Programming literature, this rule is known as the `AllDifferent` global constraint [15]. It is the set of all N-tuples $(a_1, \ldots, a_N)$ such that for all $i \neq j$, $a_i \neq a_j$ and $|a_i - a_j| \neq |i - j|$. The $a_i \neq a_j$ condition enforces that no columns are shared (two queens on different rows cannot be on the same column), while the $|a_i - a_j| \neq |i - j|$ condition enforces that no diagonals are shared (if the difference in rows is equal to the difference in columns, we are on the same diagonal).

It may seem natural to model the N-QUEENS problem with N^2 variables (one for every

square on the board) and with $D = \{0, 1\}$, to choose whether a queen is on a square or not. However, this formulation results in an excessive number of constraints just to enforce only one queen per row. Instead, the standard approach, taken above, exploits the universal rule of the puzzle by making each variable x_i represent the column index where the queen in row i is placed [15]. The options to choose from are exactly the columns $\{1, \ldots, N\}$, which are represented by the domain D. In this formulation, the entire problem is captured by a single constraint whose arity is exactly $k_1 = N$. Because the arity of the constraint scales linearly with the problem size (the board length N in this case), the maximum arity in the limit is given by $\lim_{N\to\infty} (\max_{C_i \in \mathcal{C}_N} k_i) = \infty$. This cannot be bounded by a constant, therefore, the sequence of N-QUEENS instances is a globally constrained ensemble.

2.3 Random Constraint Ensembles

So far, we have only considered specific, deterministic instances of CSPs. However, as mentioned in the previous chapter, in the study of complexity phase transitions and average-case complexity, we often want to define probability distributions over these problems. We do this by introducing a generative random model (or generator) which relies on a *control parameter* to form the random problem instance.

By varying this control parameter, we can observe changes in the macroscopic properties of the generated instances. This is a central idea behind phase transition phenomena, which we will talk about in later sections. Because randomness can be introduced into a CSP in fundamentally different, large number of ways, there is no single universal definition for a "Random CSP". Instead, we contrast two different random ensembles: a locally constrained model driven by constraint density, and a globally constrained model driven by (structural) geometry.

2.3.1 The Random k-SAT Model

Probably the most well-studied random CSP ensemble is the random K-SAT model [16]. The randomness in this ensemble lies in the independent sampling of many local, bounded-arity constraints. The control parameter is usually called the *clause density* α, which is used to generate a probability distribution under this model. We first look at the logical formulation of random K-SAT, in which we restrict ourselves to only Conjunctive Normal Form (CNF) formulas, in which any boolean formula $\mathcal{F}$ takes the form of a conjunction of clauses, and each clause F_i is a disjunction of exactly k literals (with $k \geq 2$). For a fixed set of n boolean variables $V = \{x_1, \ldots, x_n\}$, we generate a formula by uniformly and independently selecting $m = \alpha n$ clauses, with $\alpha > 0$. Thus, a generated random K-SAT formula $\mathcal{F}$ takes the form:

$$\mathcal{F} = \bigwedge_{i=1}^{m} F_i \tag{2.7}$$

where each clause $F_i = (l_1^i \vee \cdots \vee l_k^i)$ contains k independently and uniformly selected literals, without replacement. A literal l_i is either the variable x_i or its negation, and the choice is again uniform (in this specific case, this means a negation of each variable with probability 1/2).

This generative model we described above can be further formalized as a random CSP ensemble. In line with Example 1, we notice that a disjunction of k literals forbids

exactly one specific assignment: precisely the one which makes all k literals false. Thus, we arrive at the following random CSP ensemble definition:

Definition 2.3.1. Let $V = \{x_1, \ldots, x_n\}$ be a set of n boolean variables, let $D = \{0, 1\}$ be the domain and let $k \geq 2$ be a fixed integer. For a density parameter $\alpha > 0$, a random K-SAT instance is a $\mathbf{CSP}(\Gamma_{\text{K-SAT}})$ instance generated by uniformly and independently drawing $m = \alpha n$ constraints. For each of these constraints $C_i = \langle \mathbf{s}_i, R_i \rangle$ (with $1 \leq i \leq m$):

- $\mathbf{s}_i = (x_1^i, \ldots, x_k^i) \in V^k$ is a tuple of k variables chosen uniformly at random without replacement from V.
- $R_i \in \Gamma_{\text{K-SAT}}$ is chosen uniformly at random from the language. This choice is equivalent to setting $R_i = D^k \setminus \{\mathbf{a}_i\}$, where $\mathbf{a}_i = (a_1^i, a_2^i, \ldots, a_k^i) \in D^k$ is a single k-tuple of boolean values chosen uniformly at random to represent the sole assignment that violates the constraint.

Corresponding to Example 1, we take the constraint language $\Gamma_{\text{K-SAT}}$ to be the set of all relations over D expressible by k-clauses.

As established in the previous section, because the constraint arity k is strictly bounded as $n \to \infty$, random K-SAT forms a locally constrained ensemble. This can also be seen by looking at the logical formulation given above, as each new conjunction of a clause incrementally restricts the set of satisfying assignments by only adding a rule with respect to a small subset of the variable set.

Similarly to [8], we will from now on denote a random K-SAT instance generated under this model as $F_k(n, m = \alpha n)$, where k, n and m assume the same definitions as above, and the generated instance takes the form given in Equation 2.7 (the notion of a random K-SAT instance will be used interchangeably with this notation).

Remark 2.3.1. Notice the equivalence between the two methods used to describe the ensemble, which allows us to clearly characterize the constraint language. Recalling from Definition 2.2.1 that $\Gamma_{\text{K-SAT}} \subseteq R_D$, we can see that $\Gamma_{\text{K-SAT}}$ is exactly the set of all k-ary relations that exclude a single tuple. That is:

$$\Gamma_{\text{K-SAT}} = \{D^k \setminus \{\mathbf{v}\} \mid \mathbf{v} \in D^k\}$$

Because there are 2^k possible k-tuples in D^k, the language contains exactly $\binom{2^k}{2^k-1} = 2^k$ distinct relations in which one k-tuple is always omitted. The tuple $\mathbf{a}_i$ forbidden by R_i is the CSP model equivalent of the unique boolean assignment that falsifies the clause F_i.

2.3.2 The Random ITE Tree Model

In contrast to the local ensemble we presented, we introduce a globally constrained model based on the structure of a ternary boolean tree. Just as for the random K-SAT ensemble, the generative random model takes two parameters: the tree height h (which is universal, since the tree is regular) and a structural scaling parameter β.

This control parameter β is used to determine the number of distinct variables $m = m(\beta, h)$ (i.e. the variable pool size). The role of β then corresponds to that of the clause density α of the random K-SAT model, since both of them describe a scaling

relation. However, since a universal theoretical scaling relation for this exact model is not well-defined in literature, we will keep this relation abstract. In the later chapters of this thesis, we investigate different scalings (such as $m = \beta \cdot 3^h$ and $m = \beta \cdot 1.5^h$) and give experimental evidence to suggest that the exact form of this parameter is crucial to observing macroscopic changes in the probability distribution of the ensemble.

Since the structure of this model's instances take the form of a tree, we further make a distinction between the internal nodes of the tree and the leaf nodes. Every internal node is an If-Then-Else gate of the form $ITE(c, t, e)$, with c, t and e being subtrees of the same height (or leaves if they correspond to a height $h = 0$). The randomness in this model comes entirely from the uniform assignment of symbols to the leaves of the tree from a fixed leaf alphabet Σ, defined as such, conditional on the inclusion of negation:

$$\Sigma = \begin{cases} V \cup \{0, 1\} & \text{if negation is not allowed} \\ V \cup \{\neg x \mid x \in V\} \cup \{0, 1\} & \text{if negation is allowed} \end{cases} \tag{2.8}$$

where $V = \{x_1, \ldots, x_m\}$ is the pool of m boolean variables. A random tree instance is formed by labeling each of the $n = 3^h$ leaves with a symbol sampled uniformly and independently at random, with replacement, from Σ. Figure 2.1 presents an instance of this model.

As we have done for the $F_k(n, m)$ model, we can further formalize the generator given above as a random CSP ensemble, only now, the ensemble will be globally constrained. To achieve this, we additionally define how a variable assignment evaluates across the tree structure of this model:

Definition 2.3.2. For a given tree height h and a control parameter $\beta > 0$, let $m = m(\beta, h)$ be the size of the variable pool $V = \{x_1, \ldots, x_m\}$. Let the alphabet Σ be defined in the same way as above and let $D = \{0, 1\}$ be the domain. Furthermore, let T be a regular ternary tree of height h whose leaf nodes are labeled by a random function $g : \text{leaves}(T) \to \Sigma$.

For any variable assignment $\mathbf{a} = (a_1, \ldots, a_m) \in D^m$, we define the evaluation of any node v in T, denoted as $f_T(\mathbf{a}, v)$, recursively as follows:

- **Leaf Nodes:** If v is a leaf node, its evaluation is determined directly by its random label $g(v)$ and the assignment $\mathbf{a}$:

$$f_T(\mathbf{a}, v) = \begin{cases} a_j & \text{if } g(v) = x_j \\ 1 - a_j & \text{if } g(v) = \neg x_j \\ g(v) & \text{if } g(v) \in \{0, 1\} \end{cases} \tag{2.9}$$

- **Internal Nodes:** If v is an internal node with three child subtrees representing the condition branch c, the then branch t, and the else branch e, it evaluates to:

$$f_T(\mathbf{a}, v) = \big(f_T(\mathbf{a}, c) \wedge f_T(\mathbf{a}, t)\big) \vee \big(\neg f_T(\mathbf{a}, c) \wedge f_T(\mathbf{a}, e)\big) \tag{2.10}$$

The global evaluation of the whole tree instance is then defined as the evaluation of its root node: $f_T(\mathbf{a}) := f_T(\mathbf{a}, \text{root}(T))$.

The corresponding random CSP instance is defined as $I = (V, D, \mathcal{C})$, where the constraint set $\mathcal{C} = \{C_{ITE}\}$ contains a *single global constraint* $C_{ITE} = \langle \mathbf{s}_{ITE}, R_{ITE} \rangle$, where:

- $\mathbf{s}_{ITE} = (x_1, \ldots, x_m) \in V^m$ is a tuple containing all variables in the pool.
- $R_{ITE} \subseteq D^m$ is the m-ary relation containing all variable assignments for which the global evaluation function f_T returns 1. In other words, R_i is defined as the preimage of 1 under the f_T function:

$$R_{ITE} = \{\mathbf{a} \in D^m \mid f_T(\mathbf{a}) = 1\} \tag{2.11}$$

As we have done for the $F_k(n, m)$ model, for readability purposes, we will denote an instance generated under this globally constrained probability distribution as being drawn from $\mathcal{T}(h, \beta, \Sigma)$, with h, β and Σ assuming the same definitions as above. Because this model includes only a single constraint mapping the entire tree state, whose relation arity scales with the size of the variable pool, $\mathcal{T}(h, \beta, \Sigma)$ forms a globally constrained ensemble.

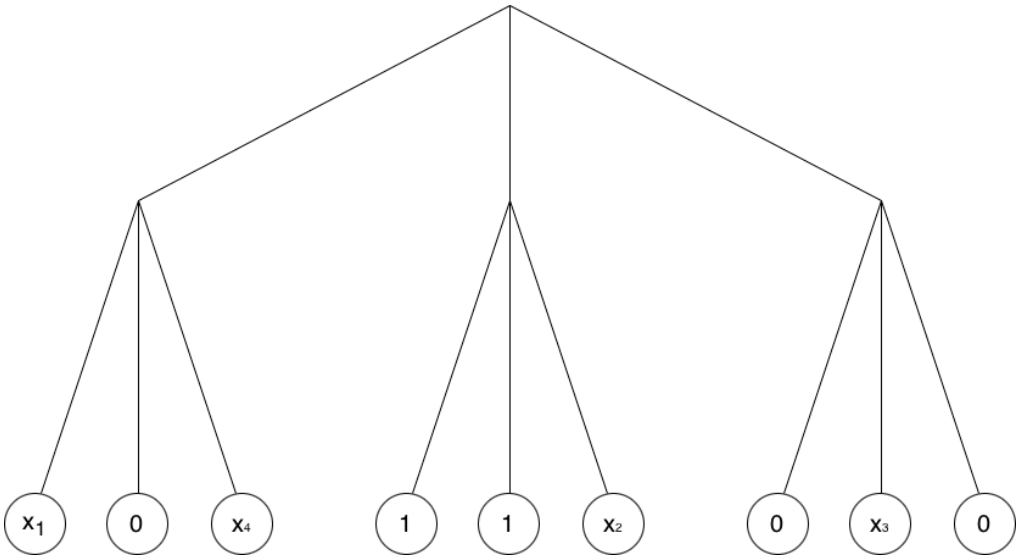


Figure 2.1: A random instance of the ITE tree model, represented as a ternary, regular tree of depth 2. Notice that each leaf node corresponds to a symbol in the current alphabet, while the internal nodes correspond to ITE gates which are evaluated using their three child branches.

2.4 Phase Transitions and Sharp Thresholds

Generally, the phase transition phenomenon can be described as an abrupt, sometimes discrete change in the macroscopic features of a system as a control parameter smoothly crosses a critical value [17]. In statistical physics, this phenomenon has been studied extensively, from transitions of states of matter to the magnetization of spin glasses. In complexity theory, phase transitions are a central focus in the study of random CSPs, giving insights into how the probability distribution (and geometry of the solution space) changes as constraints are tightened. A very successful collaboration between the methods of statistical physicists and complexity theorists has brought to light reliable results and predictions, which we will use in later chapters.

To express how the probability distributions of these random ensembles behave, we first define two standard probabilistic bounds:

Definition 2.4.1. A sequence of events E_n occurs *with high probability* (w.h.p.) if

$$\lim_{n\to\infty} \Pr[E_n] = 1 \tag{2.12}$$

and it occurs *with uniform positive probability* (w.u.p.p.) if

$$\liminf_{n\to\infty} \Pr[E_n] > 0 \tag{2.13}$$

2.4.1 Monotone Properties and Thresholds

To define a phase transition in our context, we introduce the concept of a *monotone property*. Let $\mathcal{P}$ be a property of a CSP instance (for example, "the instance is satisfiable").

Definition 2.4.2. A property $\mathcal{P}$ is *monotone decreasing* if removing a constraint from an instance $I \in \mathcal{P}$ guarantees the resulting instance I' remains in $\mathcal{P}$. Conversely, $\mathcal{P}$ is *monotone increasing* if adding a constraint preserves the property.

If we let x be a control parameter driving the randomness of the ensemble (such as the constraint density α), we define the probability of the property holding as a function of the system size n and the parameter x: $\phi(n, x) := \Pr(\mathcal{I}_n \in \mathcal{P})$.

Definition 2.4.3. Let $\mathcal{I}_n$ be a random sequence of CSP instances parameterized by x. If, for some critical sequence $x_c(n)$ and for every $\epsilon > 0$, it holds that:

$$\lim_{n\to\infty} \phi(n, x) = \begin{cases} 1 & \text{if } x = (1-\epsilon)x_c(n) \\ 0 & \text{if } x = (1+\epsilon)x_c(n) \end{cases} \tag{2.14}$$

then $x_c(n)$ is a *threshold* for the property $\mathcal{P}$, and the system undergoes a phase transition at this critical point.

2.4.2 Sharp versus Coarse Transitions

Not all thresholds behave identically. We can quantify the abruptness of the phase transition by looking at the *transition window*.

Definition 2.4.4. Let $\delta \in (0, 1/2)$ be an arbitrarily small constant. We define the critical bounds $x_-(n, \delta)$ and $x_+(n, \delta)$ such that $\phi(n, x_-(n, \delta)) = 1-\delta$ and $\phi(n, x_+(n, \delta)) = \delta$.

The *transition window width* is then defined as:

$$w(n, \delta) := |x_+(n, \delta) - x_-(n, \delta)| \tag{2.15}$$

If the transition window shrinks to 0 relative to the critical point as the number of variables grows, that is:

$$\lim_{n\to\infty} \frac{w(n, \delta)}{x_c(n)} = 0 \tag{2.16}$$

then we say that the threshold is *sharp*. If the window remains strictly positive as $n \to \infty$, the threshold is *coarse*.

2.4.3 The Satisfiability Threshold in Random k-SAT

For $F_k(n, m)$, the property $\mathcal{P}$ of interest is satisfiability (which is a monotone decreasing property). The control parameter is the clause density $\alpha = m/n$. As α increases, the probability of satisfiability drops from 1 to 0 at some critical point. We will refer to this behavior as a satisfiability transition (or threshold).

In a seminal paper, Friedgut [7] proved the existence of a non-uniform satisfiability threshold, which entails the existence of a sequence $\alpha_k(n)$ around which (as we mentioned) the probability of satisfiability goes from 1 to 0:

Theorem 2.4.5. *For each $k \geq 2$, there exists a sequence $\alpha_k(n)$ such that, for every $\epsilon > 0$,*

$$\lim_{n\to\infty} \Pr[F_k(n, \alpha n) \text{ is satisfiable}] = \begin{cases} 1 & \text{if } \alpha = (1-\epsilon)\alpha_k(n) \\ 0 & \text{if } \alpha = (1+\epsilon)\alpha_k(n) \end{cases} \tag{2.17}$$

Much work has been done towards finding the exact critical point for both $k = 2$ and $k \geq 3$. For $k = 2$, Chvátal and Reed [18] and Goerdt [19] independently proved in 1992 the existence of a sharp transition at $\alpha_2 = 1$. For $k \geq 3$, no exact proof has been developed yet. While Friedgut proved that the transition is sharp around a sequence $\alpha_k(n)$, it remains unproven whether this sequence converges to a constant. This open question led to a popular conjecture known as the *Satisfiability Threshold Conjecture* [8]:

Conjecture 2.4.6. *For all $k \geq 3$, there exists a constant α_k such that*

$$\lim_{n\to\infty} \Pr[F_k(n, \alpha n) \text{ is satisfiable}] = \begin{cases} 1 & \text{if } \alpha < \alpha_k \\ 0 & \text{if } \alpha > \alpha_k \end{cases} \tag{2.18}$$

Although this behavior of the probability space of $F_k(n, m)$ at the thermodynamic limit ($n \to \infty$) was originally proposed as a conjecture, a monumental result by Ding, Sly and Sun [20] formally proved the conjecture for all sufficiently large k, finally verifying the exact critical threshold predicted by statistical physicists. However, for small values of $k = 3$, the conjecture remains open, though experimental evidence heavily points to a critical value $\alpha_3 \approx 4.26$ [21].

2.4.4 The Variable Influence (Threshold) in Random ITE

For the $\mathcal{T}(h, \beta, \Sigma)$ ensemble, understanding its dynamics requires measuring properties other than satisfiability. We are also interested in how individual pieces of information propagate through the recursive structure of the tree instance. We measure this property by observing *boolean variable influence* [22], a standard metric when analyzing boolean functions:

Definition 2.4.7. Let $f_T : \{0,1\}^m \to \{0,1\}$ be the global evaluation function of $\mathcal{T}(h, \beta, \Sigma)$ from Definition 2.3.2, and let $\mathbf{a} = (a_1, \ldots, a_m)$ be a uniformly random boolean assignment from D^m. We denote $\mathbf{a}^{\oplus i}$ as the assignment $\mathbf{a}$ with its i-th bit flipped. That is, $\mathbf{a}^{\oplus i} = (a_1, \ldots, 1 - a_i, \ldots, a_m)$.

The influence of the i-th variable on the boolean function f_T, denoted as $\mathrm{Inf}_i(f_T)$, is defined as the probability over all assignments that flipping the i-th variable changes

the global output of the tree:

$$\mathrm{Inf}_i(f_T) = \Pr_{\mathbf{a}\in\{0,1\}^m}[f_T(\mathbf{a}) \neq f_T(\mathbf{a}^{\oplus i})] \tag{2.19}$$

Because the ensemble is generated from the variable pool V uniformly, the expected influence of any single variable is statistically identical. Therefore, rather than defining a binary property (such as satisfiability), we treat the individual variable influence as a macroscopic *order parameter* of the system. The control parameter of this transition is the structural scaling relation $\beta(m, h)$. In Chapter 5, we provide empirical evidence to show that, as this scaling parameter β crosses a critical threshold, the ensemble undergoes a continuous phase transition. The system shifts from a state of information propagation, where variables heavily influence the global result, to information dilution, where the individual variable flips are absorbed before ever reaching the root.

Chapter 3

Theoretical methods and findings

Much work has been done theoretically in the search for critical densities of phase transitions, both in the complexity theory field (especially for random k-SAT) and statistical physics. [23, 6, 24, 4]. As a result, strong mathematical tools have been used and even developed to approach tighter and tighter bounds for these conjectured points.

A widely used statistical technique in probability distributions is called the *method of moments*, in which the expected values of the random variable and that of the variable squared (e.g. the first and second moment, respectively) are used to determine upper and lower bounds on the critical threshold where the probability changes. In this chapter, we aim to give an intuitive explanation of the method of moments for both bounds on the threshold. For the lower bound, we will provide intuitive reasoning for the methodology used by Achlioptas and Moore in their breakthrough paper [8], as well as more detailed explanations on their specific mathematical tools for investigating the probability space.

3.1 The First Moment Method (1MM): An Upper Bound on the Threshold

We start with the goal of finding an upper bound for the critical phase transition point α_k, where k is the constraint arity. While the first moment method can be formulated for general constraint satisfaction problems, we will be primarily studying $F_k(n, m)$ with $\alpha = m/n$, using the setting for n, m and k which has been used until now. To analyze the probability distribution of $F_k(n, m)$, we define a random variable X to represent the number of satisfying assignments of F:

Definition 3.1.1. Let $\sigma \vDash F$ denote that assignment $\sigma \in \{0,1\}^n$ satisfies F. Then

$$X = |\{\sigma \in \{0,1\}^n \mid \sigma \vDash F\}| \tag{3.1}$$

is the random variable counting the number of satisfying assignments of the boolean formula F.

Note. *This definition is equivalent to summing over the indicators for all the possible assignments, $X = \sum_\sigma \mathbb{1}_{\{\sigma \vDash F\}}$, see Definition 2.1.1.*

Then, we have that, for a random k-SAT instance, F is satisfiable iff $X > 0$. To make this relevant for us, we will relate the expected value of the number of satisfying assignments (or the *first moment* of X) with the probability that we have such assignments. This relation is done using Markov's Inequality:

Lemma 3.1.2. *For a nonnegative function g and a random variable X the following holds:*

$$\Pr[g(X) \geq t] \leq \frac{\mathbb{E}[g(X)]}{t} \tag{3.2}$$

for any $t > 0$.

If X is an integer-valued random variable (such as the number of assignments), after setting $t = 1$ we have:

$$\boxed{\Pr[X > 0] \leq \mathbb{E}[X]} \tag{3.3}$$

which we call the *first moment method* (abbreviated as 1MM).

The inequality can be intuitively understood by realizing that, since X is a non-negative integer, if its average is less than 1, X must be 0 at least sometimes (for a full proof see [25]). As it happens, X is integer-valued for us, as it represents how many assignments satisfy a formula (an integer which cannot be less than 0), thus we can apply the method. Denoting $X_{n,\alpha}$ as the number of satisfying assignments for $F_k(n, \alpha n)$, the motivation for using this method is clear: if $\mathbb{E}[X_{n,\alpha}] \to 0$ as $n \to \infty$, then we know that $\Pr[X_{n,\alpha} > 0] = 0$. In other words, we can find a density after which F is unsatisfiable w.h.p., an upper bound for the critical threshold α_k.

Recalling that F is a conjunction of i clauses C_i (see Equation 2.7), we find the expectation of X to be:

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{\sigma} \mathbb{1}_{\sigma \vDash F}\right] = \sum_{\sigma} \mathbb{E}\left[\prod_{C_i} \mathbb{1}_{\sigma \vDash C_i}\right] = \sum_{\sigma} \prod_{C_i} \mathbb{E}\left[\mathbb{1}_{\sigma \vDash C_i}\right] = 2^n (1 - 2^{-k})^m \tag{3.4}$$

Intuitively, this can be understood as follows: there are 2^n possible assignments for n values. For a single clause C_i with k literals, an assignment will fail to satisfy it if and only if all k literals evaluate to false (recall that C_i only includes disjunctions). Each literal is false with probability $1/2$, so the probability a clause is satisfied is $1 - 2^{-k}$. Choosing m such clauses gives us the probability for an assignment $(1 - 2^{-k})^m$. Then, by linearity of expectation, we get $\mathbb{E}[X] = 2^n (1 - 2^{-k})^m$.

So, using $m = \alpha n$, we have

$$\mathbb{E}[X] = \mathbb{E}[X_{n,\alpha}] = [2(1 - 2^{-k})^{\alpha}]^n \tag{3.5}$$

Notice that, for $\mathbb{E}[X] \to 0$ to hold as $n \to \infty$, we must have that $2(1 - 2^{-k})^{\alpha} < 1$. Solving for α gives us $\ln 2 + \alpha \ln(1 - 2^{-k}) < 0$, or, since $\ln(1 - 2^{-k}) < 0$,

$$\alpha > \frac{-\ln 2}{\ln(1 - 2^{-k})} \tag{3.6}$$

This is the precise bound which the classical application of 1MM produces; it tells us that, for $\alpha > \frac{-\ln 2}{\ln(1-2^{-k})}$, F is unsatisfiable w.h.p., which means that the transition

threshold α_k must be below this value:

$$\boxed{\alpha_k \leq \frac{-\ln 2}{\ln(1-2^{-k})}} \tag{3.7}$$

For large k, the bound is simplified by Taylor expanding the denominator using $\ln(1-x) \approx -x$, for small x (cf. [10]):

$$\alpha_k \leq \frac{-\ln 2}{-2^{-k}} = 2^k \ln 2 \tag{3.8}$$

This approximation is only reasonable as long as k is sufficiently large. For example, for $k = 3$, the 1MM upper bound using Equation 3.7 gives $\alpha_k \leq 5.1909$, while the approximation above gives $\alpha_k \leq 5.545$. Table 3.1 shows how the upper bounds found by applying 1MM compare with currently known upper bounds for α_k (cf. [26]). The latter are bounds which have been derived using more sophisticated approaches, however, the main contribution to the improvement comes from applying the second moment method (2MM) [8], or the replica/cavity method [27].

k	3	4	5	7	10	20
1MM Bound	5.191	10.740	21.832	88.376	709.436	726,817
Known Bound	4.489	10.217	21.330	87.880	708.947	726,817

Table 3.1: Comparison of 1MM and Known Upper Bounds for k-SAT

Notice how the bounds (and critical thresholds) quickly grow as k increases. In fact, the increase is exponential, as the bound we found is proportional to 2^k. Another noteworthy observation is the fact that our 1MM derived bound quickly approaches the best known bounds. This is due to the fact that, as k increases, Markov's Inequality (Equation 3.1.2) becomes a better tool for determining the exact threshold.

3.2 The Second Moment Method (2MM): Attempt at a Lower Bound

We have successfully derived an upper bound on α_k with the classic application of the first moment method. We would like to achieve the same success for a lower bound as well, however, as we shall see, 1MM will not work for this. The direction of Markov's Inequality allows us to find where solutions almost certainly do not exist (w.h.p.), but it cannot show us whether a solution exists. To understand this intuitively, we can think of a distribution that looks like the one in Figure 3.1. The expectation $\mathbb{E}[X]$ at any point is large due to the peak at $x = 3$, however, the probability to find a solution is nearly 0.

Recall $F_k(n, \alpha n)$ and its probability distribution. To find a lower bound on α_k, we need to show where $\Pr[X > 0]$ is bounded away from 0 (w.u.p.p.). Therefore, we need a way to determine if the distribution of satisfying assignments is concentrated enough that the expected value is representative of a typical formula, rather than resulting from a couple of highly satisfiable instances. This is exactly what the measurement of the *variance* does, it tells us how much X varies from one formula to another:

$$\mathrm{Var}[X] = \mathbb{E}\left[(X - \mathbb{E}[X])^2\right] = \mathbb{E}\left[X^2\right] - \mathbb{E}[X]^2 \tag{3.9}$$

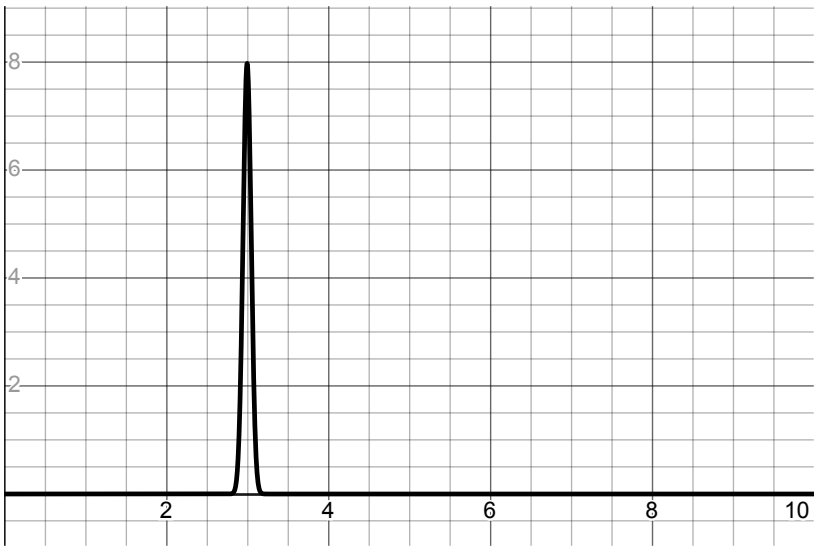


Figure 3.1: A probability distribution function tightly peaked at $x = 3$.

where $\mathbb{E}\left[X^2\right]$ is the expectation of the variable squared, or the *second moment* of X. If the variance is small, the number of solutions is distributed tightly around the average, so $\mathbb{E}[X]$ is representative. If the variance is large, we have a distribution of a couple of highly satisfiable formulas, so the average does not tell us much about a random typical formula.

Note. *As mentioned, we will see that 2MM only gives us a lower bound for which F is satisfiable w.u.p.p., not w.h.p., as 1MM achieves for the upper bound. This issue will be resolved later on in Section 3.4 by using Friedgut's Theorem (Theorem 2.4.5).*

Replacing $g(X)$ with $|X - \mathbb{E}[X]|^2$ and t with t^2 in Markov's Inequality (3.1.2) we get *Chebyshev's Inequality*:

$$\Pr[|X - \mathbb{E}[X]| \geq t] \leq \frac{\mathrm{Var}[X]}{t^2} \tag{3.10}$$

Setting $t = |\mathbb{E}[X]|$ we get:

$$\Pr[|X - \mathbb{E}[X]| \geq \mathbb{E}[X]] \leq \frac{\mathrm{Var}[X]}{\mathbb{E}[X]^2} \tag{3.11}$$

The inequality $|X - \mathbb{E}[X]| \geq \mathbb{E}[X]$ holds whenever $X \leq 0$ or $X \geq 2\mathbb{E}[X]$, so we can safely say that

$$\Pr[X = 0] \leq \Pr[|X - \mathbb{E}[X]| \geq \mathbb{E}[X]] \tag{3.12}$$

where we used that X is non-negative, so $\Pr[X \leq 0] = \Pr[X = 0]$. Combining (3.11) and (3.12) we get

$$\Pr[X = 0] \leq \frac{\mathrm{Var}[X]}{\mathbb{E}[X]^2} \tag{3.13}$$

which is known as Chebyshev's second moment method. We will however use the version of 2MM more frequently seen in lower bound derivations for CSPs ([8, 9]), which we

derive using the well-known *Cauchy-Schwarz Inequality*:

$$\mathbb{E}\left[YZ\right]^2 \leq \mathbb{E}\left[Y^2\right]\mathbb{E}\left[Z^2\right] \tag{3.14}$$

Taking $Y = \mathbb{1}_{\{X\neq 0\}}$, $Z = X$, and recognizing that $\mathbb{1}^2_{\{X\neq 0\}} = \mathbb{1}_{\{X\neq 0\}}$ and the fact that $\mathbb{E}\left[\mathbb{1}_{\{X\neq 0\}}\right] = \Pr[X \neq 0]$, we find:

$$\mathbb{E}\left[X\right]^2 \leq \Pr[X \neq 0]\mathbb{E}\left[X^2\right] \tag{3.15}$$

Again, since the number of satisfying assignments is a non-negative integer, we have $\Pr[X \neq 0] = \Pr[X > 0]$, so

$$\boxed{\Pr[X > 0] \geq \frac{\mathbb{E}\left[X\right]^2}{\mathbb{E}\left[X^2\right]}} \tag{3.16}$$

which we call the *second moment method* (abbreviated as 2MM).

In terms of intuition, this inequality is not as easy to understand as 1MM, however it is clear that, if the ratio of moments on the right is not 0 (and does not approach 0 as $n \to \infty$), we have a non-zero probability that at least one satisfying assignment exists. In other words, we get a lower bound on α_k, to the left of which F is satisfiable w.u.p.p.

Let us now calculate the two moments for X and see whether the ratio we get is useful. The method we use for visualizing the second moment is standard practice in the derivation of the lower bound for random k-SAT [8, 9, 24]. First, we find the expectation (first moment) in the same way we have already done in (3.4):

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{\sigma} \mathbb{1}_{\sigma\vDash F}\right] = \sum_{\sigma}\mathbb{E}\left[\prod_{C_i}\mathbb{1}_{\sigma\vDash C_i}\right] = \sum_{\sigma}\prod_{C_i}\mathbb{E}\left[\mathbb{1}_{\sigma\vDash C_i}\right] = 2^n(1-2^{-k})^m \tag{3.17}$$

Next, we will calculate the second moment. Since $X^2 = \left(\sum \mathbb{1}_{\sigma}\right)\left(\sum \mathbb{1}_{\tau}\right)$ with $\sigma, \tau \in [0,1]^n$ two assignments, we have:

$$\mathbb{E}[X^2] = \mathbb{E}\left[\left(\sum_{\sigma}\mathbb{1}_{\sigma\vDash F}\right)^2\right] = \mathbb{E}\left[\sum_{\sigma,\tau}\mathbb{1}_{\sigma,\tau\vDash F}\right] = \sum_{\sigma,\tau}\mathbb{E}\left[\prod_{C_i}\mathbb{1}_{\sigma,\tau\vDash C_i}\right] = \sum_{\sigma,\tau}\prod_{C_i}\mathbb{E}\left[\mathbb{1}_{\sigma,\tau\vDash C_i}\right] \tag{3.18}$$

where $\mathbb{1}_{\sigma,\tau} = \mathbb{1}_{\sigma}\mathbb{1}_{\tau}$. As was intuitively shown in the 1MM case, $\mathbb{E}\left[\mathbb{1}_{\sigma,\tau\vDash C_i}\right]$ is just the probability that both fixed assignments σ, τ satisfy clause C_i. This probability is entirely dependent on the number of variables to which σ and τ assign the same truth value [8, 9, 24]. We will find this probability intuitively: let the overlap z be the number of variables which share the same assignment between two assignments σ and τ. Then we have $z = rn$ for $0 \leq r \leq 1$. Let f_S denote the probability that the pair (σ, τ) satisfies the clause, so that $f_S = 1 - \Pr[\sigma \text{ fails } \cup \tau \text{ fails}]$. By the principle of inclusion-exclusion, we know that $\Pr[\sigma \text{ fails } \cup \tau \text{ fails}] = \Pr[\sigma \text{ fails}] + \Pr[\tau \text{ fails}] - \Pr[\sigma \text{ fails } \cap \tau \text{ fails}]$. As shown before, only one assignment out of the 2^k possible ones fails to satisfy a clause, so $\Pr[\sigma \text{ fails}] = \Pr[\tau \text{ fails}] = 2^{-k}$. To find $\Pr[\sigma \text{ fails } \cap \tau \text{ fails}]$, we recognize that the only way for both σ and τ to fail the same clause C_i, is that they both must share the same assignment for C_i, precisely the one which fails it. This gets us to $\Pr[\sigma \text{ fails } \cap \tau \text{ fails}] = 2^{-k}r^k$, with $r = z/n$ the overlap ratio. So

$$\Pr[\sigma \text{ fails } \cup \tau \text{ fails}] = 2^{-k} + 2^{-k} - 2^{-k}r^k \tag{3.19}$$

and

$$f_S = f_S(r) = 1 - 2^{1-k} + 2^{-k} r^k \tag{3.20}$$

We now have a formula that gives the correlation between the events that two assignments are satisfying. As mentioned above, we can see that this correlation is entirely dependent on the overlap between the two assignments. Taking for instance two completely uncorrelated assignments, their overlap is $z = n/2$, corresponding to $r = 1/2$. This gives $f_S(1/2) = (1 - 2^{-k})^2$, which is exactly the probability that two independent assignments satisfy a clause. On the other hand, if the two assignments are identical, their overlap is $z = n$, corresponding to $r = 1$. This gives $f_S(1) = 1 - 2^{-k}$, which is the probability that both assignments satisfy a clause if they were the same. We can also see that f_S is an increasing function of r, which means that the more two assignments overlap, the more likely they are to both satisfy a clause. This fact will prove to be very important later on.

Going back to the second moment, since the clauses are independent, we have

$$\mathbb{E}[X^2] = \sum_{\sigma,\tau} \prod_{C_i} \mathbb{E}\left[\mathbb{1}_{\sigma,\tau \vDash C_i}\right] = \sum_{\sigma,\tau} f_S(z/n)^m \tag{3.21}$$

where z is the overlap between the fixed pair of assignments (σ, τ). Taking one of these assignments σ as reference, we know that there are 2^n possible choices for σ, and for each of them, we want to find out how many assignments τ share an overlap of exactly z variables with σ. There are $\binom{n}{z}$ ways to choose the z variables that will agree between σ and τ, and so each of these variables only has one choice of assignment. For the remaining $n - z$ variables, their value must be the opposite of the one in σ, so that exactly z variables overlap, meaning that they also only have one choice of assignment. So then we have that the total number of ordered pairs of assignments with overlap z is $2^n \binom{n}{z}$. This gives us

$$\mathbb{E}\left[X^2\right] = 2^n \sum_{z=0}^{n} \binom{n}{z} f_S(z/n)^m \tag{3.22}$$

We can apply Stirling's approximation to the binomial coefficient (more concretely, to the factorial terms of the coefficient) [8], since $z = rn$ for $0 \leq r \leq 1$ (see [10, 25] for a detailed application of the approximation). We then arrive at the asymptotic identity:

$$\binom{n}{rn} = (r^r (1-r)^{1-r})^{-n} \cdot \text{poly}(n) \tag{3.23}$$

and since $m = \alpha n$, we have

$$\mathbb{E}\left[X^2\right] = 2^n \cdot \text{poly}(n) \cdot \sum_{\substack{z=0 \\ r=z/n}}^{n} \left(\frac{f_S(r)^\alpha}{r^r (1-r)^{1-r}} \right)^n \tag{3.24}$$

All terms in the sum are positive, so, keeping in mind that f_S is increasing in r, we can find a lower bound by taking the maximum term:

$$\mathbb{E}\left[X^2\right] \geq 2^n \left[\max_{0 \leq r \leq 1} \left(\frac{f_S(r)^\alpha}{r^r (1-r)^{1-r}} \right) \right]^n \cdot \text{poly}(n) \tag{3.25}$$

Defining $\Lambda_S(r) := \frac{2f_S(r)^\alpha}{r^r(1-r)^{1-r}}$ for the entire overlap contribution, we rewrite the second moment bound as

$$\mathbb{E}\left[X^2\right] \geq \left(\max_{0\leq r\leq 1} \Lambda_S(r)\right)^n \cdot \text{poly}(n) \tag{3.26}$$

We now take a look back at the first moment (3.17), whose square gives us the relation $\mathbb{E}\left[X\right]^2 = [2^n(1-2^{-k})^{\alpha n}]^2$. We recall that $f_S(1/2) = (1-2^{-k})^2$, so we can rewrite the squared first moment as $\mathbb{E}\left[X\right]^2 = [4f_S(1/2)^\alpha]^n$, and since $r^r(1-r)^{1-r}\big|_{r=1/2} = 1/2$, we get

$$\mathbb{E}\left[X\right]^2 = \Lambda_S(1/2)^n \tag{3.27}$$

We now have the two moment terms required to apply the second moment method (3.16). However, as explained in [8], note that if we find any value of r such that $\Lambda_S(r) > \Lambda_S(1/2)$, then the ratio of the two moment terms exponentially approaches 0. In other words, in these cases, 2MM can only give us an exponentially small lower bound for $\Pr[X > 0]$. To see this, we first consider some value $r^* \in [0,1]$ for which it holds that $\Lambda_S(r^*) > \Lambda_S(1/2)$. Then, by definition we have

$$\max_{0\leq r\leq 1} \Lambda_S(r) \geq \Lambda_S(r^*) > \Lambda_S(1/2) \tag{3.28}$$

Calling $r_{\max}$ the overlap ratio for which the Λ_S contribution is largest among all $r \in [0,1]$, that is, $\max_{0\leq r\leq 1} \Lambda_S(r) \equiv \Lambda_S(r_{\max})$, we can now try to find a result for the ratio in the second moment method expression (3.16) by plugging in (3.27) and (3.26):

$$\frac{\mathbb{E}\left[X\right]^2}{\mathbb{E}\left[X^2\right]} \leq \frac{\Lambda_S(1/2)^n}{\Lambda_S(r_{\max})^n \cdot \text{poly}(n)} = \left(\frac{\Lambda_S(1/2)}{\Lambda_S(r_{\max})}\right)^n \cdot \frac{1}{\text{poly}(n)} \tag{3.29}$$

Notice that this result yields an upper bound on the right-hand side (r.h.s.) of (3.16); this is a consequence of the lower bound we derived for the second moment in (3.25). Now we can see that if *any* r^* exists for which $\Lambda_S(r^*) > \Lambda_S(1/2)$, then, by (3.28), the fraction in the exponential term of (3.29) is proper, i.e. less than 1. And as

$$\lim_{n\to\infty} c^n \cdot \frac{1}{\text{poly}(n)} = 0 \tag{3.30}$$

holds for all values of c where $|c| < 1$, we get an exponentially small upper bound on the r.h.s. of the 2MM expression. In other words, since this ratio of moments constituted the lower bound of $\Pr[X > 0]$ in (3.16), 2MM proves to be useless in this case; we arrive at an exponentially small lower bound, with no meaningful result.

Note that the $\frac{1}{\text{poly}(n)}$ term is not significant here, nor is the sign of its degree: if the degree of the $\text{poly}(n)$ is positive, then the exponential term (provided its base is less than 1) decreases much faster. If the degree is negative, then the exponential decay completely overcomes the polynomial growth. However, this behavior of the bound we found in (3.29) is fully reliant on the crucial assumption we made just above and when

using (3.28): the base of the exponential term, that being the fraction of the two Λ_S terms, is proper *if and only if* there exists an $r^* \in [0, 1]$ such that $\Lambda_S(r^*) > \Lambda_S(1/2)$. In other words, unless the global maximum of Λ_S is at $r = 1/2$, the second moment method fails to provide a useful bound. Since this r corresponds to an overlap of $z = n/2$, this means that the only way to get a useful bound for $\Pr[X > 0]$ (one which does not go to 0 exponentially fast) is if the dominant contribution to the second moment $\mathbb{E}\left[X^2\right]$ comes from pairs of completely uncorrelated assignments, with overlap $n/2$ [8].

What remains now is to see whether this hopeful assumption of a global maximum of Λ_S at $r = 1/2$ is actually true. A closer look at Λ_S reveals an unfortunate truth: even though the denominator $r^r(1-r)^{1-r}$ is symmetric around $1/2$, the numerator, and more specifically, the correlation function f_S, is strictly increasing in the interval $[0, 1]$. This means then that the derivative of Λ_S will not be 0 at $r = 1/2$, but at some other $r^* > 1/2$, where instances are more correlated [8].

An instance of this unfortunate fact can be clearly seen in Figure 3.2, where, as the clause density α increases, the maximum of Λ_S occurs at points closer and closer to $r = 1$. In other words, as the clause density increases, the dominant contribution to $\mathbb{E}\left[X^2\right]$ comes from pairs of assignments with increasing overlap $r > 1/2$, and so, under the current context, the second moment method gives us an exponentially small lower bound as $n \to \infty$, which, as we argued, is not useful to find the critical threshold.

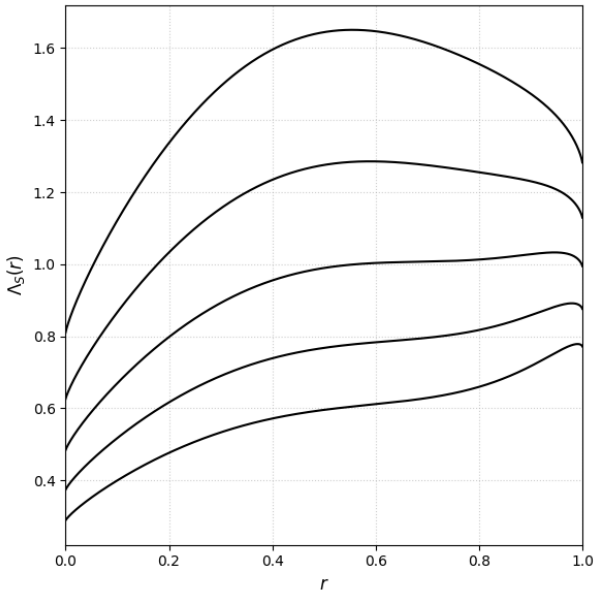


Figure 3.2: Figure adapted from [8], depicting the contribution Λ_S as a function of the overlap r. For all curves $k = 5$ and their corresponding clause density is (from top to bottom) $\alpha \in \{14, 18, 22, 26, 30\}$.

3.3 Restricting the Solution Space: NAE k-SAT

In the previous section, we arrived at the conclusion that, unfortunately, naively applying the second moment method in the same manner as we did for the first moment method does not yield the desired lower bound. For any k, as n increases, the ratio derived in Equation (3.29) drops quickly to 0.

However, this analysis can be rescued by redefining our target random variable. Recall that the previous analyses focused on the random variable X, which counts the total number of satisfying assignments of a formula F. We will now restrict our focus to a specific *subset* of those assignments, and we will investigate the implications of this restriction. In particular, we examine the Not-All-Equal (NAE) satisfying truth assignments, whose counting random variable we denote as X_N.

An assignment σ is an NAE-satisfying assignment for a formula F iff, under σ, every clause in F contains at least one satisfied literal and at least one unsatisfied literal. Notice that this definition implies a symmetrical property: if σ is NAE-satisfying, then $\overline{\sigma}$, its *bitwise complement*, must also be NAE-satisfying. This symmetry is shown to drastically lower the variance of the solution space [8]. As we will demonstrate, this variance reduction is the crucial mechanism that rescues the probabilistic method presented above.

We will proceed with the probabilistic analysis of X_N, in largely the same manner as we have done for X. However, we must first justify why examining a strictly constrained subset of solutions is a valid means of locating a threshold bound for F. Because every NAE-satisfying assignment is, by definition, a valid satisfying assignment subject to an additional constraint (and since not all satisfying assignments are necessarily NAE-satisfying), it follows that $X_N \leq X$ for any given formula. This directly results in the inequality:

$$\Pr[X > 0] \geq \Pr[X_N > 0] \tag{3.31}$$

or the following implication:

$$\Pr[X_N > 0] > 0 \implies \Pr[X > 0] > 0 \tag{3.32}$$

In other words, if we can successfully apply 2MM to prove that a formula has a non-zero probability of having a NAE-satisfying assignment, we have simultaneously proven that it has a non-zero probability of possessing *some* satisfying assignment.

By applying the second moment method on the new random variable X_N, we can explicitly bound the probability of standard satisfiability using only the moments of the NAE subset:

$$\boxed{\Pr[X > 0] \geq \Pr[X_N > 0] \geq \frac{\mathbb{E}\left[X_N\right]^2}{\mathbb{E}\left[X_N^2\right]}} \tag{3.33}$$

Therefore, proving that the ratio of moments for X_N does not end up dropping exponentially fast is a worthwhile endeavor for the analysis on the entire set of satisfying assignments and for $F_k(n, m)$ in general.

Going over our previous analysis and using X_N instead, we start by determining the first moment $\mathbb{E}\left[X_N\right]$. Since we now only focus on NAE-satisfying assignments, we proceed as in Equation (3.4) to get:

$$\mathbb{E}\left[X_N\right] = 2^n(1 - 2^{1-k})^m \tag{3.34}$$

We find this answer intuitively by noticing that, for each random clause C_i in F, exactly two assignments are not NAE-satisfying: the one which satisfies all literals, and the one which satisfies no literals. Then, under some assignment, the probability that C_i is NAE-satisfied is $1 - 2 \cdot 2^{-k}$ for any assignment and any i.

Moving on to the second moment, we apply the same reasoning as we did in Equation (3.18) to find a sum over the 4^n ordered pairs of assignments. Writing $\sigma \vDash_N C_i$ to mean that assignment σ NAE-satisfies the clause C_i of F, we get:

$$\mathbb{E}\left[X_N^2\right] = \sum_{\sigma,\tau} \prod_{C_i} \mathbb{E}\left[\mathbb{1}_{\sigma,\tau \vDash_N C_i}\right] \tag{3.35}$$

In the same manner as before, $\mathbb{E}\left[\mathbb{1}_{\sigma,\tau \vDash_N C_i}\right]$ is the probability that a fixed pair of assignments σ, τ NAE-satisfy the i-th random clause C_i. This probability, which we denote $f_N = f_N(r)$, only depends on the overlap $z = rn$. We again find this intuitively by first looking at $\Pr[\sigma \text{ fails}]$ for some assignment σ, the probability that an assignment σ NAE-falsifies the clause. As reasoned above, we have two NAE-falsifying assignments for every random clause, giving us $\Pr[\sigma \text{ fails}] = 2^{1-k}$. Looking at $\Pr[\sigma \text{ fails } \cap \tau \text{ fails}]$, we recognize that both σ, τ must be one of the two assignments which NAE-satisfy the clause C_i *and* the variables in C_i must either all be in the overlap or all outside the overlap. This results in $\Pr[\sigma \text{ fails } \cap \tau \text{ fails}] = 2^{1-k}(r^k + (1-r)^k)$. The inclusion-exclusion principle then gives us $\Pr[\sigma \text{ fails } \cup \tau \text{ fails}] = \Pr[\sigma \text{ fails}] + \Pr[\tau \text{ fails}] - Pr[\sigma \text{ fails } \cap \tau \text{ fails}]$, from which we find our desired probability:

$$f_N(r) = 1 - \Pr[\sigma \text{ fails } \cup \tau \text{ fails}] = 1 - 2^{2-k} + 2^{1-k}(r^k + (1-r)^k) \tag{3.36}$$

Notice the distinction between f_N and f_S; the most significant change is the addition of the $(1-r)^k$ term, which, as we will see, plays the deciding role in f_N being symmetric around $r = 1/2$. Substituting f_N in Equation (3.35) gives:

$$\mathbb{E}\left[X_N^2\right] = 2^n \sum_{z=0}^{n} \binom{n}{z} f_N(z/n)^m \tag{3.37}$$

where we used the fact that there are $2^n\binom{n}{z}$ ordered pairs of assignments with overlap z, as we have shown for Equation (3.22).

We are once again faced with a sum over all possible overlap sizes between two assignments. However, our bounding strategy must change from the previous analysis. Recall that for the analysis over X, we required a *lower* bound on the second moment to prove the method's failure, which we achieved by simply bounding the sum from below by its maximum term (Equation (3.25)). Now, to prove that the ratio in Equation (3.33) does not bring us to the same unusable conclusion, we want the second moment to not blow up exponentially. That is, we require an *upper* bound on the sum.

We continue by applying Stirling's approximation to the factorial terms of the binomial coefficient, as we have done before (Equation (3.23)), and keeping in mind that $m = \alpha n$, we get

$$\mathbb{E}\left[X_N^2\right] = 2^n \cdot \text{poly}(n) \cdot \sum_{\substack{z=0 \\ r=z/n}}^{n} \left(\frac{f_N(r)^\alpha}{r^r(1-r)^{1-r}}\right)^n \tag{3.38}$$

Looking at the sum of the second moment, we recognize that it has $n+1$ terms (there are $n+1$ distinct overlap sizes from $z=0$ to $z=n$). We can easily bound the entire sum from above by setting every term equal to the maximum one, which gives:

$$\mathbb{E}\left[X_N^2\right] \leq 2^n \cdot (n+1) \cdot \left[\max_{0\leq r\leq 1}\left(\frac{f_N(r)^\alpha}{r^r(1-r)^{1-r}}\right)\right]^n \cdot \text{poly}(n) \tag{3.39}$$

Since $(n+1)$ is a polynomial, we absorb it into the single $\text{poly}(n)$ term we got from Stirling's approximation, to separate the problematic exponential term from the polynomial ones. The exponential base will then allow us to determine if we have successfully circumvented the blow-up of the variance which we observed in the previous analysis. Defining $\Lambda_N(r) := \frac{2f_N(r)^\alpha}{r^r(1-r)^{1-r}}$ then results in:

$$\mathbb{E}\left[X_N^2\right] \leq \left(\max_{0\leq r\leq 1}\Lambda_N(r)\right)^n \cdot \text{poly}(n) \tag{3.40}$$

We have arrived at a similar relation for the second moment as we have seen before in our analysis for all standard satisfying assignments, only now, we have achieved an upper bound on $\mathbb{E}\left[X_N^2\right]$ as opposed to a lower bound. Looking back at the first moment (Equation (3.34)) and taking its square, we find that we can express it in terms of f_N:

$$\mathbb{E}\left[X_N\right]^2 = [4(1-2^{1-k})^{2\alpha}]^n = \left[\frac{2(1-2^{2-k}+2^{2-2k})^\alpha}{1/2}\right]^n = \Lambda_N(1/2)^n \tag{3.41}$$

Looking back at Equation (3.33), we find that both moment terms we required are now defined. The situation has changed in two significant ways, compared to our previous analysis on X: firstly, we now want to show the method's success on the subset of NAE-satisfying assignments, which is why an upper bound on $\mathbb{E}\left[X_N^2\right]$ was achieved. If our result in Equation (3.33) is not exponentially bounded, then we have shown that restricting our analysis to the subset $X_N \leq X$ successfully eliminated the variance problem. Secondly, from Equation (3.41) we find that, if $\Lambda_N(1/2) > \Lambda_N(r)$ for any $r \neq 1/2$, then the ratio of the two moments $\mathbb{E}\left[X_N\right]^2$ and $\mathbb{E}\left[X_N^2\right]$ is polynomially bounded, and the exponential term which dominated the ratio's drop in Equation (3.29) is neutralized. Defining r_{max} as the overlap ratio where the Λ_N contribution is largest, i.e. $\max_{0\leq r\leq 1}\Lambda_N(r) \equiv \Lambda_N(r_{max})$, the ratio in Equation (3.33) gives us:

$$\frac{\mathbb{E}\left[X_N\right]^2}{\mathbb{E}\left[X_N^2\right]} \geq \left(\frac{\Lambda_N(1/2)}{\Lambda_N(r_{max})}\right)^n \cdot \frac{1}{\text{poly}(n)} \tag{3.42}$$

Therefore, we are now interested in determining the conditions under which $r_{max} = 1/2$. As in the previous section, we find that the denominator of $\Lambda_N(r)$ is symmetric around $1/2$, however this time, the numerator, or rather the function f_N, *is symmetric* around $1/2$ as well, and therefore, so is $\Lambda_N(r)$. This result, crucially, tells us that $\Lambda_N(r)$ has a local extremum at $r = 1/2$: at worst, this at least means that the new overlap contribution we found is not generally increasing, so, although the probability of pairs of assignments to NAE-satisfy a clause (f_N) is lower than that of the pair satisfying the clause in any way (f_S), this disadvantage is compensated by the fact that f_N is not biased towards pairs of increasingly similar assignments ($r \approx 1$). This trade-off between similarity and entropy can be seen in Figure 3.3 (left). At best, the symmetry of $\Lambda_N(r)$

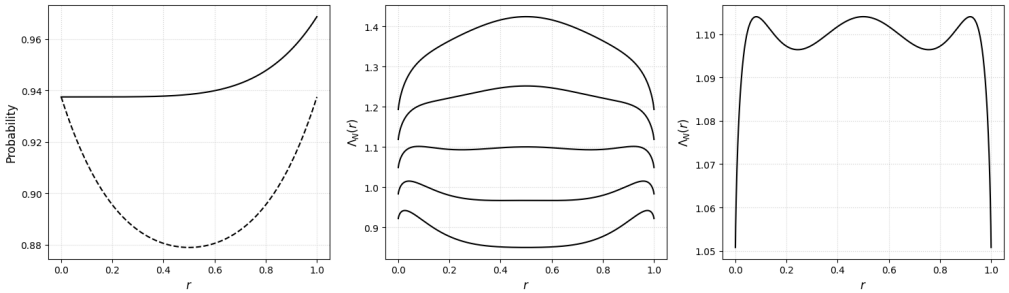


Figure 3.3: **Left:** The functions f_S (full line) versus f_N (dotted line). **Middle:** Figure adapted from [8], depicting the contribution $\Lambda_N(r)$ for different clause densities (from top to bottom) $\alpha \in \{8, 9, 10, 11, 12\}$. **Right:** Figure adapted from [8], depicting $\Lambda_N(r)$ at $\alpha = 9.973$, where the peak at the center $r = 1/2$ is equal to the two peaks near the borders.

tells us that its maximum term is at $r_{max} = 1/2$, in which case the exponential factor's base in Equation (3.42) is 1, and the exponential factor completely vanishes.

Indeed, the factor $(r^r(1-r)^{1-r})^{-1}$ of $\Lambda_N(r)$ is always maximized at $r = 1/2$, while the factor $f_N(r)^\alpha$ is always minimized at that same point, with its maximum at $r = 0$ and $r = 1$. However, since the first factor does *not* depend on the clause density α, for sufficiently small α, the maximum of $\Lambda_N(r)$ is at the extremum point $r_{max} = 1/2$. In Figure 3.3 (middle), we show this behavior for $k = 5$ and α increasing from 8 to 12, from top to bottom. For $\alpha = 8, 9$, it is clear that $r_{max} = 1/2$, which means that the ratio of moments collapses, and, as we will show in the next section, this results in 2MM succeeding. For $r = 11, 12$, r_{max} is seen at the two peaks around the boundaries, while $r = 1/2$ is actually a global minimum.

Thus, we have shown that, for sufficiently small clause density α, we can combine Equation (3.33) and Equation (3.42) to get a lower bound which is itself polynomially bounded:

$$\Pr[X > 0] \geq \left(\frac{\Lambda_N(1/2)}{\Lambda_N(1/2)}\right)^n \cdot \frac{1}{\text{poly}(n)} = \frac{1}{\text{poly}(n)} \tag{3.43}$$

Comparing this with Equation (3.29), we can see that this is already a much better result, as the lower bound does not drop to 0 exponentially fast. However, the dependence on n still remains in the form of the poly(n) term, which continues to restrict us from at least finding a lower bound for $\Pr[X > 0]$ which does not go to 0 as $n \to \infty$ (this refers to the event $\{X > 0\}$ occurring w.u.p.p., recall Definition 2.4.1). In the next section, we will take a much closer look to our sum approximation (Equation 3.39) and show that the ratio of moments for the subset of NAE-satisfying assignments is actually bounded by a constant [8, 9].

To understand why the symmetry of $\Lambda_N(r)$ drastically lowers the variance, it is helpful to visualize the solution space geometrically. In our previous analysis, we saw the increase in f_S as the overlap $z = rn$ increases. Put differently, in standard k-SAT, satisfying assignments tend to "cluster" together: if an assignment satisfies a formula, flipping the value of just one or two variables is highly likely to result in another satisfying assignment. This results in the solutions being grouped tightly together (being very similar to each other), and so the expected overlap between any two random solutions is heavily biased toward $r \approx 1$. This correlation is precisely what causes the second

moment to explode.

By only allowing NAE-satisfying assignments, we actively destroy these clusters. Because every valid assignment σ guarantees the existence of its exact complement $\overline{\sigma}$, assignments are forced to be distributed symmetrically across the solution space. In a sense, NAE-satisfying assignments "repel" each other; moving from σ to $\overline{\sigma}$ requires traversing a vast space of non-satisfying assignments. This forces the overlap between two random solutions to concentrate strictly around $r = 1/2$ (the overlap of two uncorrelated random assignments), which eliminates the clustering variance, making the use of the second moment method possible.

3.4 Asymptotic Evaluation and the Lower Bound

In the previous section, we applied the 2MM method focusing only on the subset of NAE-satisfying assignments, which turned out to completely eliminate the exponential term present in our derivation of a lower bound for $\Pr[X > 0]$. However, a lower bound on the threshold w.u.p.p. could still not be achieved because of the poly(n) term of Equation 3.43. We will now briefly present how this issue is resolved by Achlioptas and Moore [8], which finally reveals to us a lower bound on the satisfiability threshold.

We must first address the distinction between what the 1MM and 2MM methods have been able to achieve. The first moment method guaranteed that above the threshold, the formula is unsatisfiable w.h.p. (recall Equation 3.7 and Definition 2.4.1). The second moment method however only guarantees that below the threshold, the formula is satisfiable w.u.p.p.

To elevate this result from w.u.p.p. to w.h.p., Achlioptas and Moore [8] invoke the following "boosting" corollary of Friedgut's theorem (Theorem 2.4.5):

Corollary 3.4.1. *If $F_k(n, \alpha^* n)$ is satisfiable w.u.p.p., then $F_k(n, \alpha n)$ is satisfiable w.h.p. for all $\alpha < \alpha^*$.*

To intuitively understand why this corollary directly follows from Theorem 2.4.5, recall that satisfiability is a monotonic property. Friedgut's theorem then tells us that its transition from probability 1 to 0 must be sharp. If, at a certain clause density α^*, the probability of satisfiability is bounded away from zero, the sharp drop must not have occurred yet. Therefore, the probability of satisfiability must asymptotically go to 1 for any $\alpha < \alpha^*$. Then, establishing that $\Pr[X_N > 0] > 0$ at density α^* is sufficient to prove that random K-SAT is satisfiable w.h.p. for all densities below α^*, which is what we refer to as "boosting" the probability.

With this "boosting" method established, the task simplifies to finding the clause density α up to which the second moment method succeeds. To do this, we must eliminate the problematic poly(n) term in our bound from Equation 3.39. Recall that to derive that term, we first used Stirling's approximation to the factorial terms of the binomial coefficient in Equation 3.37, after which we bounded the sum loosely by taking its maximum term and multiplying it by the total number of terms $(n+1)$. This results in bounding the sum inside a large rectangle (of width equal to the maximum term of the sum), which heavily overestimates the true value and introduces the polynomial issue.

To evaluate the sum accurately and extract a constant instead of a term dependent on n, Achlioptas and Moore [8] apply *Laplace's method* for asymptotic evaluation (cf. [28],

Chapter 4). Laplace's method is a standard technique for approximating integrals and sums of exponential functions. It relies on the principle that, as $n \to \infty$, certain kinds of exponential functions become incredibly "spiked" at their maximum value. They drop quickly from the global maximum and the majority of the sum's "mass" (or area) is concentrated in a tiny, Gaussian-like neighborhood around that peak. Since $\Lambda_N(r)$ is slightly peaked at $r = 1/2$ (for sufficiently small α, see Figure 3.3 (right)), raising it to $n \to \infty$ turns it into this type of tightly peaked function. Therefore, instead of evaluating the sum as a wide rectangle, Laplace's method approximates it by integrating over this sharp Gaussian peak.

Importantly, the area under this Gaussian peak scales proportionally to $1/\sqrt{n}$, while the poly(n) factor from Stirling's approximation carries a $\sqrt{n}$ term in the numerator [8]. Therefore, these polynomial terms perfectly cancel out. For the summation to evaluate to a constant (and thus for 2MM to succeed without exponential blow-up), this global maximum must occur strictly at the center, $r = 1/2$, as we discussed in the previous section. We will now evaluate in closer detail for what α values this is indeed the case.

Because $\Lambda_N(r)$ is symmetric (recall Figure 3.3), we are primarily interested in the extreme boundaries at $r = 0$ and $r = 1$ not overcoming the value at $r = 1/2$. While a complete proof requires demonstrating that no local maxima exceed the center across the entire interval $[0, 1]$ [8], we will only focus on the boundaries not overtaking the center value, as they are the most critical constraint (recall Figure 3.3 (right)). Additionally, we will focus only on the $r = 1$ point, since $\Lambda_N(r)$ is completely symmetrical. Therefore, we evaluate the boundary requirement:

$$\Lambda_N(1/2) > \Lambda_N(1) \tag{3.44}$$

We can evaluate both sides of this inequality directly. Recall that $\Lambda_N(r) = \frac{2f_N(r)^\alpha}{r^r(1-r)^{1-r}}$ and $f_N(1/2) = (1 - 2^{1-k})^2$ (from Equation 3.36). We then have that:

$$\Lambda_N(1/2) = 4(1 - 2^{1-k})^{2\alpha} \tag{3.45}$$

At the boundary $r = 1$, the denominator $1^1 0^0 = 1$, where we used that $\lim_{x\to 0^+} x^x = 1$, aligning with the condition $\binom{n}{n} = 1$ of Equation 3.37. We also have $f_N(1) = 1 - 2^{1-k}$. Thus:

$$\Lambda_N(1) = 2(1 - 2^{1-k})^\alpha \tag{3.46}$$

Substituting these into Equation 3.44 and dividing by $2(1 - 2^{1-k})^\alpha$ gives:

$$2(1 - 2^{1-k})^\alpha > 1 \tag{3.47}$$

Solving for α as we did in Equation 3.6 for the upper bound gives us:

$$\alpha < -\frac{\ln 2}{\ln(1 - 2^{1-k})} \tag{3.48}$$

where we took into account that $\ln(1 - 2^{1-k}) < 0$.

This expression provides the exact boundary condition required for the center peak of the overlap distribution to dominate the extreme edges. By Taylor expanding the logarithm using $\ln(1-x) \approx -x - x^2/2$ (cf. [10]), this bound simplifies to an asymptotic approximation of (see [8, 9] for an exact derivation):

$$\alpha < 2^{k-1} \ln 2 - \frac{\ln 2}{2} \tag{3.49}$$

However, it is important to note that our boundary check $\Lambda_N(1/2) > \Lambda_N(1)$ is a simplified condition. It only guarantees that the center of the overlap distribution is taller than the extreme edges. To completely apply Laplace's method, it must be proven that $r = 1/2$ is the strict global maximum across the entire interval $r \in [0, 1]$.

Achlioptas and Moore rigorously prove that this is the case, resulting in a very similar bound [8]. This bound tells us that, for any clause density α lower than it, the probability of satisfiability is 1 w.h.p. (thanks to Corollary 3.4.1). So, by reversing the inequality as we have done for the upper bound and using the rigorous form of the inequality from [8], we find our lower bound for the critical clause density α_k:

$$\boxed{\alpha_k \geq 2^{k-1} \ln 2 - \frac{\ln 2}{2} - \frac{1 + t_k}{2}} \tag{3.50}$$

for a sequence $t_k \to 0$. As such, we have finally arrived at a lower bound for the critical threshold α_k, using the restricted subset of NAE-satisfiable assignments as our random variable.

This bound represents an important result in the study of random CSPs, as many other methods have been developed on the back of this methodology to give tighter windows for the critical threshold [9, 29, 26]. When compared to the upper bound derived from the first moment method ($\alpha_k \leq 2^k \ln 2$), we see that the lower bound matches the upper bound up to a factor of 2. This effectively determined the random K-SAT phase transition to a significantly small window, proving that the second moment method (when applied to the correct subspace of solutions) provides the necessary analytical power to tightly bound the sharp threshold. Table 3.2 presents the lower bounds we get from Equation 3.50, compared to the currently known bounds. The factor of 2 which has been "sacrificed" to analyze only the NAE-satisfying subset is clearly seen in the differences between the values. Nevertheless, they do constitute actual lower bounds and we can see their proportional scaling as k increases.

k	3	4	5	7	10	20
2MM Bound	1.93	4.69	10.24	43.51	354	363,408
Known Bound	3.52	7.91	18.79	84.82	704.94	726,809

Table 3.2: Comparison of 2MM and known lower bounds for random K-SAT

Chapter 4

Empirical Analysis of Random k-SAT

In the previous chapter, we reconstructed the probabilistic methods developed in breakthrough papers [8, 9, 26, 19] to bound the satisfiability threshold of the $F_k(n, m)$ ensemble. We have shown that the 1MM and 2MM both succeed in giving non-trivial bounds (with some extra work required for the second moment), and that the application of symmetric subsets (such as NAE-SAT) allows us to derive a tight lower bound as $k \to \infty$. While this analysis is arguably fundamental to proving the (existence and) location of complexity phase transitions, theoretical bounds alone do not explain the underlying structure of the formulas, nor do they explain how algorithms behave when attempting to solve them.

In this chapter, we turn to an empirical analysis of the $F_k(n, m)$ ensemble, where we investigate the solution space in terms of complexity theory and statistical physics formulations. The intersection of these fields has been tremendously successful in explaining the sharp threshold phenomenon [24, 21, 27, 18]. We begin by visualizing the phase transition alongside the algorithmic runtime, and we show the results' compatibility with our previous theoretical analysis. Next, we will investigate the apparent connection between the phase transition and inherent hardness, as well as clarify the distinction between average-case algorithmic barriers and worst-case complexity.

Furthermore, we introduce the statistical mechanics formulation of k-SAT, which will allow us to interpret the solution space as its geometry evolves. We will explore how the solution space of $F_k(n, m)$ shatters into many isolated, frozen clusters, and how this geometric shattering phenomenon is related to algorithmic barriers for SAT solvers. All experiments are implemented in the Python programming language, which has been chosen for memory and formula solving efficiency (see Section 4.1), as well as for convenience of plotting results.

4.1 Background and Prerequisites

In this section, we briefly discuss tools and terminologies used in this experiment and others, as well as give insight into our empirical methodology and usage of external

tools.

4.1.1 The DIMACS Encoding

In our experiments, we will have to encode boolean formulas in text. We do this by means of the DIMACS encoding, the standard encoding used for CNF formulas (of the form in Equations 2.7 and 4.3). In this convention, boolean variables are mapped to non-zero integers (e.g. $x_i \mapsto i$) and logical negation is represented by mathematical negation (e.g. $\neg x_i \mapsto -i$). A clause C_i is therefore encoded as an array of integers. Generally, when looking for a textual representation of a formula, one adapts the DIMACS (CNF) *format*, which translates the formula into a text file with arrays representing clauses, as mentioned above (and a header line above the clauses). However, rather than writing these encodings to disk as standard DIMACS files, we only generate the instances in the DIMACS convention and, using the `PySAT` toolkit [30], we avoid having to write massive formulas in text files, which would then have to be read by our solving algorithm. Instead, the encoded array is simply passed to our algorithm the moment it is generated, so that disk input/output (I/O) bottlenecks are completely eliminated. In Example 4 we show how a random instance is translated to an array of integers in the DIMACS convention.

Example 4. The boolean formula

$$(x_1 \vee \neg x_4 \vee \neg x_2) \wedge x_3 \wedge (\neg x_3 \vee \neg x_1) \tag{4.1}$$

has the following DIMACS encoding:

```
[[1,-4,-2],[3],[-3,-1]]
```
(4.2)

4.1.2 SAT Solvers

Since we want our Monte Carlo approach to be as accurate as possible, we aim to experiment with large samples of formulas. To be able to evaluate this massive amount of instances with reasonable time and space resources, we require efficient algorithms for solving formulas, i.e. SAT solvers. In our experiments, we use the `CaDiCaL` SAT solver [31] provided by the `PySAT` library. `CaDiCaL` is based on the Conflict-Driven Clause Learning (CDCL) algorithm, which is a sound and complete algorithm for SAT [32]. That is, `CaDiCaL` (and all other CDCL-based solvers) is guaranteed to determine a formula's (un)satisfiability, given enough resources. In later experiments, we also talk about Stochastic Local Search (SLS) solvers, which are sound but incomplete, meaning they cannot prove that a formula is unsatisfiable [33]. However, SLS-based solvers come with the advantage of significantly increased speed and efficiency, and are generally the preferred option if one does not wish to determine unsatisfiability. In our experiments, for both of these kinds of solvers, we pass generated instances directly as DIMACS encoded arrays, as discussed in Section 4.1.1. This bypasses the creation of huge text files having to be read by the solvers constantly, which would otherwise have had to be written for every single random instance which we sample.

4.2 The Phase Transition as an Empirical Feature

Our primary goal in this section is to visualize the sharp threshold of the probability space of $F_k(n, m)$ experimentally, as well as to determine the well-known correlation of this phenomenon with algorithmic runtime. We begin by reformulating the definition

of the $F_k(n, m)$ CSP ensemble (Definition 2.3.1) into an equivalent model which aligns with a more experimental setting. We will think of the instances of $F_k(n, m = \alpha n)$ as being generated by a *generator*, where we recall the following parameters: (1) n, the number of distinct variables and (2) α, the clause density. Given these two parameters, we generate $m = \alpha n$ clauses, where each clause C_i is made up of k independently and uniformly selected literals without replacement. Each literal l_j is generated as either the variable x_j or its negation. Then, the generated formula F will have the following familiar form (see Definition 2.3.1 and Section 2.3.1 describing CNF formulas):

$$F = \bigwedge_{i=1}^{m} C_i \tag{4.3}$$

where $C_i = (l_1^i \vee \cdots \vee l_k^i)$ and l_j^i is either x_j or $\neg x_j$ with probability 1/2.

Note. *As mentioned, the same formulation is given in Section 2.3.1, however the notation C_i is exchanged with F_i for clauses, for the purpose of not confusing general constraints and clauses. From now on, we refer to the C_i notation and explicitly state when mentioning general constraints.*

The above generator formulation then completely describes how a random instance of $F_k(n, m = \alpha n)$ is formed. With this picture in mind, we can now lay out our experiment.

4.2.1 Experimentation with Random 3-SAT

Recall from Section 2.4.3 in what setting we are referring to a satisfiability phase transition: we are talking about a *probability space* in which the probability distribution of random instances is mapped. It is of interest to construct experiment in which we can in fact visualize this distribution as a function of the clause density α. For the purposes of this thesis, our experiments on $F_k(n, m)$ will mainly focus on the $k = 2$ and $k = 3$ ensembles. That is, we will perform the experiments on random 2-SAT and random 3-SAT. The reason for this choice is twofold: firstly, there is a fundamental difference between the inherent complexity of these two ensembles. Their deterministic counterparts have been extensively studied and it is known that 2-SAT $\in \mathbf{P}$ [34], while 3-SAT is **NP**-complete [35]. This gives us reason to investigate the probability spaces of both ensembles, since it might as well be the case that the hardest subset of random instances of these two ensembles might be of fundamentally different hardness (assuming $\mathbf{P} \neq \mathbf{NP}$). Secondly, we already know from Friedgut's Theorem (Theorem 2.4.5) that the probability spaces of the ensembles for $k > 3$ (and $k = 1$) will also exhibit a phase transition of similar properties, a fact which has also been shown experimentally [36].

Let us now look at $F_3(n, m)$, i.e. the random 3-SAT ensemble, and empirically investigate the probability distribution of its instances.

Remark 4.2.1. Since the ensemble contains approximately $\left(2^k \binom{n}{k}\right)^m$ distinct formulas, an exhaustive evaluation on the probability space is simply computationally intractable. Therefore, we repeatedly sample a large amount of random instances to approach the behavior of the distribution at the thermodynamic limit (i.e. as $n \to \infty$). That is, we use Monte Carlo methods [37] for our experiments.

Experimental Setup

As mentioned above, we want to generate a sufficiently large statistical sample of random formulas across the interval of clause densities. We implemented a random generator following the formulation of Equation (4.3).

The experiment was parameterized with the following:

- Variable set sizes: Increasing n values were chosen to better approximate (and predict) the behavior of the phase transition as $n \to \infty$.

- Clause densities: For each n, we swept the parameter $\alpha = m/n$ from 2.0 to 8.0 in different increments, depending on the proximity to the critical region (and critical value α_3). We already know that for $\alpha \ll \alpha_3$, Pr[SAT] = 1 w.h.p. (refer to Theorem 2.4.5 and Definition 2.4.1, where we denote "SAT" to be the property that a random instance is satisfiable), while for $\alpha \gg \alpha_3$, Pr[SAT] = 0 w.h.p. In other words, for clause densities far from the critical region, the probability is already known to be near 1 (to the left) or 0 (to the right), so we test less points in these regions. Thus, for $\alpha \in [2, 3.2)$ and $\alpha \in (5.2, 8]$, we test 10 discrete values each (with increment $\Delta\alpha = 0.13$), and for the critical region $\alpha \in [3.2, 5.2]$, we test 20 values for better accuracy (with increment $\Delta\alpha = 0.105$).

- Sample size: At each discrete density point α, we generated N_α independent random instances to ensure accurate probabilities. As for the increments of the discrete density values, we are more interested in the critical region, since we already know the behavior of the distribution outside of it at the thermodynamic limit. Therefore, we apply the same reasoning and sample a relatively large amount of instances for the critical region ($\alpha \in [3.2, 5.2]$), while for the rest of the interval, N_α is relatively low. The exact values of N_α are mentioned later and might differ from one experiment to another.

To evaluate generated instances, we used the `CaDiCaL` SAT solver. For each instance, `CaDiCaL` returned whether the formula is satisfiable, as well as the total solving runtime in milliseconds. From this data, we extract two macroscopic measurements for each density point α: the probability of satisfiability Pr[SAT] (calculated as the fraction of the N_α instances that were satisfiable) and the average runtime required by the solver.

4.2.2 Results and Interpretation

Using our experimental setup from Section 4.2.1, we will now analyze the resulting probability distributions and empirically visualize the well-known sharp threshold for different parameters.

Let us now look at Figure 4.1, in which the results of our experiment are presented, in the form of the probability distribution, as well as the corresponding runtime plot for the $F_k(n, m = \alpha n)$ ensemble over a sequence of system sizes n. Focusing on the top plot, we find clear confirming empirical evidence for the sharp threshold phenomenon. Not only do we see the familiar sigmoid-like drop from Pr[SAT] = 1 to Pr[SAT] = 0, but we also observe the slope of this drop steepening as n increases. This steepening empirically represents the shrinking transition window $w(n, \delta)$ of this ensemble (see Definition 2.4.4). As the system scales, the continuous curve approaches a discontinuous, vertical, infinitely steep hill, i.e. a step function. This is exactly the expected behavior in line with Friedgut's Theorem (Theorem 2.4.5).

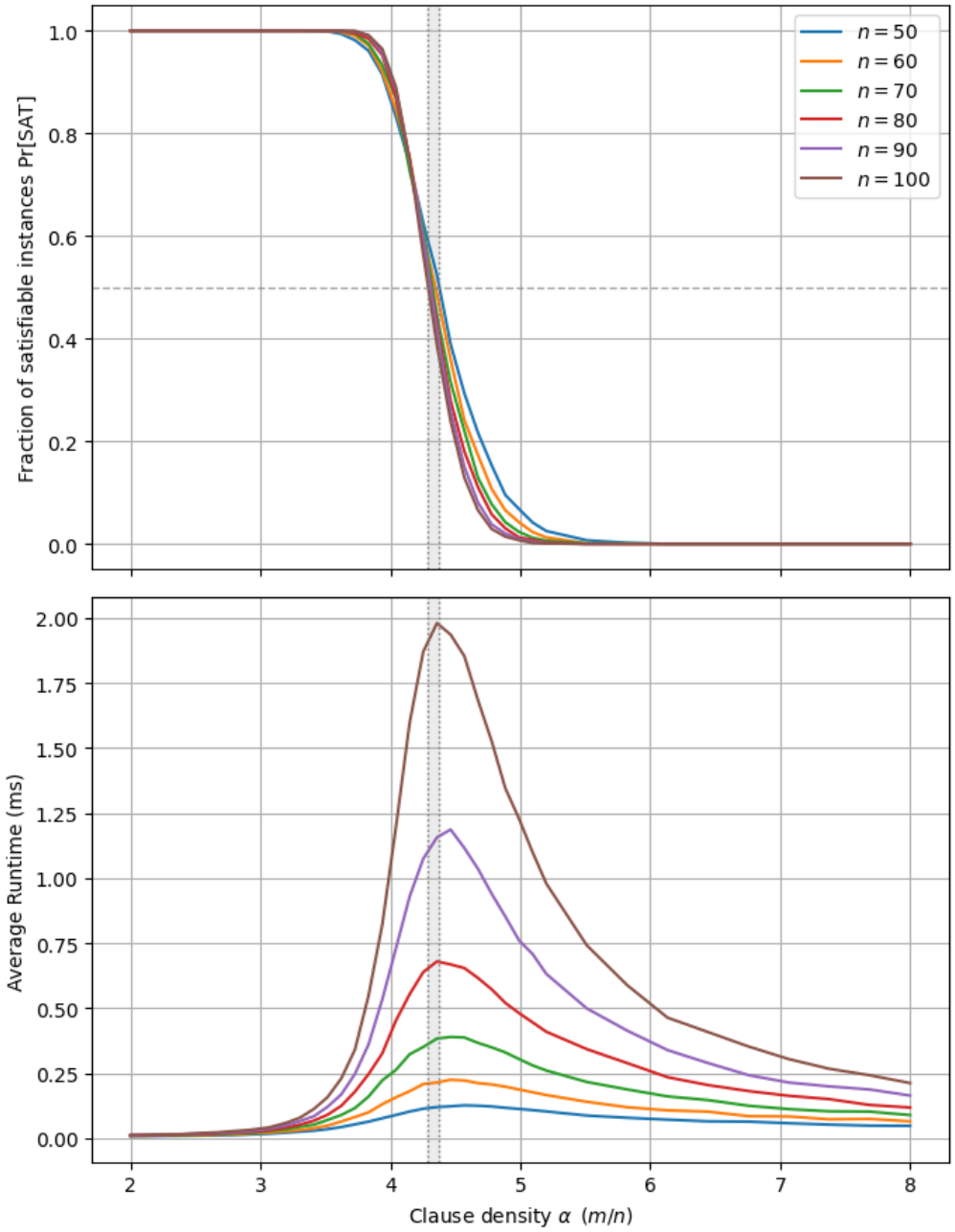


Figure 4.1: **Top:** The estimated probability Pr[SAT] that a generated random 3-SAT instance is satisfiable, for increasing system sizes $n \in \{50, 60, \ldots, 100\}$. The horizontal dashed line marks Pr[SAT] = 0.5. See Figure A.1 from Appendix A for a similar 3D version of the plot. **Bottom:** The average solver runtime (in milliseconds) evaluated over the same parameter space. In both plots, the shaded gray vertical band marks the *empirical threshold band*. This band is bounded by the 0.5 probability crossing points of the smallest ($n = 50$) and largest ($n = 100$) model sizes, capturing the drift of the effective critical point and encapsulating the theoretical asymptotic threshold $\alpha_3 \approx 4.26$. Notice how the algorithmic runtime peaks scale exponentially with increasing n and align precisely with this drifting threshold (due to local effects at low n, the corresponding peaks are shifted slightly to the right of the crossing points). For the critical region $\alpha \in [3.2, 5.2]$, $N_\alpha = 10,000$ instances were sampled per density point α, while $N_\alpha = 1000$ instances were generated in the non-critical regions.

Furthermore, we can observe a secondary phenomenon known in statistical mechanics as *finite-size scaling drift* [38]. Because we are evaluating finite formulas rather than the thermodynamic limit ($n \to \infty$), the effective critical point (measured at $\Pr[\text{SAT}] = 0.5$) shifts to the left as n grows (see again the top plot Figure 4.1). The shaded vertical band in the figure captures this finite-size drift interval. This interval encapsulates the analytically predicted critical threshold of $\alpha_3 \approx 4.26$ and, importantly, confirms the validity of the upper and lower bounds we derived in Chapter 3 (see Tables 3.1 and 3.2 for an upper and lower bound, respectively).

Looking now at the bottom plot, we observe a striking correlation between this phase transition and computational intractability. Far to the left of the effective critical point ($\alpha \ll \alpha_3$), the formula is significantly under-constrained, meaning, solutions are abundant due to a low number of constraints relative to the system size n, and the solver easily finds a satisfying assignment in minimal time. Far to the right ($\alpha \gg \alpha_3$), the formula is severely over-constrained. That is, the solver encounters logical contradictions almost immediately, allowing it to quickly prune the search tree and return unsatisfiable.

However, precisely inside the transition window, the problem experiences a dramatic shift in complexity. At (and around) this critical density, formulas are critically balanced: they contain just enough clauses to eliminate obvious solutions, but not enough to create quick contradictions. As we will later show, the solver is thus forced to exhaustively explore vast portions of the search space. We also notice that the runtime peaks present in the critical region scale exponentially with increasing n, and that they almost perfectly align with the threshold band (the gray region in the plots of Figure 4.1).

Remark 4.2.2. It is worth mentioning that early literature on the 3-SAT phase transition often measured computational effort via the size of the search tree (number of backtracks) to remain hardware-neutral. However, our experiments utilize `CaDiCaL`, a modern CDCL-based solver. Because CDCL algorithms employ dynamic restarts and aggressive clause learning, the concept of a simple "search tree size" breaks down. Therefore, we measure raw wall-clock runtime. Despite the massive pruning optimizations in modern solvers, which resolve $n = 100$ instances in mere milliseconds, the exponential peak remains structurally bound to the $\alpha_3 \approx 4.26$ threshold. This suggests that the transition represents an intrinsic algorithmic barrier, rather than a hardware or implementation limitation.

Looking purely at the linear scale of this runtime plot, one might conclude that the over-constrained phase rapidly returns to polynomial "easiness", a historically popular idea known as the "easy-hard-easy" pattern [3]. However, a closer look at the data on a logarithmic scale leads to a different conclusion.

As illustrated in Figure 4.2, when plotted on a logarithmic scale, an exponential function manifests as constant vertical spacing between curves of evenly spaced n. While the absolute runtime drops after the critical peak, the vertical gaps between the $n = 50, \ldots, 100$ curves remain proportional in the over-constrained phase. This provides strong experimental evidence that proving unsatisfiability remains exponentially hard, merely with a smaller exponent, leading to the better named "easy-harder-less hard" pattern. This pattern has been repeatedly shown not to be a feature of a specific solver's efficiency, such as the `CaDiCaL` solver we are using. Instead, it is the definitive pattern observed in random 3-SAT runtime plots for any currently used algorithm [4].

Regardless of the exact scaling in the over-constrained region, the visually striking align-

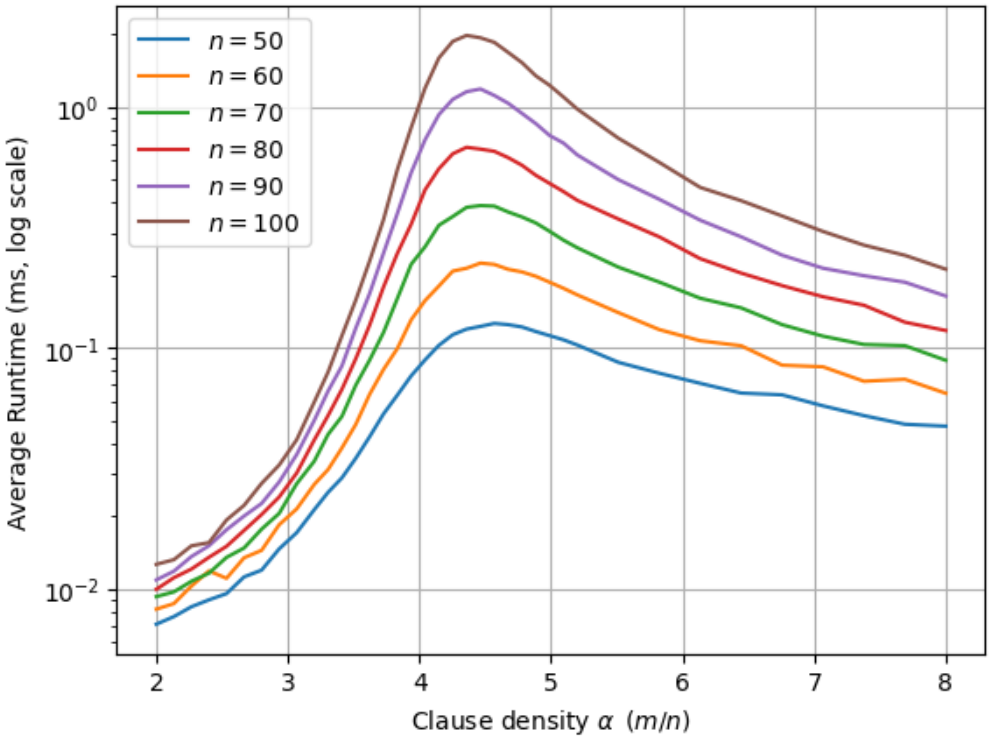


Figure 4.2: The average solver runtime from the bottom graph of Figure 4.1 plotted on a logarithmic scale. Notice the constant vertical spacing between any two consecutive peaks of the curves in the critical region. This indicates a constant multiplicative change in peak runtime, i.e. an exponential increase, since the y-axis is logarithmically scaled. Similarly, we find that in the over-constrained region ($\alpha \gtrsim 5$), the spacing between the curves for consecutive n remains relatively constant, with all the curves decreasing linearly. Again, this indicates that the runtime remains exponential, albeit with a decreasing base, refuting the "easy-hard-easy" pattern in favor of a more accurate "easy-harder-less hard" flow first described by Vardi and Alfonso San Miguel [4]. Additionally, note that the evenly spaced curves are a consequence of the fact that evenly spaced sizes n are plotted, which is why we can observe the exponential behavior of the over-constrained region. Figure A.2 in Appendix A presents the same data plotted in three dimensions, which results in the evenly spaced regions forming straight lines.

ment of the runtime peak with the critical threshold led researchers in the 1990s to a profound and popular hypothesis [5]: perhaps the sharp threshold phenomenon is the "true" signature of **NP**-completeness itself. It was highly intuitive to assume that phase transitions generated computational intractability, i.e. the "inherent hardness" of the problem. If this were true, problems in **P** would not exhibit such abrupt computational threshold behavior [5].

To determine whether phase transitions are truly artifacts of inherent hardness, or simply structural properties not (completely) causally related to algorithmic complexity, we must apply this exact methodology to a polynomially solvable ensemble.

4.3 Inherent Hardness and Inherent Easiness: the Random 2-SAT Counterexample

In this section, we revisit the methodology from Section 4.2 and apply it to the $F_2(n, m = \alpha n)$ (random 2-SAT) ensemble. Since 2-SAT $\in$ **P** (polynomial-time solvable), we use it in hopes of determining whether the phase transition phenomenon is strictly innate to **NP**-completeness. If $F_2(n, m)$ also exhibits a sharp satisfiability threshold, the intuitive assumption that phase transitions inherently generate hardness falls apart, and a much more nuanced view of the connection to computational complexity should be taken.

4.3.1 Experimental Setup

The generative model and testing procedure remain identical to our previous experiment from Section 4.2, where we again use the `CaDiCaL` solver to evaluate Monte Carlo samples. However, we modify the parameters for the $F_2(n, m = \alpha n)$ ensemble. We test significantly larger, equally spaced system sizes of $n \in \{1000, 1500, 2000, 2500, 3000, 3500\}$. This equal spacing allows us to visually evaluate the runtime scaling factor, similar to our approach in the random 3-SAT experiment. We sweep the parameter α from 0.0 to 2.5, sampling $N_\alpha = 10,000$ instances per density point in the critical region $\alpha \in [0.6, 1.6]$, and $N_\alpha = 1000$ in the non-critical surrounding regions.

Notice that the system sizes for which we now test are an order of magnitude larger than the ones we used for our 3-SAT experiment. Testing with such large n is possible precisely because 2-SAT is a polynomial-time problem, and solvers are able to comfortably resolve 2-SAT instances in linear time. Therefore, to generate measurable computational effort and provide definitive evidence for the aforementioned hypothesis, we are required to significantly increase the system size.

4.3.2 Results and Interpretation

As illustrated in the top plot of Figure 4.3, the $F_2(n, m)$ ensemble undeniably exhibits a sharp satisfiability phase transition. Similar to 3-SAT, the probability drop steepens as n increases. We also observe finite-size scaling drift (see the shaded band in Figure 4.3), although now, the finite-size crossing points occur slightly above the true asymptotic threshold and drift leftward toward the analytically proven [18, 19] critical point of $\alpha_2 = 1$.

The most profound realization, however, lies in the bottom plot of Figure 4.3. The algorithmic runtime definitively peaks almost exactly at $\alpha = 1$. Yet, the scale and nature of this peak are fundamentally different from 3-SAT.

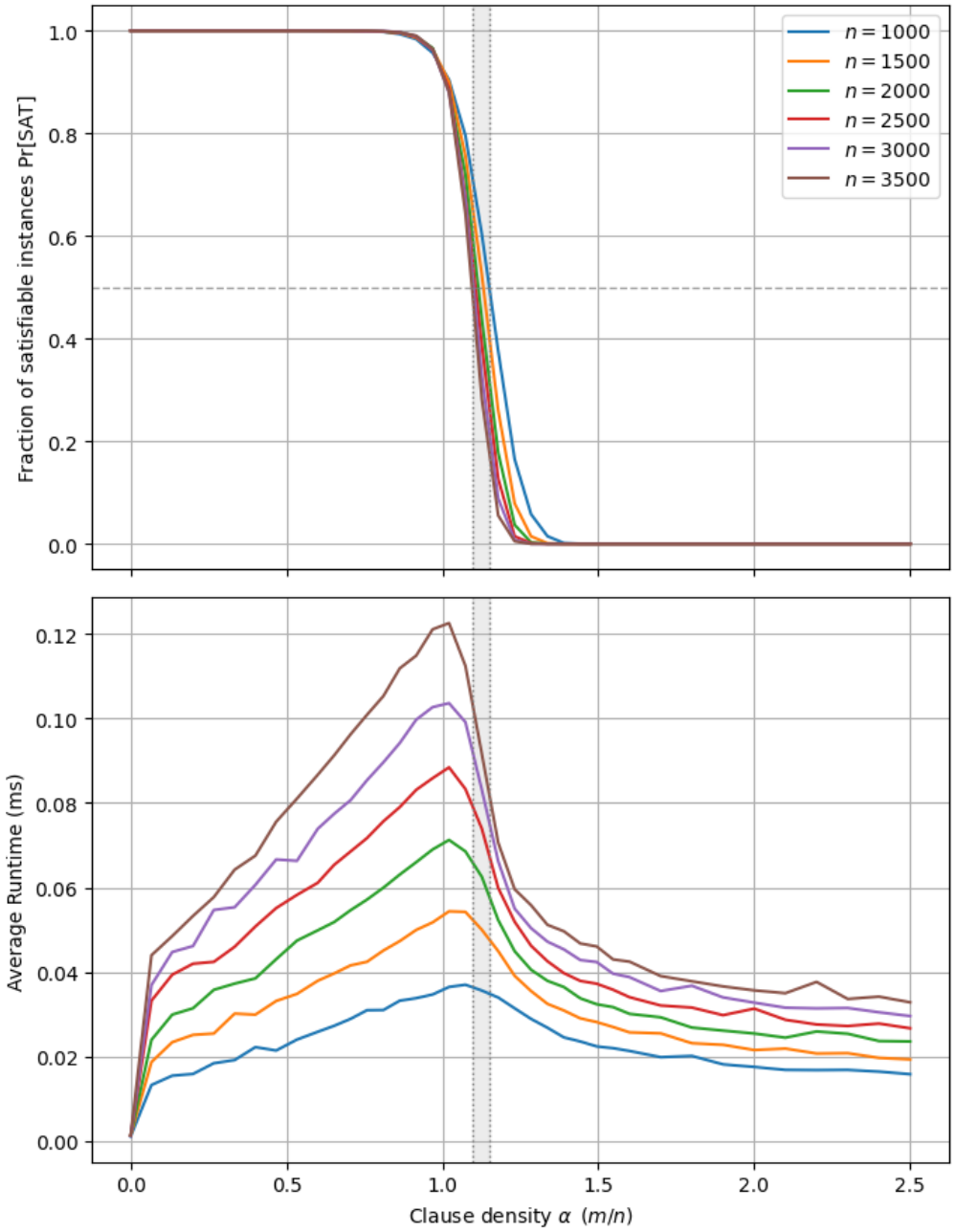


Figure 4.3: **Top:** The estimated probability Pr[SAT] for random 2-SAT instances, for system sizes $n \in \{1000, 1500, \ldots, 3500\}$. The shaded vertical band marks the finite-size drift interval bounded at the crossing points (Pr[SAT] $= 0.5$) of $n = 1000$ and $n = 3500$. **Bottom:** The average solver runtime (in milliseconds) evaluated over the same space. Notice that the runtime peaks essentially exactly at $\alpha = 1.0$, while the probability crossing point approaches $\alpha = 1.0$ from the right as n increases. Furthermore, note the constant vertical spacing between the runtime peaks on this linear scale, indicating polynomial scaling.

In our previous experiment (Figure 4.2), exponential runtime scaling was proven by constant vertical gaps between curves on a *logarithmic* axis. Here, in Figure 4.3, the vertical gaps between the equally spaced, linearly increasing system sizes of n remain constant on a *linear* axis. This confirms that the peak computational effort scales linearly ($O(n)$).

In the under-constrained region ($\alpha < 1$), formulas are satisfiable (w.h.p.). To return satisfiable, the solver must provide an assignment for all n variables. As α increases, there are linearly more clauses to verify (since $m = \alpha n$), resulting in the upward slope. In the over-constrained region ($\alpha > 1$), formulas are dense in contradictions. The solver hits a conflict almost instantly and terminates without evaluating the rest of the formula, causing the rapid drop in runtime.

This empirical result definitively refutes the hypothesis that sharp phase transitions are the defining signature of **NP**-completeness. Random 2-SAT possesses a mathematically (and visually) analogous phase transition, yet solvers move past the threshold with polynomial effort. Therefore, our results support the conclusion established by researchers in the late 1990s: the satisfiability phase transition does not intrinsically cause algorithmic hardness. A more refined picture was formed by Monasson et al. [6], who showed that it is not the mere existence of the phase transition, but its specific nature and order that ties the phenomenon with computational complexity.

We can, however, argue with much more success that the under-constrained region to the left of the phase transition is directly connected to *inherent easiness*. This idea is corroborated by our results; for both the $k = 2$ and $k = 3$ ensembles, low clause densities ($\alpha \ll \alpha_k$) correspond to instances so densely packed with valid solutions that solvers consistently find assignments in polynomially bounded time. The true algorithmic divergence between **P** and **NP** only reveals itself as we approach and cross the critical threshold.

To understand why random 3-SAT becomes an impenetrable computational wall at its threshold while random 2-SAT does not, we must look beyond the macroscopic properties of the probability curves. We must open the formulas and examine the microscopic geometric structure of the solution space.

4.4 The Geometry of the Solution Space

In the previous sections, we established that the satisfiability threshold does not inherently cause exponential algorithmic hardness, as demonstrated by random 2-SAT. To find a satisfactory answer as to why the random 3-SAT ensemble exhibits this exponential algorithmic runtime scaling, we must relate the algorithms themselves to the evolving structure of formulas in the ensemble. In this section, we attempt to achieve this by empirically investigating the set of possible solutions for a typical instance, i.e. the *solution space*.

4.4.1 The Solution Space, the Energy Landscape and the Structural Phase Transitions

To visualize how the microscopic structure of formulas in $F_3(n, m = \alpha n)$ evolves over different densities α, we employ the following formulation: for any instance (i.e. boolean formula) F with n variables, the *search space* of F is an n-dimensional boolean hypercube containing all 2^n possible assignments. Each node in the search space hypercube

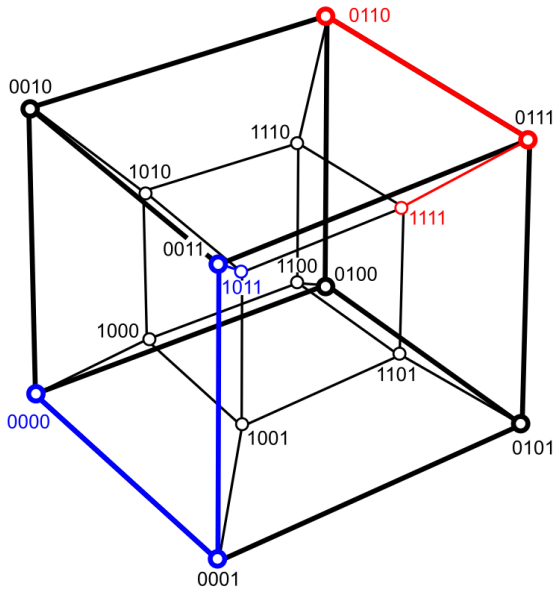


Figure 4.4: 4-dimensional boolean hypercube, reproduced from [39]. The Hamming distance d_H between nodes $(0,0,0,0)$ and $(1,0,1,1)$ is 3, while d_H between nodes $(0,1,1,0)$ and $(1,1,1,1)$ is 2.

represents an assignment, and any two nodes connected by an edge correspond to two assignments which only differ by one bit (or one variable assignment). In other words, any two assignments σ, τ in the search space are connected if their Hamming distance $d_H(\sigma,\tau) = 1$ (see Definition 2.1.4). The *solution space* of F is the strict subset of nodes in the hypercube that satisfy F.

Example 5. Consider the following formula F of 4 variables:

$$\neg x_2 \wedge (\neg x_1 \vee x_3) \wedge (\neg x_3 \vee x_4) \tag{4.4}$$

Since $n = 4$, the search space of F is a 4-dimensional hypercube, illustrated in Figure 4.4. The nodes marked in blue form the solution space of F, the strict subset of assignments which satisfy it.

Phase Transitions from Statistical Physics

Theoretical research in statistical mechanics has revealed that the solution space of random 3-SAT is not static. Rather, it undergoes profound structural phase transitions long before the ultimate satisfiability threshold ($\alpha_3 \approx 4.26$) is reached [20, 21, 27, 40, 23]. These phases all behave differently and are observed in terms of how the solution space of an instance evolves with increasing α:

1. **The Replica Symmetric Phase:** This phase occurs at low densities α, where the solution space is in a state where virtually all assignments form a giant connected component.

2. **The Clustering Phase:** At around $\alpha_d \approx 3.86$ (for random 3-SAT), the space undergoes a *shattering* (or clustering) phenomenon. The giant component breaks

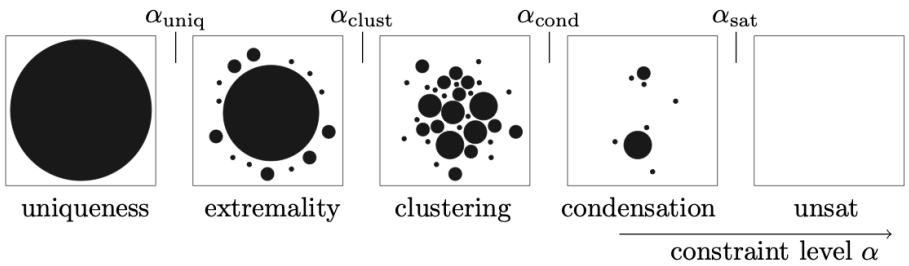


Figure 4.5: Figure reproduced from [20], depicting the conjectured structural phases of the solution space of a random instance and the corresponding phase transition. Notice the additional phase transition α_μ, representing the so-called *uniqueness transition*. For the purposes of this thesis, the theory behind this threshold point will be omitted (cf. [20]).

into exponentially many clusters separated by vast regions of unsatisfiable assignments. This is usually called the *clustering threshold* α_d.

3. **The Freezing Phase:** As density increases further ($\alpha_f \approx 4.25$), these clusters experience *freezing*, where a macroscopic fraction of variables becomes locked, drastically narrowing the volume of each cluster. This is usually called the *freezing threshold* α_f.

4. **The Satisfiability Phase:** Finally, the density increases all the way to the critical threshold α_k (in this case, $\alpha_3 \approx 4.26$). This is the *satisfiability threshold* which we have analyzed up until now.

Figure 4.5 presents a theoretical evolution of the (2-dimensionally projected) solution space of a random formula. We can see that the macroscopic satisfiability phase transition α_k is also present as a threshold in the microscopic structure of instances, since the disappearance of clusters, and thus of solutions, implies the formula is not satisfiable. Regarding the other structural phases, our focus in this experiment will be primarily on the clustering, and on visualizing the transition from one giant connected component to multiple isolated clusters. As such, for the purposes of this thesis, we limit ourselves to only discussing the clustering and satisfiability thresholds (α_d and α_k, respectively), the former only being analyzed in this section.

Furthermore, we borrow a concept often used in statistical physics literature: the energy landscape [40]. If we define the energy-cost of any assignment as the number of clauses it currently violates under formula F, a satisfying assignment is simply a ground state with an energy of zero (the more clauses are violated, the higher the cost). For example, a local search algorithm would attempt to navigate this landscape by moving "downhill" (flipping variables) toward zero energy, where all clauses are satisfied.

We have now described the principal evolutionary timeline of the solution space, from the giant component to the clustering phase, all the way to the familiar unsatisfiable phase. To intuitively understand why this sequence of geometric transitions results in algorithmically intractable instances, we must map this space and observe how search algorithms navigate it.

4.4.2 Experimental Setup: Two-Tiered Local Sampling

Our methodology in analyzing these solution spaces consists of two primary goals: (1) visualizing the evolution of the solution space by projecting it onto a 2-dimensional plane over multiple constraint densities α, and (2) recording the Hamming distances of pairs of solutions as α increases. The first goal provides an intuitive understanding of the geometric topology, while the second goal provides a more definitive view on the behavior of the structure in question. Namely, we are interested in finding the non-trivial arrangement of the solution space in the clustering phase ($\alpha_d < \alpha < \alpha_k$).

In order to empirically find the topology of the solution space, we must sample it. While modern CDCL and DPLL solvers (such as `CaDiCaL`) are exceptionally fast, their depth-first, deterministic nature makes them heavily biased. Such algorithms tend to exhaustively look into a single local region of the space, optimizing for finding one solution quickly, rather than exploring the space uniformly [41].

Instead, we present a simple two-tiered Stochastic Local Search (SLS) algorithm (see Algorithm 1), which operates on the assumption that the solution space is indeed arranged in clusters separated by large Hamming distance. The algorithm consists of a two-tier heuristic as follows:

1. **Inter-cluster Tier:** The algorithm starts from a random assignment σ in the hypercube and descends into lower energy states by traversing the edges of around σ (flipping one variable at a time), eventually reaching a solution in a random region of the search space.

2. **Intra-cluster Tier:** The algorithm repeatedly perturbs the solution reached in the above tier by a small Hamming distance (a "kick" temporarily increasing energy) and allows the SLS to descend again. This attempts to map the local region around the discovered solution.

Our algorithm records all solutions it has found on the n-dimensional hypercube. However, to visualize the space in a comprehensible manner, we must project the data onto a lower dimensional space. For this purpose, we map the results into a 2D representation using `UMAP` (Uniform Manifold Approximation and Projection), a well-known dimensionality reduction technique [42]. `UMAP` is particularly useful when the objective is to preserve (as much as possible) distances within and between clusters, which is preferred under the assumption that the solution space indeed shatters into clusters at α_d.

It is important to note that `UMAP` is a non-linear technique, meaning that its projection is not the result of linear transformations, such as projecting on a plane. This allows us to find connected components which are stretched in more complex manners, otherwise not visible by simple projection. As a consequence, this means that the two axes generated in the projections do not have much physical meaning and cannot help us interpret the data. In our case however, this is not a serious issue, as what is most important in our experiment is the similarity metric *between* each solution, namely, the Hamming distance. Therefore, as long as we are able to observe and compare similarities in our plots, this projection is sufficient.

For our second goal, we are interested in computing the Hamming distances between pairs of satisfying assignments. This can be done by simply analyzing the solution space sampled by our two-tiered algorithm and computing the distance between each pair of

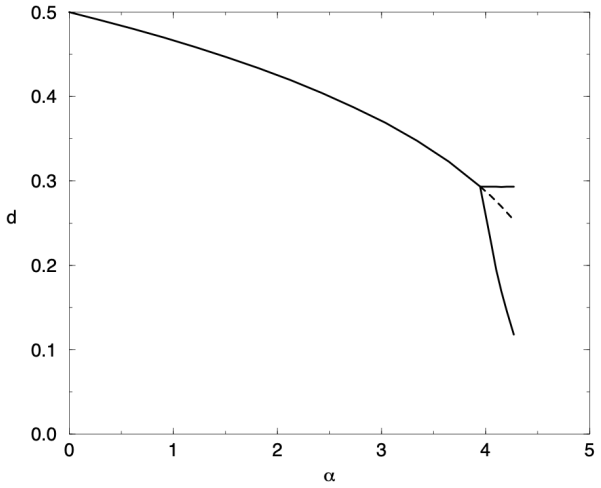


Figure 4.6: Figure reproduced from [40], depicting a theoretically derived model for the Hamming distance between two solutions of a random 3-SAT instance. In the Replica Symmetric phase ($0 \leq \alpha < \alpha_d \approx 3.96$), we see the distance d (measured as the fraction of the Hamming distance with the total bits n) decreasing as α increases. At the clustering phase ($\alpha_d < \alpha < \alpha_3$), we see the curve split in two distances: one which remains almost constant, representing the distance between two clusters, and one which drops near 0, representing the shrinking size of the clusters (i.e. the distance between points in a cluster).

points in the space. The results will be visualized using histograms. Theoretically, at low densities, we expect to see a single, unimodal distribution, indicating that any two solutions belong to the same giant connected component. As α increases, we expect to see this distribution drift slowly towards lower distances, analogous to the decrease in Hamming distance of a single pair of solutions, as shown in Figure 4.6. Going past the clustering threshold α_d, we expect this distribution to become bimodal: a large peak representing the vast distance between different clusters, and a smaller peak near zero representing the distance between solutions inside the same cluster. This is again analogous to the single pair case depicted in Figure 4.6, where we see the bifurcation of typical Hamming distance near α_d.

Algorithm 1 Two-Tiered Stochastic Local Search (SLS) Sampler

Require: CNF Formula F with n variables, Clusters C, Local Steps S
Ensure: A set $\mathcal{A}$ of valid solutions

1: $\mathcal{A} \leftarrow \emptyset$
2: **for** $i = 1$ **to** C **do** ▷ **Inter-cluster Tier (Global jumps)**
3: $\sigma_{rand} \leftarrow$ random boolean assignment in $\{0,1\}^n$
4: $\sigma_{base} \leftarrow$ SLS_Search(F, σ_{rand})
5: **if** σ_{base} is not found **then**
6: **continue**
7: **end if**
8: $\mathcal{A} \leftarrow \mathcal{A} \cup \{\sigma_{base}\}$
9: **for** $j = 1$ **to** S **do** ▷ **Intra-cluster Tier (Local jumps)**
10: Draw $k \sim \mathcal{U}\{3,4,5,6\}$
11: $\sigma_{kick} \leftarrow \sigma_{base}$ with k randomly flipped variables
12: $\sigma_{local} \leftarrow$ SLS_Search(F, σ_{kick})
13: **if** σ_{local} is not found **then**
14: **continue**
15: **end if**
16: $\mathcal{A} \leftarrow \mathcal{A} \cup \{\sigma_{local}\}$
17: Draw $r \sim \mathcal{U}(0,1)$
18: **if** $r < 0.5$ **then**
19: $\sigma_{base} \leftarrow \sigma_{local}$
20: **end if**
21: **end for**
22: **end for**
23: **return** $\mathcal{A}$

Remark 4.4.1. Note that the SLS_Search procedure refers to our simplified stochastic local search subroutine. Given an initial assignment, it iteratively selects a random violated clause and flips one of its variables until the formula is satisfied (or a maximum step limit is reached). Lines 5–7 and 13–15 cover these cases where solver does not reach the ground state of a valid assignment. Lines 17–20 present a simple heuristic for a more uniformly distributed intra-cluster search, where we either jump back to the base assignment or continue on the chain of assignments, with probability 50%.

4.4.3 Results and Interpretation: The Algorithmic Barrier

Using our Two-Tiered SLS algorithm, we sampled the solution space of random 3-SAT formulas at various significant constraint densities. For each sampled ensemble, we generated a 2-dimensional `UMAP` projection and a pairwise Hamming distance distribution to observe the evolution of the space. The results presented below have all been generated from ensembles with $n = 100$ variables. This allows us to avoid the strong local effects found for lower n, as well as to test with reasonably large samples.

The Replica Symmetric Phase ($\alpha < \alpha_d$)

In this phase, the non-rigorous statistical physics methods predict the solution space to be a giant connected component, with the typical Hamming distance between pairs of assignments decreasing in a monotonic manner with increasing densities. Our empirical results seem to align well with the theoretical model in this phase.

As shown in Figure 4.7a, the `UMAP` projection for density $\alpha = 1$ presents a massive,

uniform distribution of the solutions. The similarity between the points in the projection corresponds to small Hamming distance in the hypercube, rendering the projection analogous to the giant connected component prediction. Figure 4.7c presents a similar but slightly smaller projection, as we reach $\alpha = 2$. Looking now at the distribution of Hamming distances, a similar evolution of the solution space is found in Figures 4.7b and 4.7d. We start with a unimodal distribution around Hamming distance of 45, consistent with the theoretical predictions at $\alpha \approx 1$ (see Figure 4.6). As we increase the density to $\alpha = 2$, the distribution has its peak now at a slightly lower Hamming distance.

Figure 4.7e presents a noticeably different solution space at $\alpha = 3$, where we can see evidence of clusters forming, almost separate from the large connected component. This seems to align with the statistical physics picture of the solution space geometry at around this clause density [40, 20, 21, 27]. The corresponding histogram in Figure 4.7f again presents evidence for the decreasing typical Hamming distance, with a slightly more significant decrease than before.

With this visual representation of the solution space, it is easy to see how an algorithm can easily find solutions, and go from one solution to another without having to violate clauses in between. That is, the algorithm is not required to pass regions of high energy-cost. Additionally, this representation perfectly aligns with the polynomial "inherent easiness" observed in our earlier experiments no the macroscopic probability distribution.

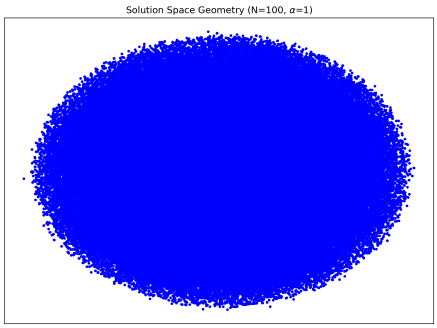
The Clustered Phase ($\alpha_d < \alpha < \alpha_3$)

As the density increases toward the dynamical clustering threshold ($\alpha_d \approx 3.86$), the geometry of the solution space drastically changes. Long before the formulas become logically unsatisfiable at α_3, the giant component shatters into isolated islands [40].

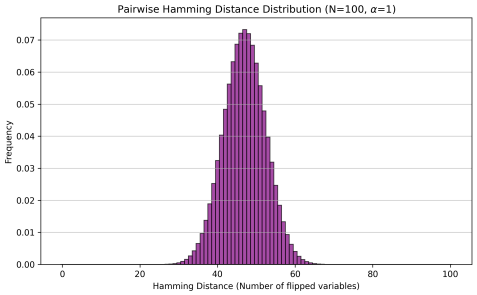
Our mapping at $\alpha = 3.8$ (Figure 4.7g) presents visual evidence for this clustered phase. The solution space is shattered into sparse, disconnected components. This also allows for the intuitive explanation of increasing algorithmic in this geometry. To get from one cluster to another, (local) solvers have to pass through vast regions of unsatisfiable assignments. This amounts to the energy landscape being full of high cost "hills", where many clauses are violated. As α increases in this phase, These effects are only amplified. As previously discussed, theoretical models predict that the Hamming distance distribution in this phase should be bimodal. We should find a large peak representing the distances between clusters, and a smaller peak near zero representing the size of a single cluster.

However, as seen in our histogram distribution for $\alpha = 3.8$ (Figure 4.7h), our Two-Tiered SLS algorithm predominantly captured only the large inter-cluster peak.

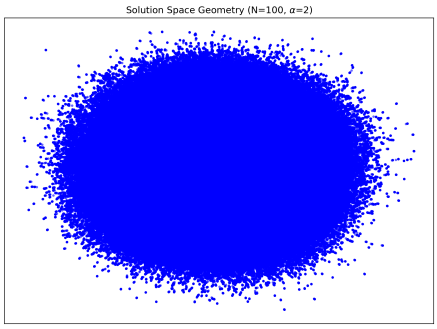
To explain this discrepancy between the predicted behavior and our results, we propose a possible explanation. This explanation relates the absence of the intra-cluster peak with the difference in magnitude of the collected data. Recall that our algorithm (see Algorithm 1) operates by randomly surveying the solution space for solutions, in the first tier, and checking the local region once a solution has been found, in the second tier. Denoting the number of inter-cluster jumps as C and the number of intra-cluster local jumps as S, we find that the maximum number of distances which can be computed between pairs of solutions inside clusters is $N_{local} = C\binom{S}{2}$, where we assumed that all C and S jumps result in satisfying assignments. This results from the fact that,



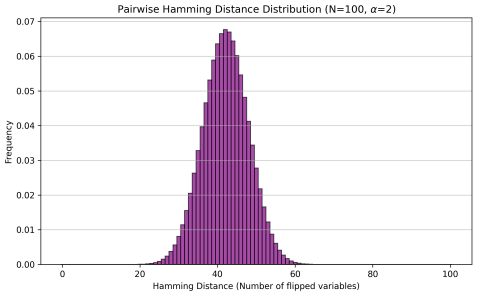
(a)



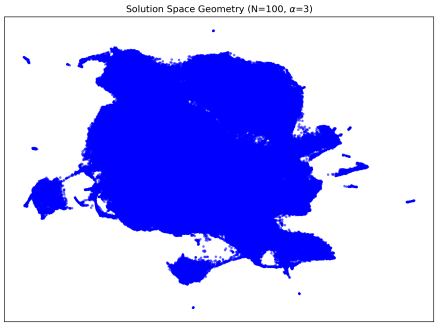
(b)



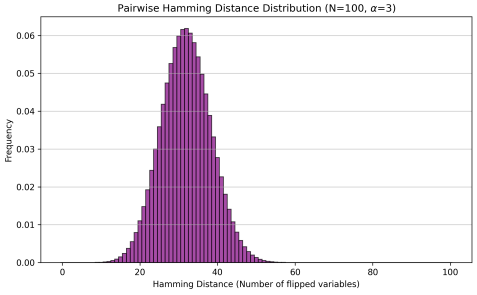
(c)



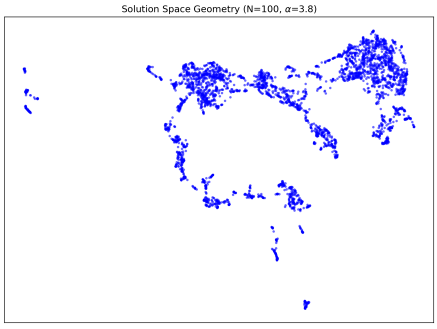
(d)



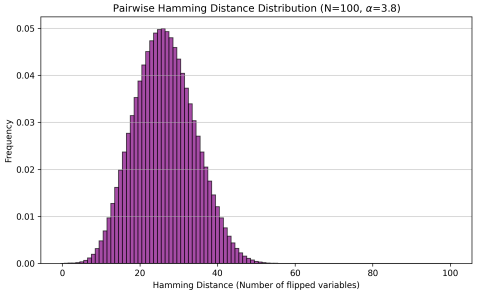
(e)



(f)



(g)



(h)

Figure 4.7: **On the left:** UMAP 2-dimensional projections of the solution space of a typical instance of $F_3(n, m)$ with $n = 100$. From top to bottom, the projections are made at $\alpha = 1, 2, 3, 3.8$. The first two projections (a) and (c) were generated with $C = S = 2000$, (e) was generated with $C = S = 800$ and (g) was generated with $C = S = 400$ (see Algorithm 1). **On the right:** The corresponding histogram representing the distribution of Hamming distances between pairs of found solutions.

with our assumptions in mind, we know that we have C clusters, each containing S solutions, so the number of pairs of solutions inside each cluster is the same. This then corresponds to the expected smaller peak in our results. We additionally find the number of computed distances with solutions between clusters to be $N_{global} = \binom{C \cdot S}{2} - N_{local}$, since the binomial coefficient in the equation amounts to the number of all possible pairs of solutions in the space. If the typical distribution of assignments in the solution space even somewhat corresponds to our assumptions, it is clear then why the smaller intra-cluster peak does not seem to appear in our histograms: the sheer difference in magnitude between these numbers would make the intra-cluster peak virtually invisible, relative to the bigger inter-cluster distribution. As an example, we take the values $C = 500, S = 400$, which means that our algorithm would search globally for 500 solutions, and at each of these solutions it would look for 400 satisfying assignments in the vicinity. We then have the following:

$$N_{local} = 500\binom{400}{2} = 39,900,000$$

$$N_{global} = \binom{500 \cdot 400}{2} - N_{local} = 19,960,000,000$$

$$\frac{N_{local}}{N_{global}} \approx 2 \cdot 10^{-3}$$

As our experiment tests for such values of C and S, this could in fact be one important reason why the bimodal distribution is not visible. Indeed, Figure 4.8 presents the same distribution of distances for $\alpha = 3.8$ as in Figure 4.7h, only now, the y-axis is logarithmically scaled. We can see that there seems to be an increase in the number of pairs of solutions with low Hamming distances. Testing for higher α might show a clearer distinction between these two distributions, however, we have limited our testing to $\alpha = 3.8$, as our algorithm is simply not efficient enough to test for much larger densities.

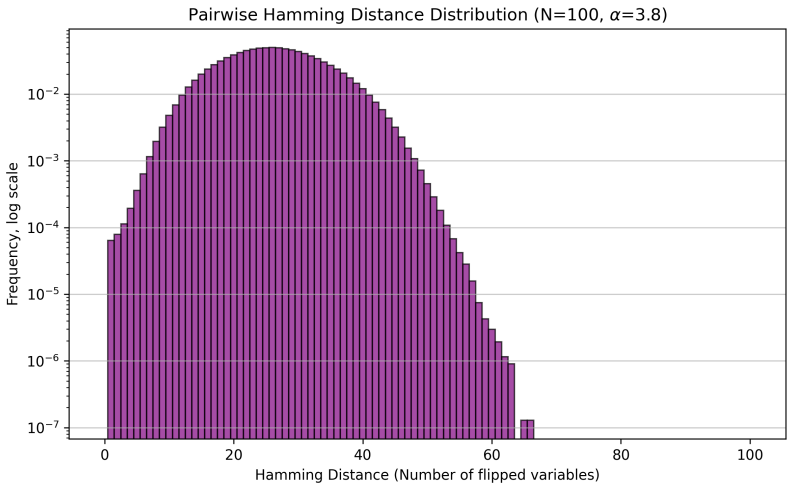


Figure 4.8: The log-scaled distribution of the Hamming distances in Figure 4.7h, for $\alpha = 3.8$. Notice the slight increase to the left of the distribution peak, near distance 0. The effect is more visible when testing for the same density and for $n = 150$, as seen in Figure A.3 from Appendix A.

The Source of Algorithmic Hardness

The mapping of the solution space geometry provides compelling evidence supporting the link between structural phase transitions and computational intractability. While rigorous proofs connecting the shattering transition to algorithmic failure are currently largely established only for large k [29], statistical physics research strongly conjectures that the same geometric mechanisms apply to random 3-SAT. Our results align with this hypothesis, suggesting that the exponential runtime barrier is not fundamentally caused by the macroscopic loss of solutions at $\alpha_3 \approx 4.26$, but is deeply tied to the microscopic shattering of the solution space beginning at $\alpha_d \approx 3.86$.

This picture offers a powerful intuitive explanation for the failure of SAT solvers. For local search algorithms (such as the SLS sampler used in our experiment), the clustered phase represents a highly disconnected energy landscape. To traverse the space, the algorithm must navigate vast regions of unsatisfiable assignments separating the isolated clusters. This results in local search algorithms getting permanently trapped single clusters.

In fact, this phenomenon explains the primary limitation of our own empirical methodology. As the density approaches $\alpha = 3.8$, our algorithm's efficiency degrades. The inability of our Two-Tiered SLS algorithm to properly map the higher density spaces is largely caused by the very phenomenon we set out to observe: the clustering of the solution space creates vast non-traversable voids that isolate our local search to specific, disconnected regions.

Furthermore, this geometry provides intuition for the exponential runtime of complete algorithms like DPLL and CDCL. To prove unsatisfiability or find a rare solution in the clustered phase, a deterministic solver must exhaustively explore a search tree. However, because the clusters by definition result in certain variables being heavily constrained to satisfy the formula, the solver constantly chooses assignments that violate these constraints, resulting in massive, exponentially scaling backtracks.

Ultimately, by comparing the polynomial scalability of $F_2(n, m)$ with the exponential runtime scaling of $F_3(n, m)$, as well as mapping the random 3-SAT solution space, we conclude that the empirically observed complexity of random 3-SAT appears deeply rooted in their microscopic geometry. It is crucially important to note, however, that this result holds in direct relation to the algorithms we have discussed. The observed intractability applies specifically to local search heuristics and resolution-based solvers, it does not constitute a formal proof of this causal connection, nor does it definitively rule out the existence of a theoretical, undiscovered global algorithm capable of bypassing this geometric clustering.

Chapter 5

Global Constraints and the ITE Tree Ensemble

In the previous chapters, we analyzed the $F_k(n, m)$ ensemble (random K-SAT), a *locally constrained* CSP, where the arity of each constraint remains fixed as $n \to \infty$ (see Definitions 2.2.3 and 2.3.1). As we have seen, this locality allows for the emergence of sharp phase transitions, leading to the geometric shattering and algorithmic intractability characterized in Chapter 4.

In this chapter, we pivot to a fundamentally different class of problems: the random If-Then-Else (ITE) Tree ensemble, denoted as $\mathcal{T}(h, \beta, \Sigma)$. This is a *globally constrained* CSP, where the arity of the single constraint (i.e. the boolean function represented by the tree) grows with the system size m (see Definitions 2.2.3 and 2.3.2).

We investigate the global nature of this model through a series of empirical analyses. First, we examine whether this ensemble exhibits a satisfiability phase transition analogous to that of random 3-SAT, which leads us to discover instead that the probability of satisfiability in $\mathcal{T}(h, \beta, \Sigma)$ behaves differently. Next, we analyze the ensemble's capacity to retain information, focusing on its recursive structure and how it determines the probability of contingency. Finally, we shift towards a different domain of interest, focusing on individual variable influence and examining a possible phase transition.

It is important to note that, in this chapter, we do not attempt to categorize potential threshold phenomena using the criteria of Friedgut's Theorem (Theorem 2.4.5), as the global symmetry of the ITE ensemble fundamentally alters the nature of these transitions. Rather, our focus is on uncovering the ensemble's structural behavior and providing empirical evidence for the emergence of a continuous (coarse) phase transition. By contrasting the geometry of the locally constrained 3-SAT with that of the globally constrained ITE tree, we conclude our analysis by identifying how the transition from local to global constraints shapes the complexity of CSPs.

5.1 Investigating for a Satisfiability Threshold

In this section, the goal is to empirically determine whether the globally constrained $\mathcal{T}(h, \beta, \Sigma)$ ensemble exhibits a satisfiability phase transition. To do so, we replicate the methodology of our first experiment from Chapter 3, where we described a generator based on the definition of the random K-SAT CSP and used a modern SAT solver to sample and evaluate random instances. We proceed in a similar manner by defining a generator for the ITE tree. However, unlike the independent clause generation of k-SAT, we must now account for the recursive and symmetric structure of the boolean tree.

Recall our definition of the $\mathcal{T}(h, \beta, \Sigma)$ ensemble. Given a tree height h and a parameter β, we construct a regular ternary tree where every internal node is an $ITE(c, t, e)$ gate, mapping to the boolean operator:

$$ITE(c, t, e) \equiv (c \wedge t) \vee (\neg c \wedge e) \tag{5.1}$$

Because the tree is perfectly regular, we can exploit its structural symmetry to optimize the generation process. Instead of building the instance using a recursive function analogous to how the model is defined in Definition 2.3.2, we construct the tree using a bottom-up, layer-by-layer approach.

The generation begins strictly at the leaf layer. A $\mathcal{T}(h, \beta, \Sigma)$ instance, which is structurally a ternary tree of height h, contains exactly 3^h leaves. The generator then forms the leaf layer by uniformly and randomly sampling from the alphabet Σ. Recall that Σ always includes the truth constants $\{0, 1\}$ and the set of m variables, but also includes their negations, if the model is allowed to have negations.

Once the leaf layer is generated, the tree is evaluated upwards. Because every internal node at layer l strictly takes three inputs from layer $l + 1$, we can iteratively parse the current layer in chunks of three (representing the *condition* c, *then* t, and *else* e branches). This results in a new, (three times) smaller layer representing the parent nodes. This iterative reduction continues until only a single element remains: the root node of the tree, which decides the global evaluation of the formula.

5.1.1 Experiment Setup

To evaluate the satisfiability of the generated instances, we rely on the same state-of-the-art solver utilized in our earlier experiments, `CaDiCaL`. However, SAT solvers require the input formula to be in Conjunctive Normal Form (CNF). While our k-SAT generator naturally produced such formulas, our ITE generator produces a nested boolean circuit, so we will have to transform our final formulas into CNF. Due to the exponential scaling of the size of instances (3^h), it is also of great value to optimize the transformation as much as possible.

The Tseitin Transformation

Converting nested boolean circuits to CNF via naive applications of De Morgan's laws and the distributive property is highly inefficient [43]. Distributivity leads to duplication of sub-formulas, which turns to a worst-case exponential blow-up. While naively expanding a single ITE gate yields a small CNF, recursively distributing across a tree of height h causes an explosion in the number of clauses. Such a formula would quickly become intractable to store or solve.

To avoid this exponential blow-up, we apply the Tseitin transformation [44]. This transformation takes any boolean formula as input and encodes it into an *equisatisfiable* CNF, which is linear in the size of the input. That is, the new CNF formula is satisfiable iff the original one is also satisfiable. Instead of duplicating sub-formulas, the transformation introduces new auxiliary boolean variables to represent each of them. The representation is done by adding a new clause of equivalence between the auxiliary variable and the sub-formula. In our specific case, the transformation results in a new variable for every internal node in the tree. By doing so, we isolate the logic of each node, ensuring that the size of the resulting CNF formula grows linearly with the number of nodes in the original circuit. Example 6 presents an optimized Tseitin transformation for the encoding of an internal $ITE(c, t, e)$ node, adapted from [45]:

Example 6. We require an encoding that strictly enforces the equivalence $v_{out} \leftrightarrow ITE(c, t, e)$, where v_{out} is the auxiliary variable representing the output of the gate. This means that if the condition c is true, v_{out} must evaluate to t, and if c is false, v_{out} must evaluate to e. We can formalize this as the conjunction of two conditional equivalences:

$$(c \rightarrow (v_{out} \leftrightarrow t)) \wedge (\neg c \rightarrow (v_{out} \leftrightarrow e))$$

By expanding these implications using standard boolean identities, we can derive a compact CNF representation. Expanding the true branch gives:

$$\begin{aligned} c \rightarrow (v_{out} \leftrightarrow t) &\equiv \neg c \vee ((\neg v_{out} \vee t) \wedge (\neg t \vee v_{out})) \\ &\equiv (\neg c \vee \neg v_{out} \vee t) \wedge (\neg c \vee \neg t \vee v_{out}) \end{aligned}$$

Expanding the false branch gives:

$$\begin{aligned} \neg c \rightarrow (v_{out} \leftrightarrow e) &\equiv c \vee ((\neg v_{out} \vee e) \wedge (\neg e \vee v_{out})) \\ &\equiv (c \vee \neg v_{out} \vee e) \wedge (c \vee \neg e \vee v_{out}) \end{aligned}$$

Combining these results, we encoded the ITE gate using exactly four clauses and a single auxiliary variable v_{out}:

$$\begin{aligned} &(\neg c \vee \neg t \vee v_{out}) \\ &(\neg c \vee t \vee \neg v_{out}) \\ &(c \vee \neg e \vee v_{out}) \\ &(c \vee e \vee \neg v_{out}) \end{aligned} \tag{5.2}$$

During the bottom-up generation, as we parse tuples of three inputs (c_i, t_i, e_i), we assign a new distinct auxiliary variable v_i and add these four clauses to our global CNF formula.

Constant Folding

To further optimize the encoding of the formula, we make use of the fact that our alphabet *Sigma* always contains the truth constants 0 and 1. Namely, we perform *constant folding* during generation of each internal node. That is, if the condition branch c evaluates directly to one of the constants, the $ITE(c, t, e)$ gate collapses to either t or e. We can then skip the creation of the auxiliary variable and the additional clauses, only having to pass the active branch (whichever one the gate collapses to) to

the parent layer. If t and e are identical, then the condition c is irrelevant, and the branch value is passed upward.

Since the DIMACS format does not support truth constants, we encode the 0 and 1 values as two variables c_0 and c_1 respectively, and we fix their boolean values by adding the (c_0) and $(\neg c_1)$ unit clauses to the final formula. Additionally, to enforce that the globally constrained formula must be satisfied, a unit clause containing the final root variable is appended to the CNF. For the purposes of this experiment, we will restrict the alphabet Σ to the non-negated variables and the constants. As we will see, this restriction has no bearing on the underlying behavior of the distribution in our results.

Now that both the generator for our $\mathcal{T}(h, \beta, \Sigma)$ instances and the optimized Tseitin transformation to CNF are formalized, we can proceed to use the rest of the experimental methodology as we have used it in our experiment in Section 4.2.

5.1.2 Results and Interpretation

One important issue we have not yet formally addressed is the actual scaling relation of the control parameter $\beta(m, h)$. In contrast to the rigorously studied random k-SAT problem, where the well-known clause density α is universally accepted, our specific $\mathcal{T}(h, \beta, \Sigma)$ ensemble does not seem to have an established equivalent parameter in the literature. Therefore, from a purely experimental perspective, we test with two potential scaling relations.

Let us first assume a scaling relation which corresponds to the size of the tree, i.e. $\beta \propto 3^{-h}$. In particular, we take $\beta = m/3^h$ for our first analysis. This ratio intuitively represents the *variable density* (or coverage) of the instance. It is the ratio of distinct variables in the pool to the number of leaves in the tree. When $\beta \ll 1$, variables are heavily repeated across the leaves, and when $\beta \gg 1$, the pool is large enough that most variables appear at most once.

Figure 5.1 depicts our results under this scaling parameter. Focusing first on the top plot, we instantly observe a fundamentally different probability distribution compared to that of $F_k(n, m)$.

The most prominent difference is the complete absence of a sharp transition to an unsatisfiable phase. Instead, all probabilities remain bounded within the sub-range $[0.5, 1]$. To understand this behavior, we must look at the origin where $\beta = 0$, and thus $m = 0$. At this point, the alphabet Σ only contains the boolean constants $\{0, 1\}$. Because our generator samples symbols uniformly from Σ, an instance at $\beta = 0$ is simply a deterministic boolean circuit composed entirely of constants, essentially a complex way of writing the constant 0 or 1. By the symmetric nature of the ITE operator, such a tree evaluates to 1 exactly 50% of the time, which is why all distributions at different heights begin at $\Pr[\text{SAT}] = 0.5$.

Another distinct visual feature is the discontinuity of the distributions. At $h = 0$, the tree size is $3^0 = 1$. Then $m = \lfloor \beta \cdot 1 \rfloor$, meaning the variable pool size only increments once per unit of β, resulting in large horizontal steps. By $h = 4$, the tree size is $3^4 = 81$, and so m updates 81 times per unit of β. This means that the discontinuous nature of our distributions has to do with finite-size effects resulting from the low heights we tested, and our control parameter scaling. In our k-SAT experiments, the parameter $\alpha = m/n$ (with m the number of clauses in the formula) was tested against relatively

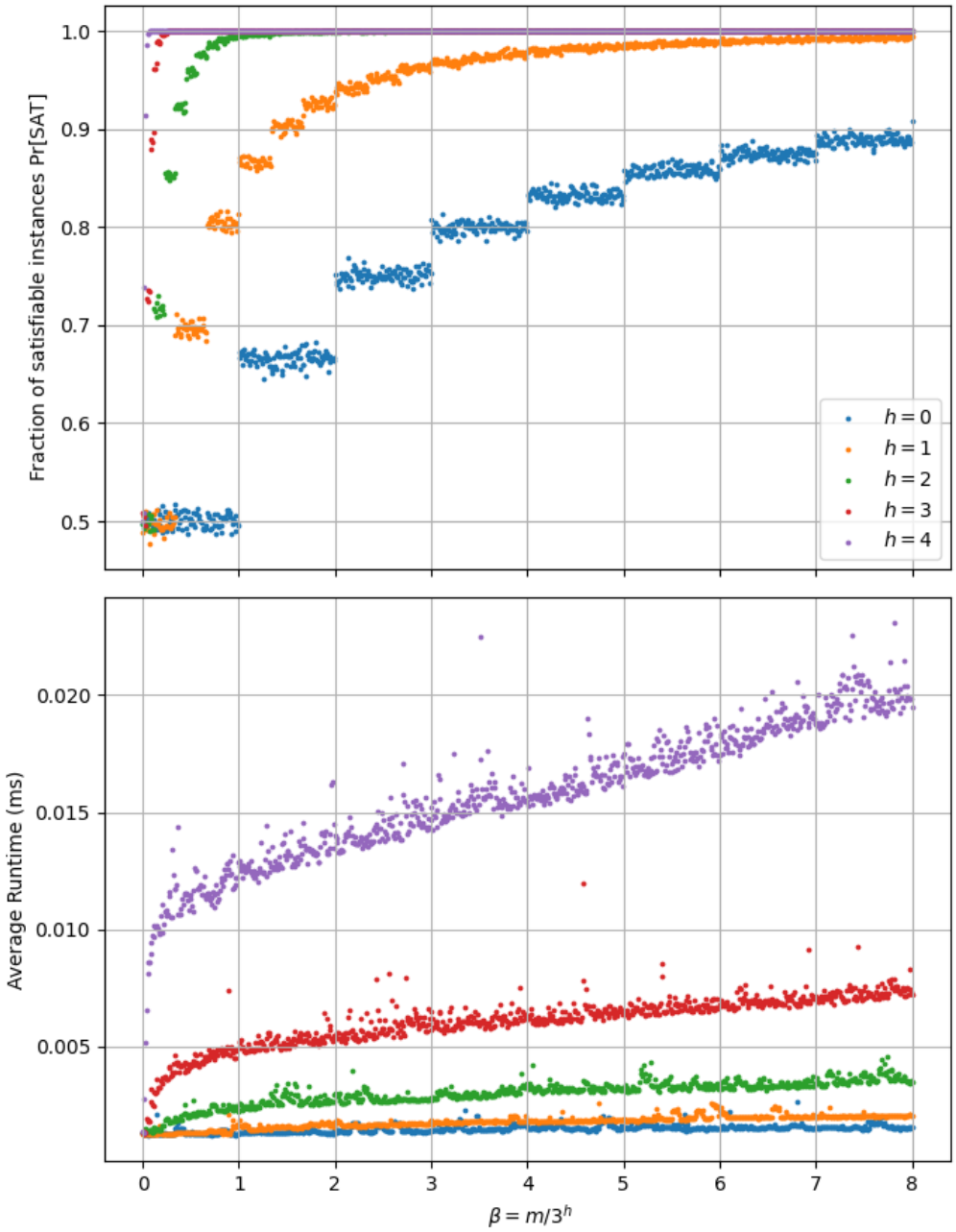


Figure 5.1: **Top:** The probability Pr[SAT] that a random ITE tree instance is satisfiable, measured for the tree heights $h \in \{0, 1, 2, 3, 4\}$ and the control parameter $\beta = m/3^h$, with m the variable pool. Notice that, for all heights, the probability starts at Pr[SAT] $= 0.5$, and that for the heights $h = 3, 4$, almost all points are at Pr[SAT] $= 1$. Each data point has been computed using 5000 samples. **Bottom:** The average solver runtime (in milliseconds) evaluated over the same parameter space. Notice that some data points in the bottom plot are relatively far from the general trend line of their respective heights, however, given the incredibly small time frames (hundredths of a millisecond), these points are not significant.

large n (e.g. $n = 100$), meaning the number of clauses m updated 100 times per unit of α, creating a smooth curve. This is what we observe, although with difficulty, in our current results' distribution as well. As h increases, we see the large discontinuous steps of the data points decrease (by a factor of 3). At $h = 3, 4$, we essentially have a continuous curve, however, since both curves almost instantly converge to probability 1, this behavior is difficult to observe.

Furthermore, the $h = 0$ case easily allows us to verify our generator. As mentioned before, we restricted the alphabet Σ to the non-negated variables and constants. Since the instances are structurally symmetric and we sample symbols uniformly, including negations in this case would not alter the results significantly. We would observe each of the discontinuous steps to be slightly higher due to more possible instances now being satisfiable, however, as we have already seen the asymptotic behavior of the distribution, this would not change our interpretation. Under this alphabet of size $m + 2$, a single-leaf formula (i.e. a node) is unsatisfiable iff it evaluates to the constant 0. Because the generator samples uniformly, the probability of drawing the constant 0 is exactly $1/(m + 2)$. Thus, for $h = 0$, the satisfiability probability is

$$\Pr[\text{SAT}_{h=0}] = 1 - \frac{1}{m+2} = \frac{m+1}{m+2}$$

This matches our empirical data, which we observe to be around 0.5 at $m = 0$, and $2/3$ at $m = 1$. Note then that if negations were allowed, the alphabet size would increase to $2m + 2$, which would merely decrease the probability of drawing the constant 0 and slightly shift the probabilities upward.

Looking now at the bottom runtime plot, we observe other discrepancies compared to our random k-SAT results. For any fixed height h, the runtime increases linearly with variable density β. An interesting characteristic of this distribution is the scaling between the lines of each height. We observe what seems to be an exponential scaling of the slope of each runtime line itself. We attribute this behavior to how modern SAT solvers manage memory. The number of clauses generated by the Tseitin transformation is of order $O(3^h)$ regardless of the variable density, since new auxiliary variables (and clauses) are added per internal node, no matter how many variables our pool contains. However, the maximum variable identifier in the formula increases as m increases. `CaDiCaL` dynamically allocates arrays based on the maximum variable index encountered [31]. Thus, the initialization memory overhead scales as $O(m)$. Assuming then a runtime proportional to $m = \beta \cdot 3^h$, as β increases, the slope of the runtime lines is then proportional to 3^h, which seems to explain the exponential scaling of the runtime.

The results in Figure 5.1 indicate two potential conclusions. On one hand, while $\beta = m/3^h$ parameter scales the structural density of the variables, it fails to produce a satisfiability phase transition. On the other hand, a more compelling explanation of the absence of this phase transition has to do with the actual ensemble itself. In particular, we saw that the probability of unsatisfiability for this model, regardless of the control parameter, scales like $1/m$. The more variables we add, no matter how slowly we do so, the closer we are to satisfiability w.h.p. Not only that, but allowing negations into our alphabet only amplifies this effect, determining the probability of unsatisfiability to scale like $1/2m$. Finally, the fact that the alphabet always contains the two truth constants, uniformly sampled, definitively bounds the probability in question to a minimum value of 0.5. This tells us that the ensemble's satisfiability threshold being absent is intrinsic to the model itself, not on the control parameter.

To provide additional evidence for the inherent absence of the threshold, we change the scaling relation of the control parameter to $\beta = m/h$, making it (inversely) proportional to the height itself, rather than the size of the tree. Figure 5.2 represents the distributions we find under this parameter. The top plot presents us with a familiar probability distribution: for all heights, we start at probability 0.5 and converge to probability 1. Notice now that the $h = 0$ distribution is removed, since under this parameter relation, this height is not allowed. We seem to find a more or less "zoomed in" version of the distribution we observed in Figure 5.1, which is expected, given the much more slowly scaling $1/h$ factor (as opposed to $1/3^h$). Looking at the bottom plot, we again find almost the same behavior as we did previously. We find an exponential scaling between the lines corresponding to increasing heights, only now, the slope of the lines grows slower. Assuming our explanation for the previous plot, this is because we are not scaling the data by the size of the tree anymore.

We have now tested two differently scaling control parameters, neither of which resulted in a satisfiability phase transition. On top of that, neither of our results presented a runtime scaling analogous to the "easy-harder-less hard" pattern we found when experimenting with the random 3-SAT problem. From this data, we then conclude that the structure of the tree instances in the $\mathcal{T}(h, \beta, \Sigma)$ ensemble are inherently satisfiable and show no satisfiability (sharp) threshold.

This inherent bias towards satisfiability can also be understood intuitively by analyzing the mechanisms that cause a boolean formula to become unsatisfiable. For any formula to be unsatisfiable, it must evaluate to 0 for all 2^m possible variable assignments, i.e. it must be a global contradiction. In the $\mathcal{T}(h, \beta, \Sigma)$ ensemble, an assignment only evaluates to unsatisfiable if its corresponding path through the recursive path through ITE gates hits a dead end. There are two ways a path can die: it either terminates at a leaf labeled with the constant 0, or it encounters a variable conflict along the way (e.g. a path that requires $x_i = 1$ at one node, but later requires $x_i = 0$ to continue).

While increasing the variable pool m and the depth h does indeed generate more variable conflicts, it also generates exponentially more branching paths. Because the ITE gates route information symmetrically, the exponential increase of paths completely overcomes the rate of path-deaths caused by conflicts. As long as at least one path avoids conflicts and terminates in a 1 (or in a variable which does not contradict previous assignments), the entire tree remains satisfiable.

However, establishing that the ensemble rarely collapses to a global contradiction raises a question regarding its internal logic. At large depths h and low m, the few variables available in the pool are forced to repeat frequently in the paths. Intuitively, this should result in a much more constrained tree, potentially turning into an unsatisfiable one as h continues to increase. Looking at our results, however, seems to contradict this intuition. We see in both Figures 5.1 and 5.2 that even for low β, the distributions for increasing heights quickly shoot to $\Pr[\text{SAT}] = 1$. Even with massive variable repetition, the instances are almost always satisfiable, indicating that conflicts alone cannot pull the formula down to a contradiction.

This (seemingly) counter-intuitive satisfiability persistence leaves us with a fundamental question. If the exponential branching outpaces the rate of conflicts, the tree might remain *contingent*, meaning that its final evaluation depends on the specific assignment of its variables. Therefore, the absence of a satisfiability threshold leads us to ask: do $\mathcal{T}(h, \beta, \Sigma)$ instances retain their contingency as $h \to \infty$, or do variable conflicts

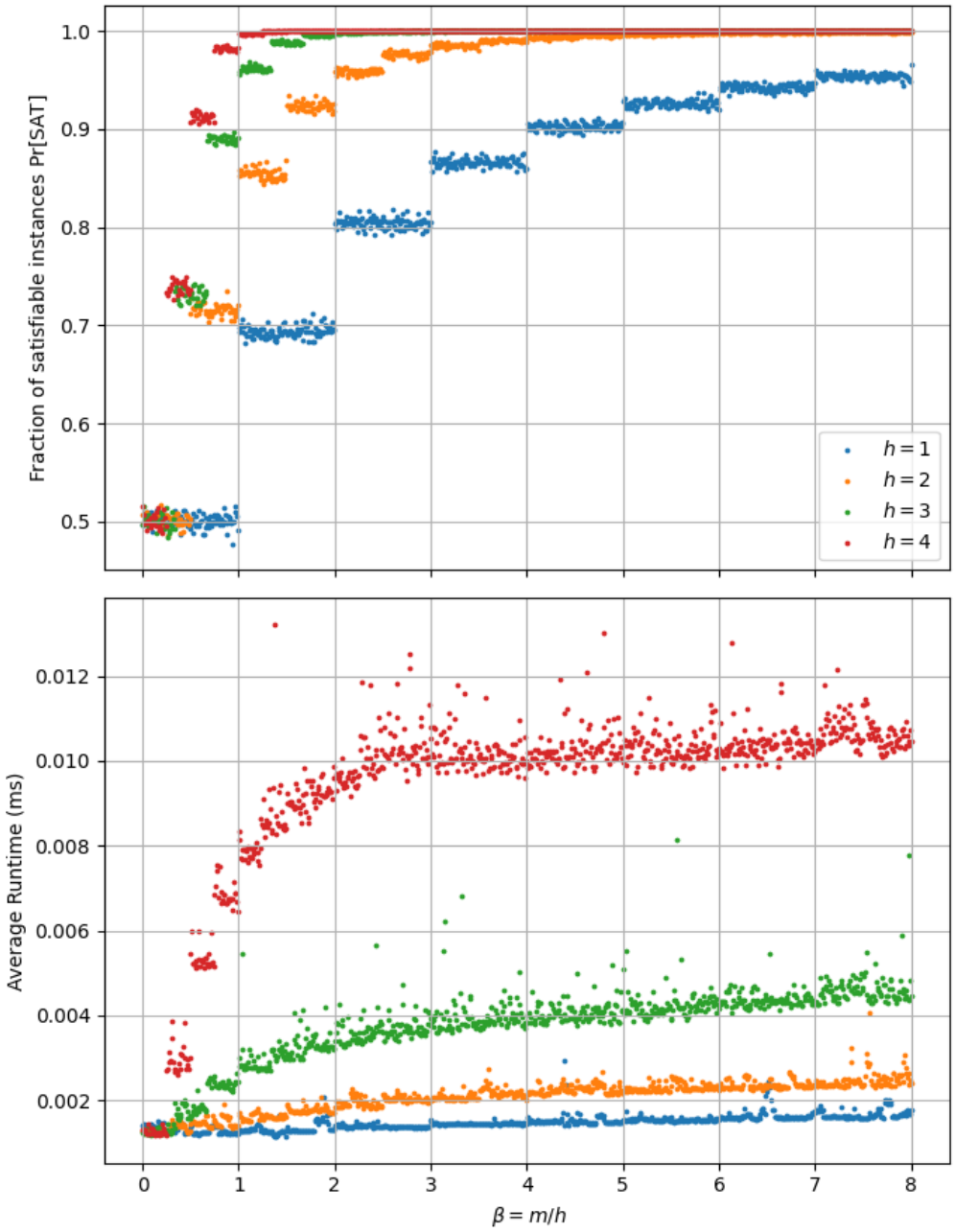


Figure 5.2: **Top:** The probability Pr[SAT] that a random ITE tree instance is satisfiable, measured for the tree heights $h \in \{1, 2, 3, 4\}$ and the control parameter $\beta = m/h$. **Bottom:** The average solver runtime (in milliseconds) evaluated over the same parameter space. Each data point has been computed using 5000 samples.

cause all contingency to disappear, leaving only deterministic instances? As we will see, our intuition does not account for the symmetry of *contradiction* (where all paths lead to a 0) and *tautology* (where all paths lead to a 1). To answer this, we shift our methodology from checking for satisfiability to measuring the global probabilities of tautology, contradiction and contingency of the model.

5.2 The Contingent Behavior of the ITE Ensemble

In the previous section, we empirically discovered that the random ITE tree ensemble does not exhibit a satisfiability phase transition. In fact, the probability distribution quickly reaches $\Pr[\text{SAT}] = 1$ as the variable density β increases. It is this rapid convergence to satisfiability that drives us to look for the model's intrinsic contingency, in the limit of the height h. In particular, we are interested in the cases where the variable pool m is very low and the height h is large. Essentially, if we do not observe an increase in tautological and contradictory trees at these values, then we have determined definitively that for larger variable pools and/or lower heights, contingent tree instances (whose evaluation depends on the assignment of variables) vastly outnumber the rest.

It is important to understand the three types of tree instances we will be analyzing. A tree is a tautology if it evaluates to 1 no matter the assignment of variables in the leaves. Similarly, if a tree is a contradiction, then it evaluates to 0, no matter the assignment, which is equivalent to unsatisfiability. The only other possible kind of tree is a contingent one, where the tree can evaluate to 1 or 0, depending on the variable assignment. For this last type of tree, the variables in the leaves actually influence the tree's evaluation. Since these three cases are exhaustive, for our ensemble we thus have that

$$\Pr[\text{UNSAT}] + \Pr[\text{TAUT}] + \Pr[\text{Contingent}] = 1 \tag{5.3}$$

where $\Pr[\text{UNSAT}]$ and $\Pr[\text{TAUT}]$ are the probability of the random instance being a contradiction and that of the random instance being a tautology, respectively. Importantly, we also have that $\Pr[\text{TAUT}]+\Pr[\text{Contingent}] = \Pr[\text{SAT}]$, since for any contingent tree, there must be an assignment evaluating the global formula to 1. With this in mind, we proceed to analyzing how these three probabilities vary with low variable pools m and increasing heights h.

5.2.1 Experiment Setup

In this experiment, we want to determine the three probabilities of the $\mathcal{T}(h, \beta, \Sigma)$ ensemble from Equation 5.3. However, as we have discussed, it is worth focusing on the instances for which β is low, i.e. there are very few variables available in our alphabet and the height of the tree is relatively large. Therefore, we test the ITE ensemble on fixed, low m. Instead of scaling through our control parameter β, we thus simply analyze how the probabilities vary as h increases. Additionally, this experiment will be conducted using both the restricted alphabet Σ we used previously, and the extended alphabet which includes variable negations (of size $2m + 2$).

As we have done in the previous experiment, we generate random $\mathcal{T}(h, \beta, \Sigma)$ tree instances (only now, m is fixed) and Tseitin transform them into CNF formulas which `CaDiCaL` can solve. However, SAT solvers are designed to answer a single question: is the formula satisfiable? Therefore, to classify an instance into one of the three mutually exclusive states, we must query the solver twice per generated instance. First, we

pass the formula F to the solver. If it returns UNSAT, the formula is definitively a contradiction. If it returns SAT, we must determine whether the formula is a tautology or contingent. To do so, we negate the global evaluation of the tree to produce $\neg F$, and pass this negated formula to the solver. If $\neg F$ returns UNSAT, it implies that the original formula F cannot evaluate to 0, and is therefore a tautology. If both F and $\neg F$ return SAT, the formula is contingent.

5.2.2 Results and Interpretation

To acquire a good understanding of the ITE ensemble's contingency, we first evaluate the most restricted case, $m = 1$ (for $m = 0$, the tree instances simply cannot be contingent, since there are no variables). If the trees end up destroying contingency anywhere, it will be most evident here.

Figures 5.3a and 5.3b display the estimated probabilities of the three logical states as the tree height h increases, using the restricted alphabet without negations and the extended alphabet with negations, respectively.

The first interesting feature we observe in both plots is the perfect symmetry between the deterministic states. In both plots, the contradiction probability (Pr[UNSAT]) mirrors and overlaps the tautology probability (Pr[TAUT]). This provides empirical validation of the model's structural symmetry. Because the generator we defined samples the boolean constants 0 and 1 from the alphabet with equal probability, any mechanism that forces a tree to collapse to 0 has an equally probable, symmetric counterpart that forces it to 1. This alone refutes the idea that increasing variable conflicts turn the tree unsatisfiable with increasing height, exactly because of this completely symmetrical structure. Consequently, the deterministic decay of the tree is perfectly balanced.

However, the most significant theoretical result lies in the asymptotic behavior of the contingency curve (Pr[Contingent]). Despite the high variable repetition forced by $m = 1$, the contingency resists collapsing to zero. The exact nature of its survival depends on the alphabet Σ: when negations are not allowed, i.e. $\Sigma = \{0, 1, x\}$ (for some variable x), the starting probability at $h = 0$ is $1/3$ for all three states (see again Figure 5.3a). This is the expected result since, at $h = 0$, all instances are just a single node, to which the generator can assign one of three symbols. As the depth increases, the probability of contingent tree instances rises, eventually converging to a stable point of approximately 0.55. When negations are allowed, i.e. $\Sigma = \{0, 1, x, \neg x\}$, the starting probabilities are 0.5 for contingent instances and 0.25 each for contradiction and tautology trees (Figure 5.3b). Again, this is expected since now there are two symbols which result in depth-0 trees to be contingent (the variable and its negation). However, as the tree depth increases, the contingency curve remains flat at 0.50 across all heights.

This $m = 1$ equilibrium state gives us a clearer understanding of the high satisfiability we observed in our previous experiment. If a single variable in the pool is enough to perfectly preserve contingency against the exponential depth of the tree, adding more variables makes the tree overwhelmingly contingent. If we slightly relax the constraint and increase the variable pool to just $m = 2$, Figure 5.4 shows that the probability of contingency converges up to over 0.80, for both the restricted alphabet (Figure 5.4a) and the symmetrical one (Figure 5.4b).

Another interesting feature in these results is that, for both $m = 1, 2$, the contingency probabilities seem to settle slightly higher when we do not allow negations (with the

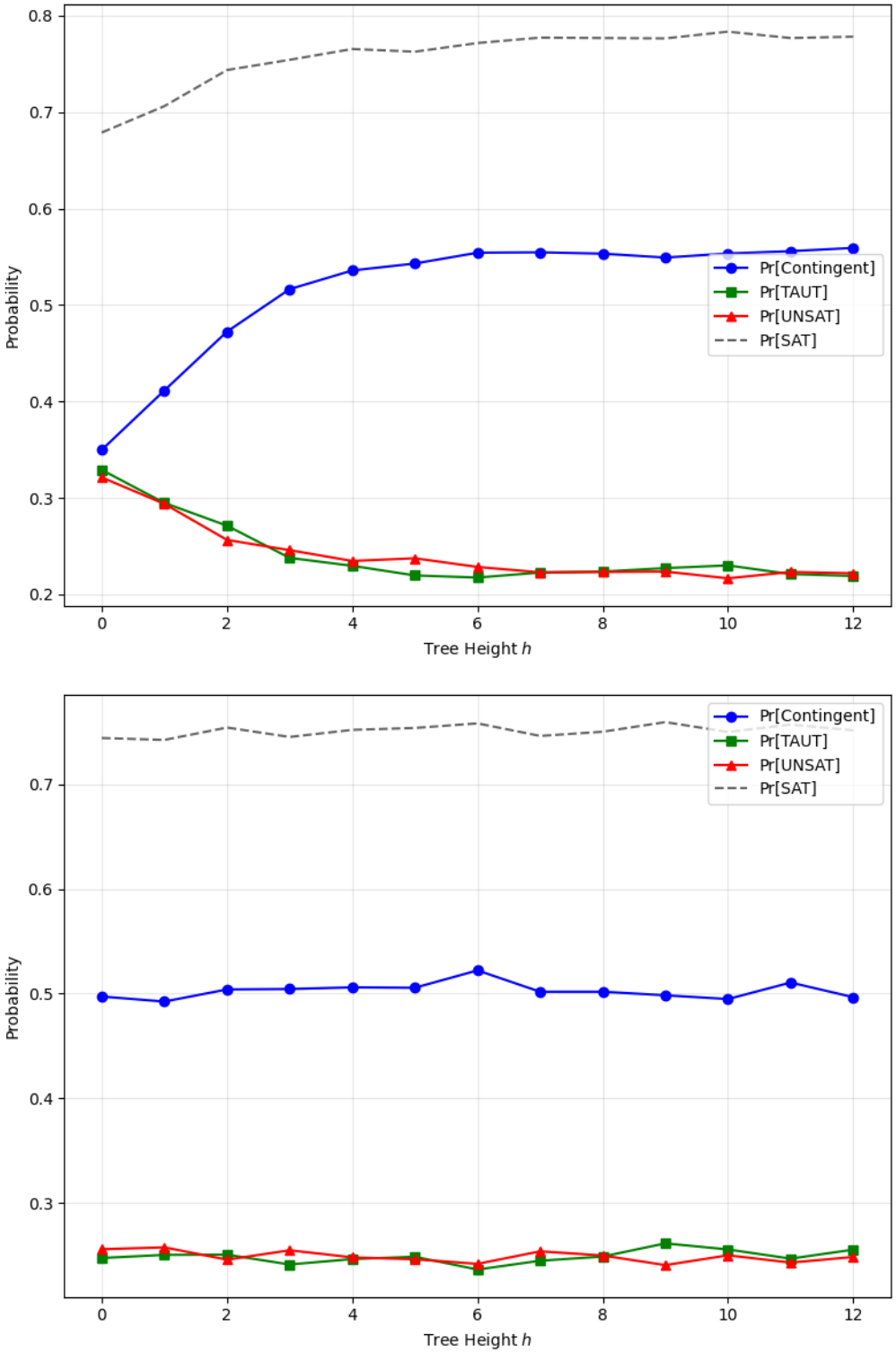


Figure 5.3: **Top:** The estimated probabilities of contingency, tautology and contradiction (Pr[Contingent], Pr[TAUT] and Pr[UNSAT] respectively) for increasing height $h \in \{0, 1, \ldots, 12\}$, with $m = 1$ and using the restricted alphabet without negations. The dotted line represents the probability of satisfiability, which is the sum of the contingency and tautology probabilities. **Bottom:** The same estimated probabilities over the same heights, this time allowing negations (with the symmetrical alphabet).

restricted alphabet). Firstly, this suggests that this asymmetry of the alphabet results in an asymmetry in the evaluation of the trees. Intuitively, this can be explained by the fact that, even though the restricted alphabet does not contain negated variables, it is still possible for "artificial" negations to arise from a specific ITE configurations, such as $ITE(x, 0, 1)$. As these resultant negations require a prior configuration, we have an asymmetry between the easily obtainable, uniformly sampled negated variables (when they are allowed) and the "artificial" negated variables (when the simple negated ones are not allowed). Secondly, this asymmetry seems to be invariant under the tree height h, as we can see that the slightly higher values for the top plots of Figures 5.3 and 5.4 remain fixed as the height increases.

As m continues to grow, the probability of generating a contradiction or a tautology tree vanishes. Because Pr[Contingent] + Pr[TAUT] = Pr[SAT], and because the contingency rapidly approaches 1 as m increases, the fraction of satisfiable instances is essentially guaranteed to be 1. Therefore, the $\mathcal{T}(h, \beta, \Sigma)$ ensemble seems to preserve global contingency, so the instances avoid collapsing into a deterministic constant and remain dependent on their assignments. However, establishing this does not mean that the instances are contingent on any *individual* variable. To understand whether any one of the variables in the trees are significant under the ensemble's inherent contingency, we must examine the model in terms of individual variable influence.

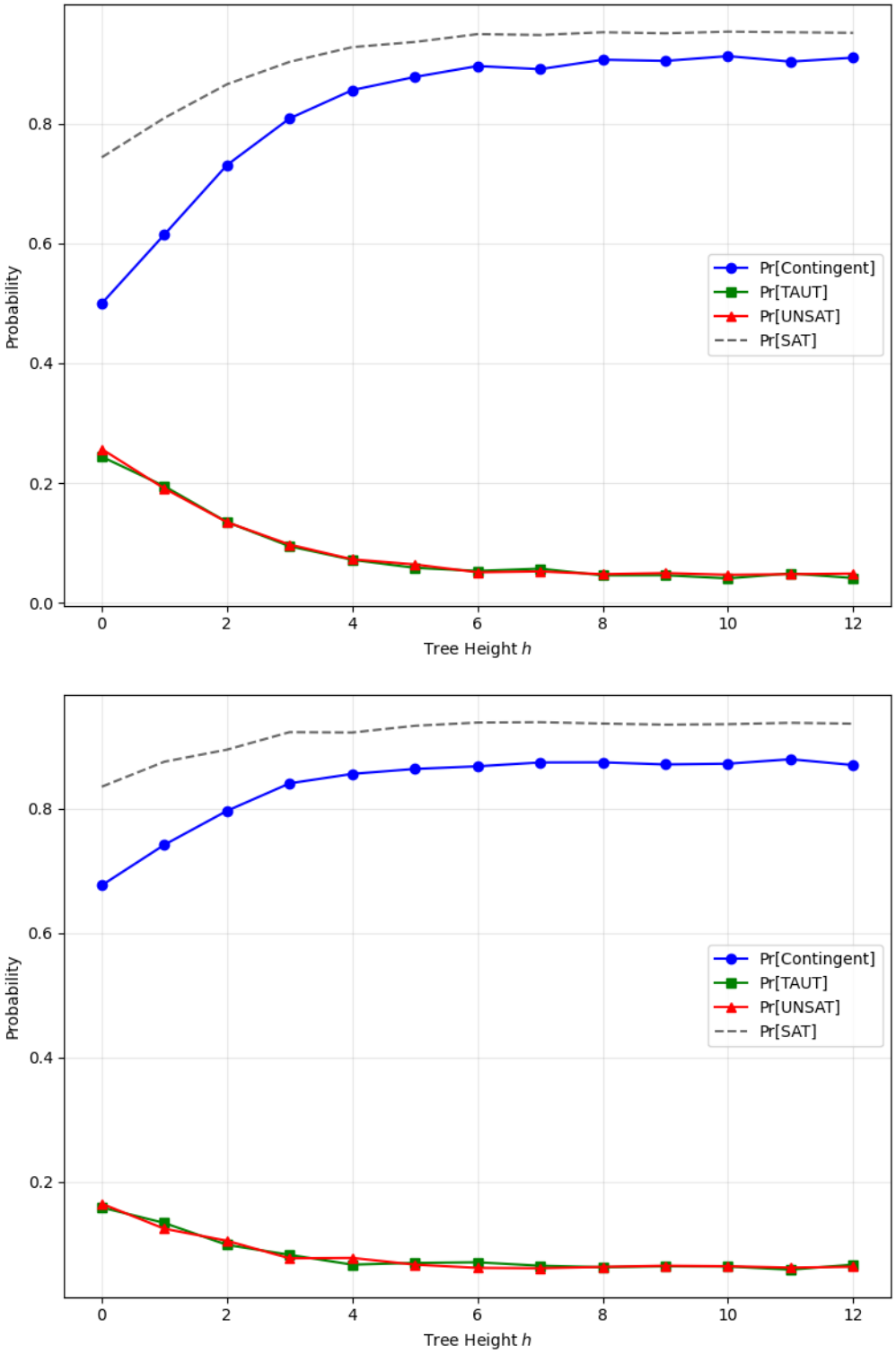


Figure 5.4: **Top:** The same estimated probabilities Pr[Contingent], Pr[TAUT], Pr[UNSAT] as the ones measured in Figure 5.3 for increasing height $h \in \{0, 1, \ldots, 12\}$, with $m = 2$ and using the restricted alphabet without negations. The dotted line represents the probability of satisfiability, which is the sum of the contingency and tautology probabilities. **Bottom:** The estimated probabilities over the same heights, with negations allowed.

5.3 The Variable Influence Phase Transition

The results of our previous experiments in this chapter lead us to conclude the following: the $\mathcal{T}(h, \beta, \Sigma)$ ensemble does not experience a satisfiability phase transition and is inherently contingent, meaning that, no matter how restrictive our scaling relations become, the system still preserves its ability to make instances evaluate to 1 under some assignment, and make those same instances evaluate to 0 under others. However, this does not mean that any *single* variable is the deciding factor in changing the truth evaluation of any instance. Intuitively, for trees of fixed height, as the variable pool size m increases, any individual variable should become less and less influential on the entire truth evaluation of the tree. Therefore, we now focus on analyzing individual variable influence over the random ITE ensemble to determine whether we are presented with a transition.

We refer to how we previously defined individual variable influence (see Definition 2.4.7), and apply our generator terminology to it. For a generated $\mathcal{T}(h, \beta, \Sigma)$ tree instance T, the influence of variable x_i tells us how likely is it that flipping the assignment of x_i results in flipping the truth evaluation of the root of T, i.e. the global evaluation of the tree.

We will thus investigate the expected individual variable influence of this ensemble. Note that this analysis is purely empirical, and we do not attempt to present a concrete interpretation of the data, although possible explanations will be given.

5.3.1 Experiment Setup

To begin our experiment we first recognize that, due to the uniform sampling of symbols from our alphabet onto the symmetrical tree instances, the expected influence of any individual variable is identical (as noted in Section 2.4.4). As such, Monte Carlo sampling assignments where we only flip a specific variable's evaluation constitutes a reliable statistical method to finding the expected variable influence of any variable. We will thus take a target variable, namely, the first one in our pool, and experiment flipping its assignment.

A second important aspect to recognize is that, as we are interested only in how a variable changes the global evaluation of the tree, satisfiability is not of interest anymore. This means that we do not need to use a SAT solver, but only to actually find the root evaluation of the instance, regardless of whether it is 0 or 1. Therefore, we can simply apply our layer-by-layer evaluation method from previous experiments and simply output the truth value of each node we encounter. Recall that, due to the ternary regular structure of instances, we know that each 3-tuple of elements from the current layer forms the input of one element of the layer above it. Then, simply evaluating the ITE gates as we normally would:

$$ITE(c, t, e) = (c \wedge t) \vee (\neg c \wedge e) \tag{5.4}$$

and passing the output upwards to the next layer, is all we need to do. Performing this evaluation again for the same instance, but for an assignment where the target variable evaluation is flipped, tells us whether the variable had influence over the root node's truth value. Since we bypass the need to Tseitin transform the formulas and hand them to a SAT solver, we have the resources to sample a larger amount of instances at larger heights.

5.3.2 Results and Interpretation

We begin by sampling instances with linear scaling of the variable pool m. Figure 5.5 presents the influence for a fixed tree height $h = 8$ as m increases linearly in increments of 100. Unsurprisingly, the data displays a rapid decay in variable influence. As the variable pool grows, the probability of the target variable appearing in "critical" influential positions decreases, diluting its signal.

However, it is important to recognize that the total number of leaves in the tree scales exponentially as 3^h, so evaluating linear increments of m provides only a microscopic view of the evolution of influence. To observe the macroscopic behavior of the ensemble, we must scale the variable pool exponentially alongside a logarithmic evaluation axis.

We analyze both the fully symmetric alphabet (allowing negations) and the restricted alphabet to isolate any change in the distribution attributable to the alphabet's contents.

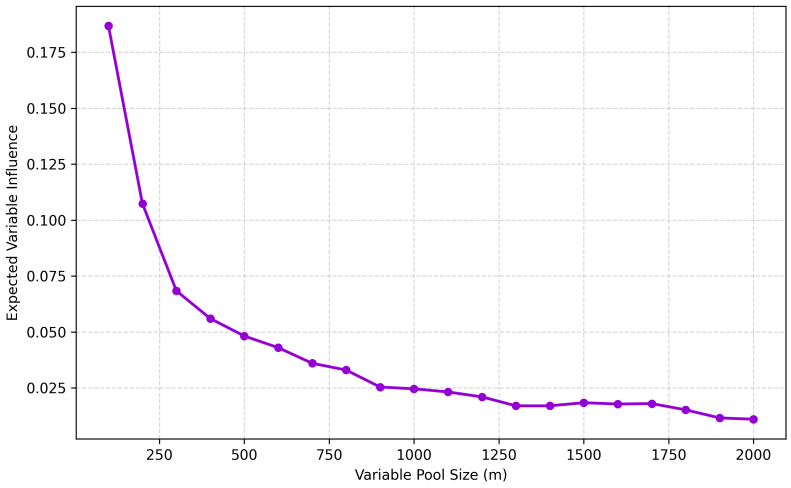


Figure 5.5: The expected individual variable influence plotted against the variable pool sizes $m \in \{100, 200, \ldots, 2000\}$, for random instances of height $h = 8$. Each data point has been computed with 5000 samples.

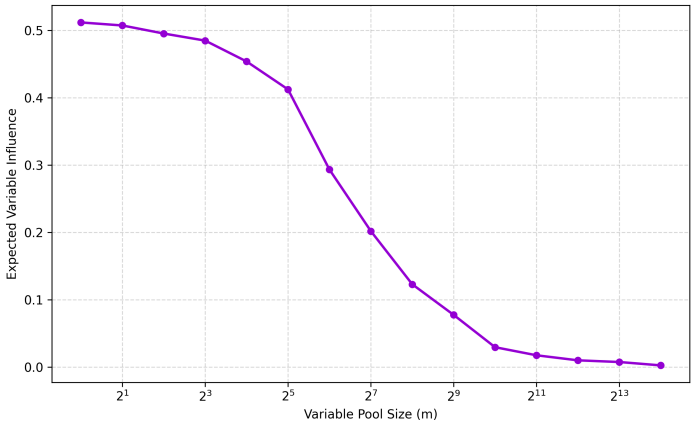
Testing The Symmetric Alphabet

We first examine the ensemble where negations are allowed, for fixed heights h and exponentially increasing pool sizes m. As seen from Figure 5.5, we know that the critical window, if it exists, must be located at low values of m where we go from relatively high individual influence to relatively low ones. For this reason, as mentioned, we logarithmically scale the plot (along the x-axis of increasing m), so that we "zoom in" on the potential phase change.

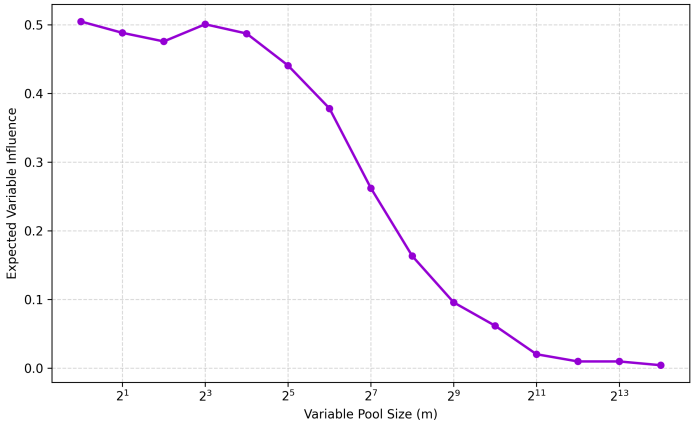
Figure 5.6 displays the individual variable influence for fixed tree heights as the variable pool m scales exponentially. The resulting data seems to form a near-perfect sigmoid curve, characteristic of a (continuous) phase transition. At low variable pool sizes ($m < 32$), the individual influence plateaus exactly at 0.50. At the extreme low point $m = 1$, this data corroborates our experiment results from Section 5.2. Since the expected influence of a variable for $m = 1$ is equivalent to the probability of contingency of the instance, the 0.50 value is expected. For slightly larger m, we do not see the sudden

increase in influence as we have seen for the probability of contingency, because the individual influence is only in reference to a single variable.

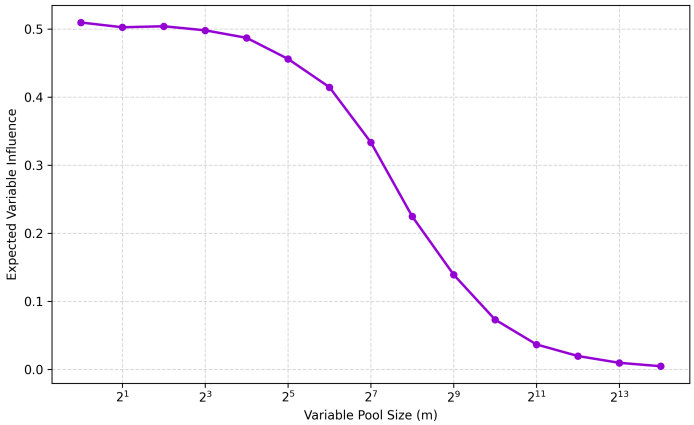
However, as m continues to increase (exponentially), we find that the influence drops quickly. The number of distinct variables competing with our target variable becomes exponentially large. The probability of a specific variable appearing on the evaluated contingent path drops drastically, and so its individual influence rapidly decays toward zero.



(a)

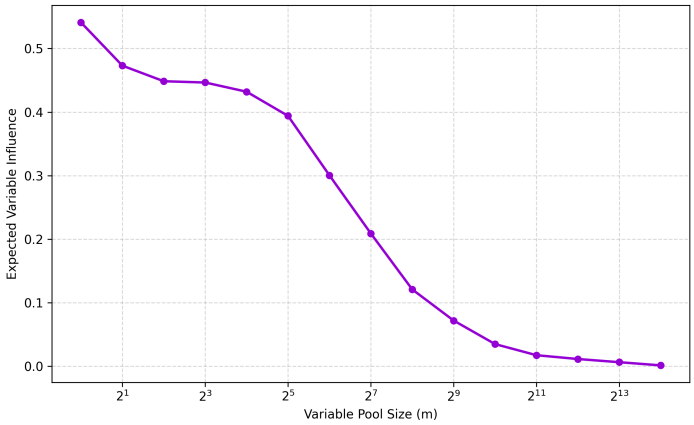


(b)

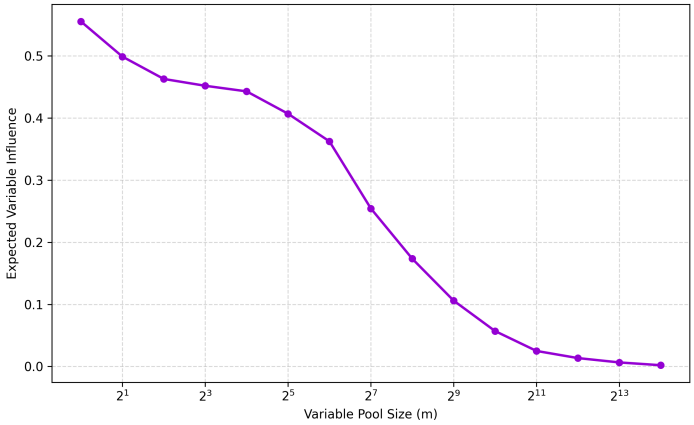


(c)

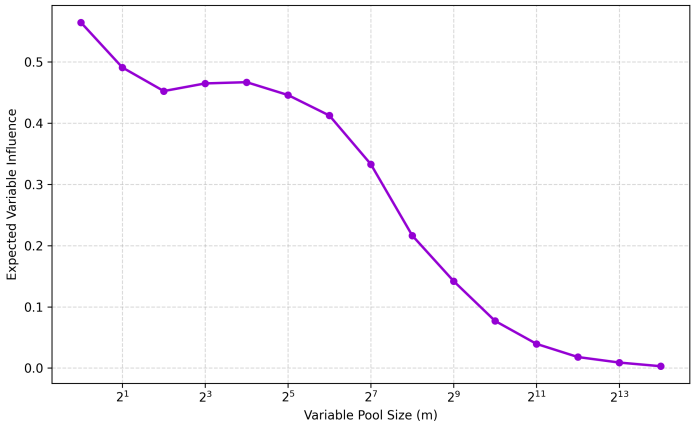
Figure 5.6: The expected individual variable influence plotted against exponentially increasing pool sizes $m \in \{2^0, 2^1, \ldots, 2^{14}\}$. **(a)**, **(b)** and **(c)** present the sampled distributions for $h = 9, 10, 11$ respectively, and negations were allowed in the alphabet for all instances. Notice that the curves all start at the expected 0.5 probability and seem to be in a "phase" of relatively high influence over the evaluation of the tree. As m increases exponentially, we find the curves dropping to a region of relatively low influence. Note that the plots are semi-log scaled.



(a)



(b)



(c)

Figure 5.7: The expected individual variable influence plotted as in Figure 5.6, only now the instances have been generated using the restricted alphabet without negations. **(a)**, **(b)** and **(c)** again present the sampled distributions for $h = 9, 10, 11$, respectively. We can see that all curves begin at the expected ≈ 0.55 probability, however, they all plateau quickly at ≈ 0.47. The rest of the curves seem to have a similar behavior to that of the curves in Figure 5.6. The "bump" observed in **(c)** at $m \approx 2^4$ seems to emerge simply as a local effect.

Testing The Restricted Alphabet

Applying the same exponential scaling to the restricted alphabet yields a similar macroscopic transition, but with notable deviations, as seen in Figure 5.7.

Rather than a perfectly flat initial plateau, the influence begins at 0.55 (aligning with the bound from our previous experiment with the restricted alphabet) and only then lowers at what seems to be a similar plateau. We additionally observe what might be a slight local "bump", before eventually influence decreases. Despite this initial change, the later sigmoid shape of the macroscopic transition seems to remain.

Finite-Size Scaling and the Control Parameter

While the phase transition is visible for the tested heights, we observe its form shifting as the tree height h increases. To provide substantially better evidence that this dilution is a universal property of the ensemble and not simply a local feature of low-height instances, we use the *finite-size scaling* (FSS) methodology [38] which we discussed in our random k-SAT experiments (Section 4.2.1). If the system undergoes a phase transition, plotting the variable influence against a correctly scaled control parameter $\beta(m, h)$ will cause the curves from different system sizes of h to collapse onto a single universal curve.

Initially, one might naturally assume that the scaling parameter should be the variable density relative to the total number of leaves, $\beta = m/3^h$, as this would correspond to a conflict between adding constraints (populating the tree with more variables) and releasing them (increasing the height to make a more dependent tree instance). However, as Figure 5.8 illustrates, plotting the influence against this parameter for all tested heights fails to align the curves. We see curves for lower heights drifting to the right of the scale, while those for larger heights drift to the right. This implies that scaling by 3^h is too aggressive, and the actual number required to maintain influence does not grow as fast as the structure of the tree.

Through an empirical analysis, we can identify the critical scaling exponent for β to be 1.5. When the control parameter is redefined as $\beta = m/1.5^h$, the disparate influence curves across all measured heights seem to align almost perfectly into a single curve path, as demonstrated in Figure 5.9 (for both restricted and non-restricted alphabets).

This data collapse, we conjecture, is a strong empirical result, as it isolates the phase transition from any finite-size noise. Importantly, it provides compelling evidence that the decay of individual variable influence is a size-invariant property (in terms of h) of the random ITE ensemble, governed by the structural scaling relation $m \propto 1.5^h$. We have thus empirically determined the existence of an individual variable influence phase transition for the $\mathcal{T}(h, \beta, \Sigma)$ ensemble.

It is again important to note that the exponential scaling of the variable pool size m and the logarithmically scaled axis have no bearing on the existence of the transition itself. It is precisely the exponential increase in system size (the 3^h growth) which imposes "matching" that increase with an exponentially scaling m. On top of that, we have seen that our finite-size scaling analysis resulted in a control parameter which universally enforces the same behavior upon all heights we have tested. This suggests that the transition is a universal property itself. Note however that this transition cannot be characterized in terms of a sharp threshold, contrary to the transition described for the random k-SAT ensemble. This is because the individual variable influence is not

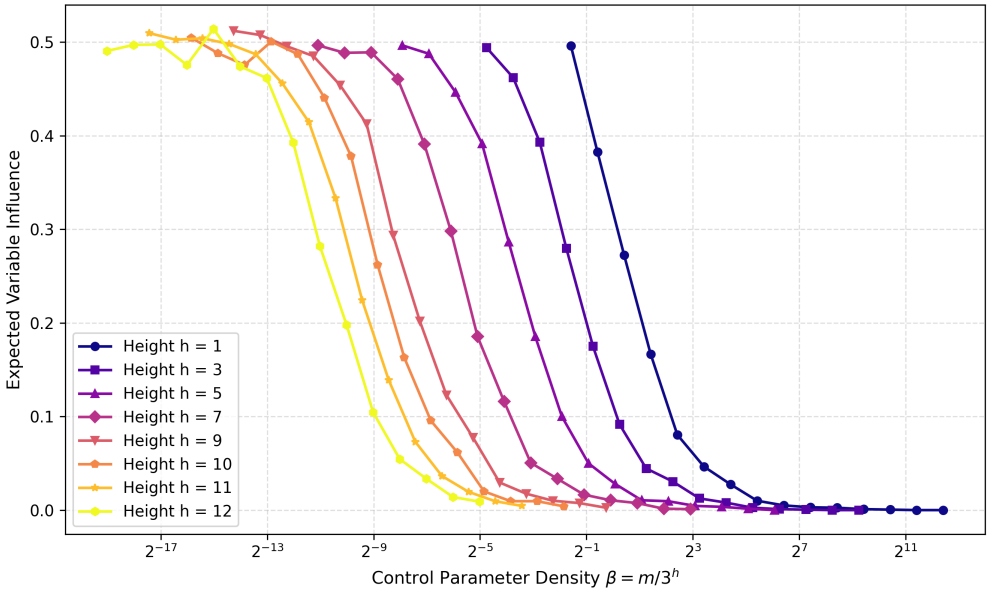
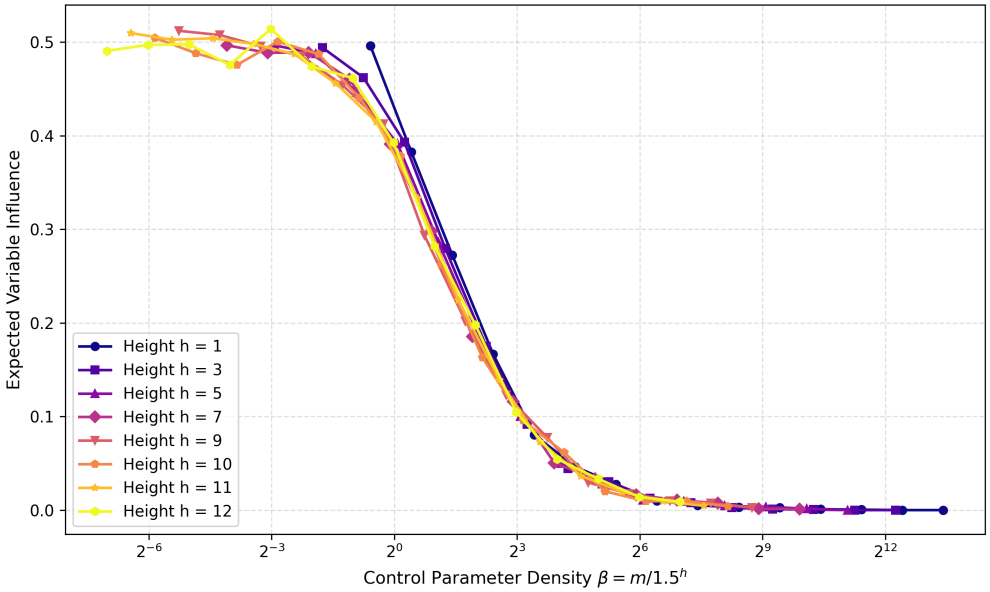


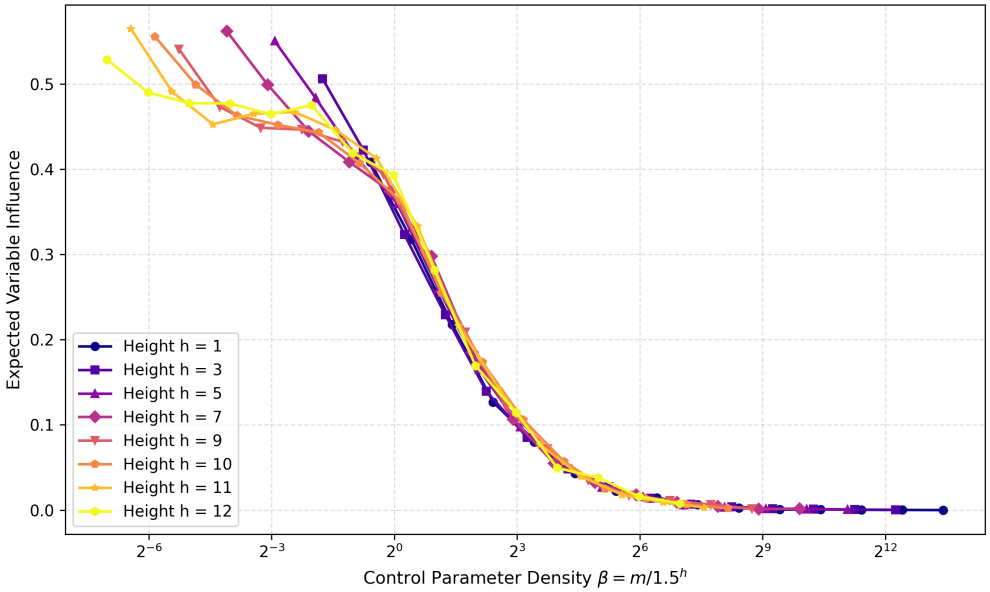
Figure 5.8: The variable influence of $\mathcal{T}(h, \beta, \Sigma)$ instances for $h \in \{1, 3, 5, 7, 9, 10, 11, 12\}$, $\beta = m/3^h$ and symmetrical alphabet Σ (allowing negations). Notice that normalizing our axis with the 3^h factor results in higher h curves drifting to the left. As h increases, the influence drops at continuously lower relative values of β, meaning that the 3^h factor overestimates the balance between adding variables to decay influence and increasing the tree size to grow variable dependence.

a boolean property, which is required under Friedgut's Theorem 2.4.5 describing phase transitions of CSPs.

We might also ask why is it the case that the universal scaling parameter for variable influence seems to be $\beta = m/1.5^h$. One possible explanation could be derived from the routing logic of the ITE operator itself. To evaluate $ITE(c, t, e)$ for a specific assignment, the condition branch c is evaluated, after which the signal is routed to either branch t or e. The unselected branch can therefore be thought of as completely "killed". This is true for any assignment and can be extended to the entire tree instance, resulting in a large portion of the tree to be muted in this manner, not being able to transmit any influence upward. Therefore, the effective number of active paths does not triple at each new layer. The 1.5^h factor seems to capture this effective branching, suggesting that the influence of the tree is determined by this logical routing capacity, rather than its structural size.



(a)



(b)

Figure 5.9: **Top (a):** The individual variable influence of $\mathcal{T}(h, \beta, \Sigma)$ instances for $h \in \{1, 3, 5, 7, 9, 10, 11, 12\}$, $\beta = m/1.5^h$ and symmetrical alphabet Σ (allowing negations). Under this control parameter scaling, all tested curves align and seem to present a phase transition of variable influence. At low m, we observe a region of influence ≈ 0.5, which drops to ≈ 0 as m increases exponentially. **Bottom (b):** the same variable influence measurement done for the restricted alphabet, where we do not allow negations. Under the exact same parameter, the curves align as well, and we see the starting points of the curves at ≈ 0.55, after which a decrease to a plateau at ≈ 0.47 is seen. The drop to zero individual influence seems to behave in the same manner as in the top plot (a).

Chapter 6

Discussion

In this chapter, we outline our general results and relate them to existing research. Additionally, we discuss limitations of our methodology and possible directions for future research.

6.1 Related Work

The study of phase transitions in CSPs was fundamentally shaped by the discovery of sharp satisfiability thresholds in random K-SAT by Mitchell, Selman, and Levesque [3], and the application of finite-size scaling to these models by Kirkpatrick et al. [38]. The pursuit of analytically proving the critical threshold phenomenon covers equally groundbreaking research, leading to Friedgut's seminal paper in 1999 [7] which rigorously proved that a satisfiability phase transition exists for random K-SAT. Later work by Achlioptas et al. [8, 9, 29] fundamentally advanced the field by providing a theoretical basis on which many bounds on the threshold have been proven (we extensively discussed one such method in Chapter 3).

6.2 Interpretation of findings

Our work resulted in two primary conclusions on the two CSP ensembles we have investigated.

Firstly, we examined the $F_k(n, m)$ ensemble, both analytically (through our discussion on the well-known methodology presented by Achlioptas et al. [8]) and empirically. Our experimental results on the sharp thresholds of random K-SAT with $k = 2$ and $k = 3$ corroborate both the existing empirical evidence and theoretical confirmation of the phenomenon. Additionally, our experiment on the solution space of this ensemble provided an intuitive visualization of its shattering at the critical clustering threshold. Our 2-dimensional mapping is consistent with the predicted state of the solution space at different clause densities.

Secondly, our empirical investigation into the random ITE ensemble provided us with evidence for a continuous, size-invariant phase transition in individual variable influence. We proposed that our $\mathcal{T}(h, \beta, \Sigma)$ model's ITE operators essentially preserve contingency,

and that their routing logic results in the system to not scale proportionally to its physical leaf count (3^h). Through finite-size scaling, we empirically identified this structural scaling parameter to be proportional to 1.5^h. When scaled in this manner, disparate tree heights collapse onto a single curve. Furthermore, testing a restricted alphabet resulted in the interesting plateau behavior which maintains the same shape as the variable pool increases.

6.3 Limitations

Our experiments on the solution space of random κ-SAT align well with the statistical physics predictions about the transition to a shattering phase, however, the implication of this shattering is precisely the fact that currently used algorithms (such as CDCL and local searches) cannot efficiently navigate this clustered space. For this reason, our two-tiered SLS algorithm could not map the space for higher clause densities at any reliable rate. This is a profound limitation regarding the visualization of the solution space.

Regarding our random ITE tree experiments, although our empirical data supports the existence of a phase transition, our methodology is constrained by computational limits. Due to the exponential growth of the tree structure, our experiments were restricted to heights $h \leq 13$ (which result in tree sizes on the order of 3^13). Evaluating the properties of tree instances in our model with millions of leaves remains computationally expensive.

Furthermore, while our finite-size scaling analysis provides empirical evidence for the control parameter $\beta = m/1.5^h$, this result remains a conjecture. We have not provided a rigorous proof deriving the 1.5^h exponent from the theory of our model.

6.4 Further Work

As mentioned above, our work on the random ITE tree ensemble is restricted to empirical results. Although we can make educated conclusions based on our experiments, we are computationally limited to only reasoning about the finite-size behavior of the system. To confirm the asymptotic tendency of the ensemble, a theoretical proof is required, which we propose as future research.

Another possible direction of research has to do with examining how (or if) the local and global properties of the two ensembles we studied are preserved under a structural combination of the two. Specifically, an interesting line of further research could investigate the behavior of a system whose instances preserve a certain type of locality by performing conjunctions on multiple global structures, e.g.:

$$ITE_1(h, \beta, \Sigma) \wedge ITE_2(h, \beta, \Sigma) \wedge \cdots \wedge ITE_k(h, \beta, \Sigma) \tag{6.1}$$

where we denoted $ITE_i(h, \beta, \Sigma)$ as a subtree instance of the $\mathcal{T}(h, \beta, \Sigma)$ ensemble. Investigating whether this "forest" of ITE trees preserves or exhibits new properties of our models (such as a satisfiability phase transition) could reveal deeper connections between different kinds of local and global constraints.

Chapter 7

Conclusion

This thesis explored the connection between phase transition phenomena of Constraint Satisfaction Problems and whether their constraints operate locally or globally, viewed through the lens of the random K-SAT and random ITE tree CSP ensembles. By examining both a well-known local CSP and a novel, global ensemble, we demonstrated how macroscopic phenomena emerge from the microscopic structure of these two ensembles.

In the first part of this work, we provided a comprehensive exposition of the random K-SAT model. By reconstructing the rigorous bounds derived via the first and second moment methods, we intuitively arrived at the analytical methodology required to bound the sharp satisfiability threshold of the random K-SAT problem. Complementing the theory, our empirical visualizations of both the phase transition and the solution space provided deeper intuition for the clustering behavior that make these problems algorithmically hard for modern Stochastic Local Search (SLS) and resolution-based algorithms.

In the second part, we applied a similar macroscopic analysis to a novel formulation of a random If-Then-Else (ITE) tree ensemble. By treating this model as a parameterized global CSP, we have provided evidence for a continuous phase transition in individual variable influence. Our finite-size scaling analysis further gave insight into the universality of this threshold, indicating that the required variable pool scales proportionally to 1.5^h, a significant reduction from the ensemble's structural size of 3^h.

Finally, this thesis demonstrates the versatility of statistical physics techniques when applied across different computational architectures. By subjecting both a local CSP and a global CSP to the same macroscopic experimental framework, we presented how the nature of the constraint dictates the resulting phase transition. We discovered that our global model exhibits a different behavior, resulting instead in a continuous transition in variable influence. This comparative approach establishes an empirical framework for the ITE ensemble, and paves the way for theoretical methodologies to investigate the model rigorously.

Appendix A

Supplementary Materials

A.1 Source Code

The complete Python code, generators and (simulation) data used in this thesis are available on GitHub at `https://github.com/TreelonMusk/SAT-phase-transition.git`.

A.2 Additional Figures

The figures in the following pages provide further detail on specific empirical results found in Chapter 4.

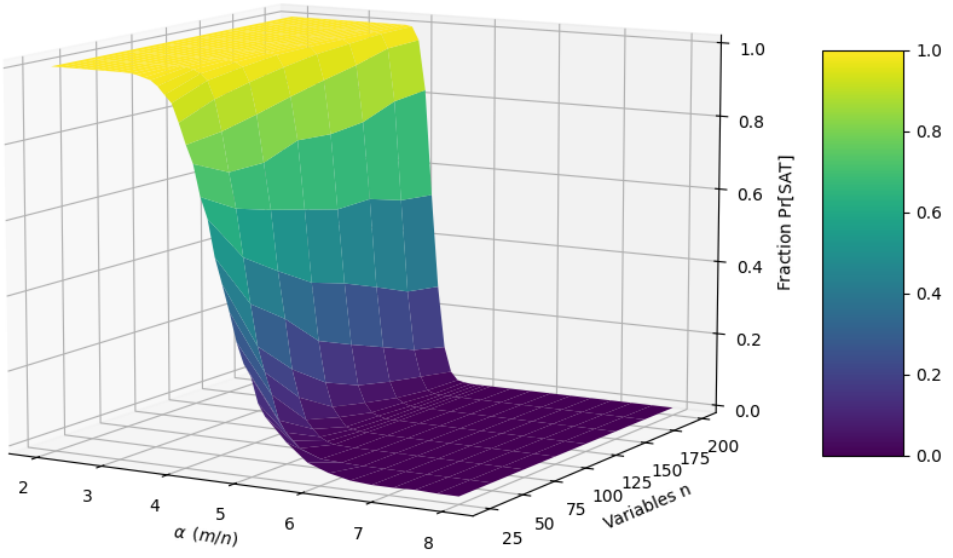


Figure A.1: The estimated probability of satisfiability Pr[SAT], for increasing system sizes $n \in \{25, 50, \ldots, 200\}$ and plotted in three dimensions. Notice the increasing steepness of the transition window as the system size increases.

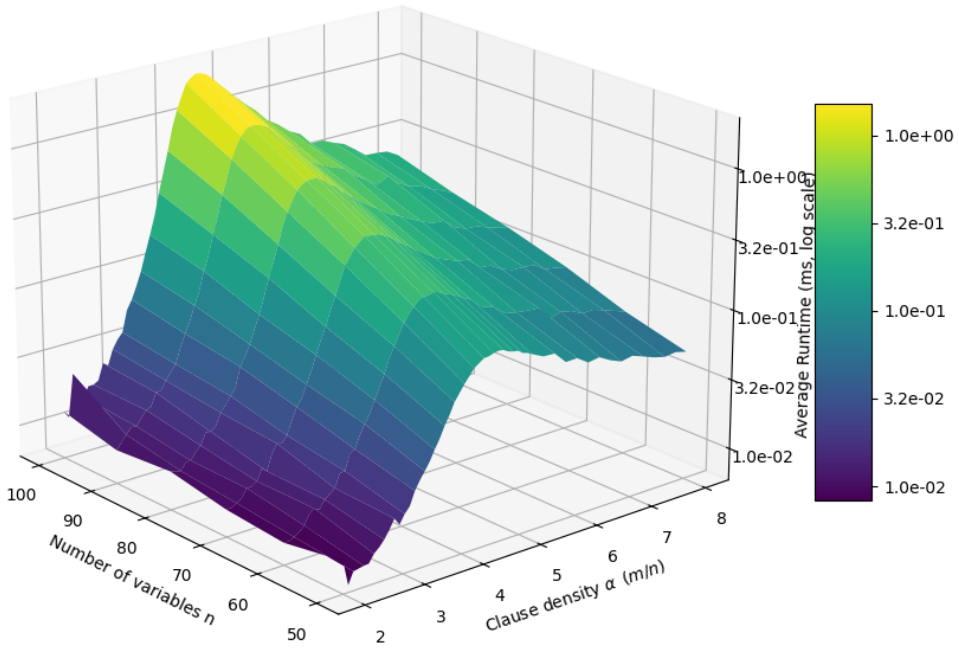


Figure A.2: The average solver runtime from Figure 4.2 plotted in three dimensions (with the system size n increasing to the left). Notice that the peaks in each runtime curve linearly increases. Exactly the same behavior is observed for the far left region of each curve, where we see a clear (almost) straight line formed by the points of each curve. This entails an exponential increase of the over-constrained region.

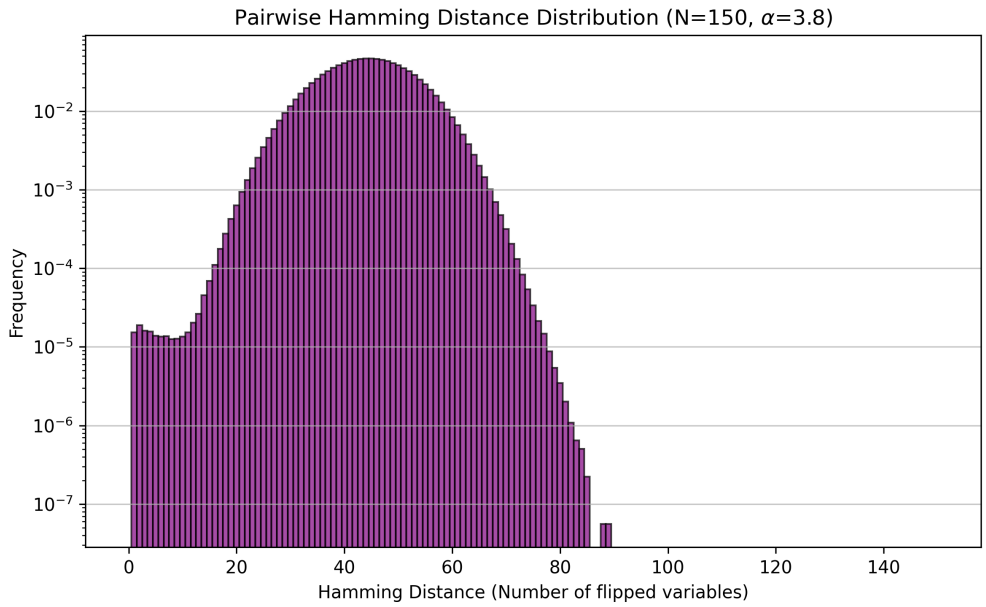


Figure A.3: Log-scaled distribution of Hamming distances for $\alpha = 3.8$ and $n = 150$. We can clearly see the increasing number of pairs with near-zero distances to the left of the distribution peak.

Bibliography

[1] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Prentice Hall Press, USA, 3rd edition, 2009.

[2] Manuel Bodirsky and Martin Grohe. Non-dichotomies in constraint satisfaction complexity. In Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfsdóttir, and Igor Walukiewicz, editors, *Automata, Languages and Programming*, pages 184–196, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg.

[3] David Mitchell, Bart Selman, and Hector Levesque. Hard and easy distributions of SAT problems. In *Proceedings of the Tenth National Conference on Artificial Intelligence*, AAAI'92, page 459–465. AAAI Press, 1992.

[4] Alfonso San Miguel Aguirre and Moshe Y. Vardi. Random 3-SAT and BDDs: The plot thickens further. In *Proceedings of the 7th International Conference on Principles and Practice of Constraint Programming*, CP '01, page 121–136, Berlin, Heidelberg, 2001. Springer-Verlag.

[5] Peter Cheeseman, Bob Kanefsky, and William M. Taylor. Where the really hard problems are. In *Proceedings of the 12th International Joint Conference on Artificial Intelligence - Volume 1*, IJCAI'91, page 331–337, San Francisco, CA, USA, 1991. Morgan Kaufmann Publishers Inc.

[6] Rémi Monasson, Riccardo Zecchina, Scott Kirkpatrick, and Lidror Troyansky. Determining computational complexity from characteristic 'phase transitions'. *Nature*, 400:133–137, 07 1999.

[7] Ehud Friedgut and Jean Bourgain. Sharp thresholds of graph properties, and the k-SAT problem. *Journal of the American Mathematical Society*, 12(4):1017–1054, 1999.

[8] Dimitris Achlioptas and Cristopher Moore. Random k-SAT: Two moments suffice to cross a sharp threshold. *SIAM Journal on Computing*, 36(3):740–762, 2006.

[9] Dimitris Achlioptas and Yuval Peres. The threshold for random k-SAT is $2^k \ln(2) - \mathrm{O(k)}$. *Journal of the American Mathematical Society*, 17:223–231, 01 2003.

[10] Ronald L. Graham, Donald Ervin Knuth, and Or Patashnik. *Concrete mathematics - a foundation for computer science*. Addison-Wesley, 1991.

[11] Sally C. Brailsford, Chris N. Potts, and Barbara M. Smith. Constraint satisfaction problems: Algorithms and applications. *European Journal of Operational Research*, 119(3):557–581, 1999.

[12] Toms Feder and Moshe Vardi. The computational structure of monotone monadic snp and constraint satisfaction: A study through datalog and group theory. *SIAM Journal on Computing*, 28:57–104, 07 2006.

[13] Andrei Bulatov. A dichotomy theorem for nonuniform CSPs. In *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 319–330. IEEE, 10 2017.

[14] Andrei A. Bulatov and Dániel Marx. Constraint satisfaction problems and global cardinality constraints. *Commun. ACM*, 53(9):99–106, September 2010.

[15] Willem-Jan van Hoeve and Irit Katriel. Chapter 6 - global constraints. In Francesca Rossi, Peter van Beek, and Toby Walsh, editors, *Handbook of Constraint Programming*, volume 2 of *Foundations of Artificial Intelligence*, pages 169–208. Elsevier, 2006.

[16] Sarah R. Allen, Ryan ODonnell, and David Witmer. How to refute a random CSP. In *Proceedings of the 2015 IEEE 56th Annual Symposium on Foundations of Computer Science (FOCS)*, FOCS '15, page 689–708, USA, 2015. IEEE Computer Society.

[17] David Tong. *Statistical physics*. University of Cambridge, 2011.

[18] Vašek Chvátal and Bruce Reed. Mick gets some (the odds are on his side)(satisfiability). In *Proceedings., 33rd Annual Symposium on Foundations of Computer Science*, pages 620–627. IEEE, 1992.

[19] Andreas Goerdt. A threshold for unsatisfiability. *Journal of Computer and System Sciences*, 53(3):469–486, 1996.

[20] Jian Ding, Allan Sly, and Nike Sun. Proof of the satisfiability conjecture for large k. In *Proceedings of the Forty-Seventh Annual ACM Symposium on Theory of Computing*, STOC '15, pages 59–68, New York, NY, USA, 2015. Association for Computing Machinery.

[21] M. Mézard, G. Parisi, and R. Zecchina. Analytic and algorithmic solution of random satisfiability problems. *Science*, 297(5582):812–815, 2002.

[22] Ryan O'Donnell. *Analysis of Boolean Functions*. Cambridge University Press, 2014.

[23] Giulio Biroli, Simona Cocco, and Rémi Monasson. Phase transitions and complexity in computer science: An overview of the statistical physics approach to the random satisfiability problem. *Physica A: Statistical Mechanics and its Applications*, 306:381–394, 04 2002.

[24] Marc Mezard, Thierry Mora, and Riccardo Zecchina. Clustering of solutions in the random satisfiability problem. *Physical review letters*, 94:197205, 05 2005.

[25] Wojciech Szpankowski. *Average Case Analysis of Algorithms on Sequences*. John Wiley & Sons, Inc., USA, 2001.

[26] Dimitris Achlioptas. Random satisfiabiliy. In Armin Biere, Marijn Heule, Hans van Maaren, and Toby Walsh, editors, *Handbook of Satisfiability - Second Edition*, Frontiers in Artificial Intelligence and Applications, pages 437–462. IOS Press, 2021.

[27] Rémi Monasson and Riccardo Zecchina. Statistical mechanics of the random k-satisfiability model. *Phys. Rev. E*, 56:1357–1370, Aug 1997.

[28] N.G. de Bruijn. *Asymptotic Methods in Analysis*. Bibliotheca mathematica. Dover Publications, 1981.

[29] Dimitris Achlioptas and Amin Coja-Oghlan. Algorithmic barriers from phase transitions. In *2008 49th Annual IEEE Symposium on Foundations of Computer Science*, pages 793–802, 2008.

[30] Alexey Ignatiev, Antonio Morgado, and Joao Marques-Silva. PySAT: A Python toolkit for prototyping with SAT oracles. In *Theory and Applications of Satisfiability Testing - SAT 2018*, pages 428–437. Springer, 2018.

[31] Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt. CaDiCaL 2.0. In Arie Gurfinkel and Vijay Ganesh, editors, *Computer Aided Verification - 36th International Conference, CAV 2024, Montreal, QC, Canada, July 24-27, 2024, Proceedings, Part I*, volume 14681 of *Lecture Notes in Computer Science*, pages 133–152. Springer, 2024.

[32] João Marques-Silva, Inês Lynce, and Sharad Malik. Conflict-driven clause learning SAT solvers. In Armin Biere, Marijn Heule, Hans van Maaren, and Toby Walsh, editors, *Handbook of Satisfiability - Second Edition*, Frontiers in Artificial Intelligence and Applications, pages 133–182. IOS Press, 2021.

[33] Bart Selman, Hector Levesque, and David Mitchell. A new method for solving hard satisfiability problems. In *Proceedings of the Tenth National Conference on Artificial Intelligence*, AAAI'92, page 440–446. AAAI Press, 1992.

[34] M. R. Krom. The decision problem for a class of first-order formulas in which all disjunctions are binary. *Mathematical Logic Quarterly*, 13(1-2):15–20, 1967.

[35] Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, STOC '71, page 151–158, New York, NY, USA, 1971. Association for Computing Machinery.

[36] Stephan Mertens, Marc Mézard, and Riccardo Zecchina. Threshold values of random k-SAT from the cavity method. *Random Struct. Algorithms*, 28(3):340–373, May 2006.

[37] Christian P. Robert and George Casella. *Monte Carlo Statistical Methods (Springer Texts in Statistics)*. Springer-Verlag, Berlin, Heidelberg, 2005.

[38] Scott Kirkpatrick. Critical behavior in the satisfiability of random boolean expressions. *Science (New York, N.Y.)*, 264:1297–301, 06 1994.

[39] Damien Parret. Master's thesis. Master's thesis, University of Neuchâtel, 08 2016.

[40] Giulio Biroli, Rémi Monasson, and Martin Weigt. A variational description of the ground state structure in random satisfiability problems. *CoRR*, cond-mat/9907343, 1999.

[41] Shaowei Cai and Xindi Zhang. Deep cooperation of cdcl and local search for SAT (extended abstract). In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI-22)*, pages 5240–5244, 07 2022.

[42] Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction, 2020.

[43] Steven D. Prestwich. CNF encodings. In Armin Biere, Marijn Heule, Hans van Maaren, and Toby Walsh, editors, *Handbook of Satisfiability - Second Edition*, Frontiers in Artificial Intelligence and Applications, pages 75–100. IOS Press, 2021.

[44] G. S. Tseitin. On the complexity of derivation in propositional calculus. In *Automation of Reasoning: 3. Classical Papers on Computational Logic 1967-1970*. Springer, 1983.

[45] Niklas Een and Niklas Sörensson. Translating pseudo-boolean constraints into SAT. *JSAT*, 2:1–26, 03 2006.